

ARQUITECTURA DE COMPUTADORAS

NICOLÁS ALEGRE

2019

- 1.1 Conceptos Introductorios
- 1.2 Organización general de una computadora
- 1.3 Hardware y Software
- 1.4 Computadores digitales y sistemas digitales
- 1.5 Generaciones de computadoras
- 1.6 Representación de los números en las maquinas
- 1.7 Sistemas de Numeración

2. SISTEMAS DE NUMERACIÓN. CODIFICACIÓN.

3. CIRCUITOS COMBINACIONALES

- 3.1 Algebra de Boole
 - 3.1.1 Elementos del Algebra de Boole
 - 3.1.2 Tabla de verdad
 - 3.1.3 Operadores lógicos
 - 3.1.4 Forma normal o canónica de una función
 - 3.1.5 Simplificación de funciones de Boole
- 3.2 Circuitos combinacionales

4. CIRCUITOS SECUENCIALES

- 4.1 Circuitos Secuenciales
 - 4.1.1 Tablas de Estados
 - 4.1.2 Ecuaciones de Estados
 - 4.1.3 Diagramas de Estados
 - 4.1.4 Tablas de excitación
 - 4.1.5 Tablas de funcionamiento
- 4.2 Biestables
- 4.3 Registros

5. MEMORIA

- 5.1 Definición de memoria. Esquema.
- 5.2 Punto de memoria
- 5.3 Características de memoria
- 5.4 Clasificación tecnológica
- 5.5 Tipos de memoria
- 5.6 Jerarquía de memoria
- 5.7 Organización de memoria (2D, 3D y 2 ½ D)
- 5.8 Memorias de núcleos.
- 5.9 Organización y búsqueda de la información en memoria.
- 5.10 Decodificador
- 5.11 Buses

6. ALU

- 6.1 Definición – Formato de instrucciones
- 6.2 Clasificación de los operadores aritméticos
- 6.3 Operadores combinacionales y con acumulador
- 6.4 Operaciones lógicas
- 6.5 Operaciones de desplazamiento
- 6.6 Adición y Sustracción binarias
 - 6.6.1 Sumador binario paralelo (Sumador en serie está en C.C.)
 - 6.6.2 Adición y sustracción de números algebraicos binarios
- 6.7 Elemento ALU para Abacus
- 6.8 Multiplicación y división binarias
 - 6.8.1 Multiplicador secuencial por suma/desplazamiento
 - 6.8.2 Técnicas de multiplicación rápida
 - 6.8.3 División por sustracción y desplazamiento
 - 6.8.4 División con restauración
 - 6.8.5 División sin restauración
- 6.9 Operaciones decimales: contador decimal y sumador decimal en paralelo.
- 6.10 Aritmética binaria en coma flotante
 - 6.10.1 Estándar IEEE
 - 6.10.2 Suma y sustracción en coma flotante
 - 6.10.3 Funcionamiento de un operador de suma flotante
 - 6.10.4 Multiplicación y división en suma flotante

7. UC

- 7.1 Definición. Esquema
- 7.2 Desarrollo de una instrucción de procesamiento
- 7.3 Instrucción de ruptura de secuencia
- 7.4 Descripción esquemática de las transferencias de informaciones
- 7.5 Descripción de UC de Abacus
- 7.6 Instrucciones básicas en Abacus: suma, almacenamiento y salto incondicional
- 7.7 Organización y componentes de la ruta de datos
- 7.8 Encaminamiento de los operandos y utilización de las unidades funcionales
- 7.9 Direccionamiento

8. EL SECUENCIAMIENTO DE LAS INSTRUCCIONES

- 8.1 Conceptos: Secuenciador, microórden, microprograma, ruta de datos
- 8.2 Entorno del Secuenciador: entradas y salidas
- 8.3 Secuenciador Síncrono y Asíncrono
- 8.4 Secuenciador Cableado
 - 8.4.1 Principio del Secuenciamiento
 - 8.4.2 Distribuidor de fases

8.4.3 Decodificador de Instrucciones

8.4.4 Biestables de Estados

8.4.5 Trazado de Cronogramas

8.4.6 Ecuaciones Lógicas

8.4.7 Ejemplos Cronogramas

8.5 Secuenciador Microprogramado

8.5.1 Definiciones

8.5.2 Modelo de Wilkes

8.5.3 Memoria de Control – Wilkes

8.5.4 Codificación de las Microinstrucciones

8.5.5 Direccionamiento de las Microinstrucciones

8.5.6 Acompasamiento

9. ARQUITECTURAS VON NEUMMAN – HARVARD

9.1 Arquitectura Von Neumann

9.2 Arquitectura Harvard

10. CISC – RISC

10.1 Arquitectura CISC

10.2 Arquitectura RISC

10.3 Comparación entre ambas

11. MICROCONTROLADORES

12. LENGUAJE DE MÁQUINA

12.1 Lenguaje de Maquina

12.2 Lenguaje Ensamblador

12.3 Lenguaje de alto nivel

12.4 Compilador e Intérprete

13. INTERCAMBIOS DE INFORMACIÓN CON EL EXTERIOR

13.1 Introducción

13.2 Simultaneidad de los procesos entrada/salida

13.3 Noción de interfase

13.4 Noción de multiplaje

13.5 Líneas ómnibus

13.6 Concentración y fraccionamiento de la información

13.7 Técnicas de ejecución de una transferencia elemental

14. PERIFERICOS

- **Aclaración**: el texto que se encuentre entre comillas y cursiva indican que fueron comentarios del profesor.

Unidad 1

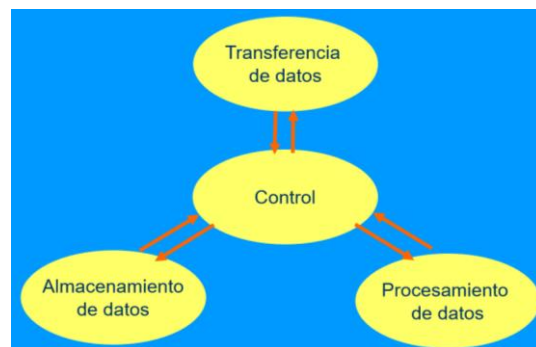
Introducción

1.1 CONCEPTOS INTRODUCTORIOS

. **Computadora:** Es una maquina digital electrónica programable para el tratamiento automático de la información, capaz de recibirla, operar sobre ella mediante procesos determinados y suministrar los resultados de tales operaciones. Resuelve problemas matemáticos y lógicos.

Las funciones generales que una computadora puede llevar a cabo son:

1. Procesamiento de datos
2. Almacenamiento de datos
3. Transferencia de datos
4. Control



. **Sistema:** Conjunto de *componentes* coordinados que trabajan juntos para lograr objetivos comunes. Es un conjunto de componentes/elementos dinámicamente *relacionados* entre sí operando sobre entradas (datos, energía, información) y proveyendo salidas (datos, energía, información). Las entradas y salidas son la relación de su existencia y lo determinan en un ámbito dado. Los sistemas pueden dividirse en subsistemas.

. **Dato:** Representación simbólica de propiedades de entes o sucesos. Son realidades cuantitativas o cualitativas determinadas en una situación. Los datos tienen la propiedad que se pueden transmitir, almacenar y transformar.

. **Información:** Representación simbólica de entes, hechos, sucesos, etc. que por el significado que le atribuye quien la recibe e interpreta, permite disminuir la incertidumbre en una decisión, lo que no ocurre con los datos. La información sirve para la toma de decisiones, que se obtiene procesando datos.

La **información descriptiva** es aquella que permite concretar la acción determinando los pasos a realizar y la secuencia a seguir. En las maquinas esta información se encuentra en la memoria constituyendo programas que indican la secuencia de operaciones a seguir para lograr el resultado deseado.

Además, existe la **información de control** para verificar si las acciones planificadas fueron realizadas correctamente.

La información también se puede transmitir, almacenar y transformar.

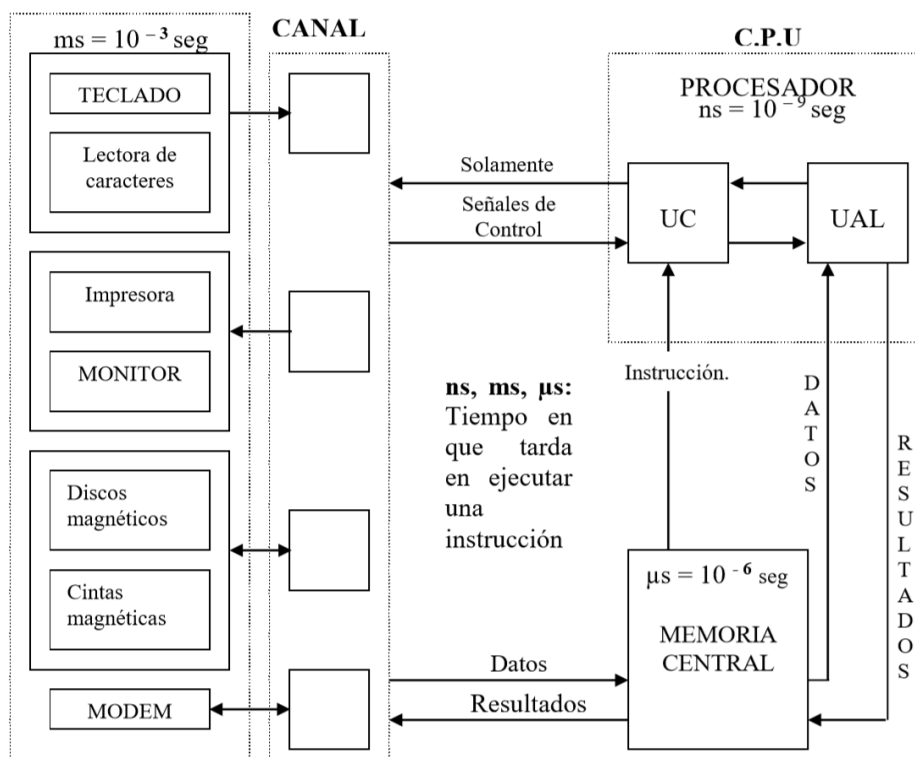
. **Estructura de Datos:** Es un conjunto de datos o símbolos pero organizado respondiendo a relaciones que no existen antes de procesarlos.

. **Proceso:** Conjunto finito de pasos u operaciones que deben llevarse a cabo en un cierto orden, relacionando los datos para obtener la información necesaria para resolver un problema.

Existen dos tipos:

- Procesos Secuenciales: operaciones totalmente ordenadas en el tiempo que se realizan una por vez. Por ejemplo, maquinas con arquitectura Von Neumann, no comienzan una operación hasta haber terminado la anterior.
- Procesos Concurrentes: Hay operaciones que mientras se ejecutan se superponen en el tiempo. Por ejemplo, procesadores actuales, siempre y cuando no utilicen los mismos recursos.

1.2 ORGANIZACIÓN GENERAL DE UNA COMPUTADORA



. **Periféricos:** todo aquello que rodea al procesador central.

- Entrada: teclado, mouse, reconocedor de voz, lápiz óptico.
- Salida: impresora, monitor, sintetizador.
- Entrada-Salida: memorias auxiliares (disco rígido, disquete), modem, etc.

Los periféricos de entrada proporcionan al sistema los datos provenientes del exterior. La función es detectar la existencia de señales que representan símbolos y convertir esa señal eléctrica detectada en señales eléctricas internas binarias. Tiene un conversor externo/interno. Los periféricos de salida envían al exterior los resultados almacenados en memoria. El periférico debe convertir en señales adecuadas las señales eléctricas binarias internas provenientes de la memoria para el desplazamiento, visualización y transmisión. Tiene un conversor interno/externo.

. **Memoria Principal o Central:** almacena instrucciones o datos provenientes desde los periféricos y guarda resultados intermedios y finales. En general, un periférico no puede pasar información a otro sin que intervenga la memoria central.

. **Procesador Central (CPU):** Lugar donde se ejecutan las instrucciones contenidas en la Memoria Central. Si el procesador central está constituido por un único circuito integrado se denomina microprocesador.

El procesador está compuesto por: unidad de control (UC), unidad aritmética lógica (ALU) y registros auxiliares.

. **Canales:** Elemento que actúa como interfaz. Son dispositivos intermedios entre los periféricos y la memoria. Su función es adaptar las diferencias de velocidad de funcionamiento existente entre la CPU y los periféricos.

- **Memoria Buffer:** utilizan algunos periféricos para adaptarse a la velocidad de la CPU e igualmente los canales, dados a que algunos periféricos y canales no trabajan a la misma velocidad que el procesador necesitan de las memorias buffer.

1.3 HARDWARE Y SOFTWARE

. **Hardware:** Es la totalidad física, conformada por todos los componentes de su equipamiento: circuitos electrónicos, cables, discos, etc. Constituye la porción material o “dura” que no cambia, permanece con cada proceso particular de datos. Todo aquello que soporta al software.

. **Software:** Se refiere a todos los programas que se pueden ejecutar en un equipo de computación. Todo aquello que es fácilmente modificable.

. **Firmware:** Es posible incluir programas y datos en una parte de la memoria principal que tenga almacenamiento permanente. Esos circuitos integrados constituyen las denominadas memorias ROM. Los programas y datos no pueden cambiarse, solo leerlos.

Los programas incluidos en firmware son de uso muy frecuente, por ejemplo, iniciadores de funcionamiento de máquina, traductores, etc. Estos son grabados electrónicamente.

1.4 COMPUTADORES DIGITALES Y SISTEMAS DIGITALES

La propiedad más importante de un computador es su generalidad. Puede seguir una serie de instrucciones, llamadas programa, que operan con datos dados. El usuario puede determinar y cambiar los programas y datos de acuerdo a una necesidad específica. Como resultado de esta flexibilidad, las computadoras digitales de uso general pueden realizar una serie de tareas de procesamiento de información de amplia variedad.

El computador digital de uso general es el ejemplo más conocido de sistema digital. La característica de un sistema digital es la manipulación de elementos discretos de información. Tales elementos discretos pueden ser impulsos eléctricos, dígitos decimales, letras de un alfabeto o cualquier otro conjunto de símbolos significativos. Por esto, un nombre más adecuado para un computador digital podría ser “sistema de procesamiento de información discreta”.

Los elementos discretos de información se representan en un sistema digital por cantidades físicas llamadas señales. Las señales eléctricas tales como voltajes y corriente son las más comunes. Las señales en los sistemas digitales electrónicos de la actualidad tienen solamente dos valores discretos y se les llama binarios.

En síntesis, un computador digital es una interconexión de módulos digitales que manipula elementos discretos de información y que estos elementos se presentan en forma binaria. Los operandos, usados en los cálculos pueden ser expresados en el sistema de números binarios. El procesamiento de datos se lleva a cabo por medio de los elementos lógicos binarios, usando señales binarias.

1.5 GENERACIONES DE COMPUTADORAS

Generación	Años	Características	Resuelven
0	Hasta 1940	Sistemas mecánicos y electromagnéticos	
1	1940 – 1950	Válvulas de vacío, tableros	1 sola tarea
2	1950 – 1965	Transistores y sistemas por lotes	Varias Tareas
3	1965 – 1980	Circuitos integrados y multiprogramación	Mainframes
4	Desde 1980	Microprocesadores, computadores personales	Intel

De una a otra generación se han conseguido progresos con respecto a la miniaturización, fiabilidad, complejidad y velocidad en los circuitos:

- Miniaturización: hace referencia a la gran disminución del tamaño físico. Si en la primera generación se utilizaba un armario de válvulas integradas, en la cuarta un circuito integrado del tamaño de una capsula.
- Fiabilidad: la calidad del funcionamiento de un componente o de un conjunto. Se mide considerando el promedio de tiempos entre averías.
- Complejidad: posibilidad de concebir conjuntos electrónicos cada vez más complejos y sin perder fiabilidad.
- Velocidad: tiempos de conmutación de los circuitos lógicos.

Generación 0: Sistemas mecánicos y electromecánicos

Calculadoras mecánicas, sistemas basados en relés.

- Difference engine (1822): computadora digital mecánica utilizada para calcular tablas numéricas, calculaba cualquier función algebraica y almacenaba números. Se programaba con tarjetas.
- Harvard Mark 1 (1930): usada para computar tablas matemáticas y de navegación.

Generación 1: Tubos de Vacío

El ordenador ejecutaba sus trabajos de manera totalmente secuencial. Un solo grupo diseñaba, construía, programaba, operaba y mantenía cada máquina. Toda la programación se hacía en lenguaje de máquina. No existían los sistemas operativos.

- ENIAC (1946): El proyecto fue una respuesta a necesidades militares durante la segunda guerra mundial. La máquina era enorme, pesaba 30 toneladas, ocupaba 15000 pies cuadrados y contenía más de 18000 tubos de vacío. Cuando funcionaba consumía 140 Kw de potencia. Era capaz de efectuar 5000 sumas por segundo, más rápido que cualquier computador electromecánico. Era decimal. Se finalizó en 1946, demasiado tarde para la guerra, y su primer uso fue ayudar a determinar la viabilidad de la bomba de hidrogeno, lo que demostró su naturaleza de propósito general.
- Colossus (1945)
- UNIVAC (1946)
- Manchester Mark I (1948): primera máquina que funcionó con un programa almacenado

Generación 2: Transistores

El primer cambio en los computadores electrónicos vino con la sustitución de los tubos de vacío por transistores. El transistor es más pequeño, barato, disipa menos calor y puede ser usado de la misma forma que un tubo de vacío en la construcción de computadores. Mientras que un tubo de

vacío requiere cables, placas de metal, capsula de cristal y vacío, el transistor es un dispositivo de estado sólido, hecho con silicio.

En esta generación, también se introdujeron unidades lógicas y aritméticas y unidades de control más complejas, el uso de lenguajes de programación de alto nivel y se proporcionó un software del sistema con el computador.

Se diferencia entre diseñadores, constructores, programadores, operadores y personal de mantenimiento.

- Johniac (1954): funcionaba con tarjetas
- IBM 704 (1955): primera máquina comercial con hardware de punto flotante
- IBM 1401 (1959): buena para leer tarjetas, copiar cintas e imprimir resultados, pero mala para cálculos numéricos; se utilizaba con fines comerciales
- IBM 7094: buena para hacer cómputos se utilizaba con fines científicos

Generación 3: Circuitos Integrados

Se introducen los circuitos integrados, lo cual es una gran ventaja en el precio y desempeño de un computador.

Se introduce la multiprogramación, que permite que cuando el programa que utiliza la unidad central se detiene en espera de una operación de entrada, otro programa toma su lugar y esto evita los tiempos muertos en la unidad central, es decir, mejora el empleo del conjunto de recursos de un sistema informático.

Se introducen los discos duros y el tiempo compartido entre usuarios.

- IBM 360 (1964): software compatible con IBM 7094, 1401, etc.
- GE 625 (1965)
- Multics (1976)
- Apple 1 (1976), Apple 2 (1978), Microsoft (1978), Altair 8800 (1975)

Generación 4

Luego de la tercera generación no hay un acuerdo general con respecto a la clasificación.

En esta generación se considera los avances de la tecnología de los circuitos integrados. Con la introducción de la integración a gran escala LSI podrá haber más de 1000 componentes en un simple chip de circuito integrado, con la integración a muy grande escala VLSI se lograron más de 10000 componentes por chip y los VLSI actuales pueden contener más de 100000 componentes. Además la aplicación comercial de nuevos desarrollos resultó uno de los principales cambios de principio de los años 70 y aún permanecen.

1.6 REPRESENTACIÓN DE LOS NÚMEROS EN LA MÁQUINA

Señales Lógicas y Analógicas

El procedimiento de la información en un calculador eléctrico consiste en tratar, transferir y memorizar señales eléctricas. En los calculadores analógicos se utilizan señales de variación continua. En los calculadores digitales, la información procesada que es de tipo binario, está materializada por señales eléctricas de dos estados. La información digital se transmite mejor, es bastante fácil realizar sistemas con dos estados estables capaces de memorizar un bit. Por el contrario, memorizar el valor de una señal eléctrica es mucho más difícil.

Lógica de Nivel

Puede representarse la información digital elemental sobre una línea eléctrica, manteniéndose una tensión A para materializar el 1 lógico y una tensión B para el 0 lógico. En realidad no son valores precisos de la tensión, sino franjas alrededor de estos valores. Se llama “zona prohibida” a la zona intermedia, en ella, el valor lógico de la señal eléctrica es indeterminado.

Se dirá que se trabaja con *lógica positiva* cuando la tensión que representa el 1 lógico es superior a la tensión del 0 lógico, en caso contrario, *lógica negativa*.

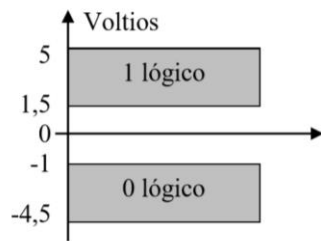


Figura 1

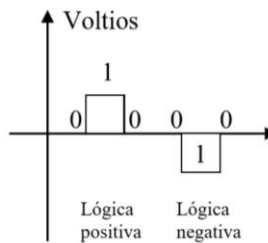


Figura 2

Se dice que se trabaja con una lógica de impulsos cuando la magnitud eléctrica representativa de la información dura un tiempo muy corto.

1.7 SISTEMAS DE NUMERACIÓN

En cualquier sistema de representación, un mismo conjunto de símbolos puede tener distintos significados según la convención que se siga. Este multisignificado también existe para las representaciones binarias numéricas que se dan dentro de las máquinas.

Dos consideraciones a tener en cuenta en la representación y operaciones de números en la máquina son:

- i. Números con precisión finita: los números solo pueden contar con una cantidad fija de dígitos
- ii. Modulo: es el número máximo de enteros diferentes que pueden representarse en un registro físico.

Características de un Sistema Numérico Posicional

1. Consta de un número finito de símbolos distintos, número que define la “base” o raíz de cada sistema
2. Cada símbolo aislado representa un número especificado de unidades
3. Existe un símbolo para indicar la ausencia de elementos a representar (cero)
4. Los símbolos pueden ordenarse en forma monótona creciente
5. Formando parte de un número compuesto por varios símbolos, un mismo símbolo tiene una significación o peso distinto según su posición
6. La posición extrema derecha corresponde a unidades (peso uno), a partir de ella, cada posición tiene el peso de la que está a su derecha multiplicada por la base

Naturaleza Posicional del Sistema Binario

La denominación “binario” o de “base dos” se refiere a los dos dígitos o símbolos (0 y 1) que combinados permiten simbolizar los números, conforme a una convención que atribuye un valor individual y otro posicional a cada símbolo.

Con los 2 símbolos que cuenta, 1 y 0, cuyo significado es el mismo que los respectivos símbolos decimales, se puede representar cualquier número. Cada uno de los símbolos que forman un número binario se llama *digito binario* o *bit*.

. **Base:** Determina la cantidad de símbolos distintos para representar los números, los símbolos son finitos pero los números no.

. **Complemento a la base:** Dado un numero N en la base m con parte entera de n dígitos, se define el complemento a m de n como $m^n - N$ para todo N distinto de cero y cero para $N=0$.

. **Complemento a la base -1:** Se define como el complemento a la base pero restado 1, es decir, se obtiene restando 1 a la base.

. **Tipo de datos:** Las computadoras digitales manejan 4 tipos básicos de datos:

1. Números enteros con signo (representación numérica)
2. Números reales con signo (representación numérica)
3. Números decimales codificados en BCD (representación numérica)
4. Caracteres (representación alfanumérica)

El número de bits que se usan para representar cada una de estos tipos de datos y el significado de cada uno varía según el tipo de dato a representar y el sistema computacional que se utilice (HW y SW).

Unidad 3

Circuitos Combinacionales

3.1 ALGEBRA DE BOOLE

El álgebra de Boole es una herramienta para representar información digital y proporciona un modelo matemático para determinar la respuesta de los circuitos lógicos, independientemente de los dispositivos que constituyen dichos circuitos.

Algunas operaciones del álgebra booleana en conmutación:

- Comparación de distintas realizaciones circuitales
- Minimizar el número de cables y dispositivos en los circuitos
- Codificación de información digital
- Herramienta auxiliar para analizar y sintetizar circuitos digitales

Circuitos Lógicos o de Conmutación

Es un circuito que trata señales lógicas.

- a. *Combinacional:* las señales de salida no dependen más que de las señales de entrada. Se basa en el álgebra de Boole.
- b. *Secuencial:* las señales de salida dependen de las señales de entrada y del tiempo. Poseen elementos de memoria. Se basa en la teoría de autómatas finitos: tiene una cantidad de estados determinados que no es infinito.

3.1.1 Elementos del Algebra de Boole

1. *Variables lógicas*: variables binarias que solo pueden tener un valor entre dos distintos.
2. *Funciones lógicas o Booleanas*: son funciones de n variables lógicas que toman valores en el conjunto $0,1$.

Existen 3 funciones fundamentales o básicas (se las denomina así debido a que toda función lógica puede formularse como una expresión compuesta por estas funciones, es decir, un grupo cerrado):

- a. Función Inversa o Complementación: función que permite invertir cualquier secuencia de ceros y unos.
- b. Función Intersección o producto lógico: operación que se materializa mediante la compuerta AND.
- c. Función Reunión o suma lógica: función que se materializa mediante la compuerta OR.

3.1.2 Tabla de Verdad

Es una tabla con todas las combinaciones posibles de las variables, que muestra la relación entre los valores que las variables pueden tomar y el resultado de la operación para un circuito digital determinado.

Cualquier función de Boole puede ser representada por una tabla de verdad. El número de filas en la tabla es 2^n donde n es el número de variables binarias de la función.

3.1.3 Operadores Lógicos

1. **Compuerta NOT**: entrega a la salida el complemento de la señal de entrada. Único operador con una sola entrada.

OR		$F = x + y$	x	y	F
			0	0	0
			0	1	1
			1	0	1
			1	1	1

2. **Compuerta AND**: designamos la salida de una compuerta AND como el producto lógico de las entradas. Es decir, la salida solamente será 1 si y solo si todas las entradas son 1.

Nombre	Símbolo gráfico	Función algebraica	Tabla de verdad															
AND		$F = xy$	<table><tr><th>x</th><th>y</th><th>F</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	x	y	F	0	0	0	0	1	0	1	0	0	1	1	1
x	y	F																
0	0	0																
0	1	0																
1	0	0																
1	1	1																

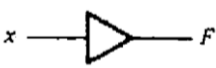
3. **Compuerta OR**: realizará la suma lógica de los valores de las entradas; la salida será 0 si y solo si todas las entradas son 0, caso contrario la salida es igual a 1.

OR		$F = x + y$	x	y	F
			0	0	0
			0	1	1
			1	0	1
			1	1	1

4. **Compuerta XOR (OR exclusivo):** similar a la compuerta OR pero excluye la combinación de ambos. La salida de la compuerta XOR es igual a 1 si una de las dos entradas es igual a 1 pero no ambas.

OR-exclusiva (XOR)		$F = xy' + x'y$ $= x \oplus y$	x	y	F
			0	0	0
			0	1	1
			1	0	1
			1	1	0

5. **Compuerta BUFFER o Separador:** un circuito separador produce la función de transferencia pero no produce ninguna operación lógica particular ya que el valor binario de la salida es igual al de la entrada. Este circuito se usa solamente para amplificación de señal y potencia y es equivalente a dos inversores conectados en cascada.

Buffer		$F = x$	x	F
			0	0
			1	1

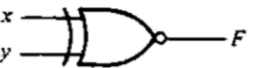
6. **Compuerta NAND:** surge de invertir la salida de una compuerta AND, por tanto, el resultado de la compuerta NAND será el complemento de su correspondiente de la compuerta AND.

NAND		$F = (xy)'$	x	y	F
			0	0	1
			0	1	1
			1	0	1
			1	1	0

7. **Compuerta NOR:** surge de invertir la salida de una compuerta OR, como consecuencia complementa cada resultado que esta genera, de modo de realizar una suma lógica negada.

NOR		$F = (x + y)'$	x	y	F
			0	0	1
			0	1	0
			1	0	0
			1	1	0

8. **Compuerta XOR-Negado:** surge de invertir la salida de una compuerta XOR, entonces, es igual a 1 cuando las dos variables son iguales. También se llama equivalencia.

NOR-exclusiva o equivalencia		$F = xy + x'y'$ $= x \odot y$	x	y	F
			0	0	1
			0	1	0
			1	0	0
			1	1	1

3.1.4 Forma Normal o Canónica de una Función

Se llama *forma normal o canónica* a aquella expresión donde cada término de la función contiene todas las variables en su forma normal o negada.

Cuando las variables se encuentran relacionadas por medio del operador AND se denomina **miniterminos**, si es por el operador OR **maxiterminos**. A las funciones de Boole expresadas como suma de miniterminos o producto de maxiterminos se dice que está en *forma canónica*.

3.1.5 Simplificaciones de Funciones de Boole

Un literal es una variable tildada o no tildada. El número de literales de una función de Boole puede ser minimizado por medio de manipulaciones algebraicas o algebra de Boole.

Existen dos métodos para simplificar funciones de Boole:

1. **Método Algebraico:** consiste en utilizar los postulados, teoremas básicos del algebra de Boole y cualquier otro método de manipulación que sean familiares con el uso. El problema de este método es que el procedimiento de minimización carece de reglas específicas para predecir cada paso sucesivo en el proceso de manipulación.
2. **Diagramas de VEITH – Mapas de Karnaugh:** constituye una forma de representar una función, siendo en esencia una tabla de verdad en dos dimensiones. Por la disposición de las variables en el diagrama, es posible hallar rápidamente una expresión sencilla de la función representada, mediante un adecuado agrupamiento de “unos” o bien de los “ceros” contenidos en el mismo.

El mapa de Karnaugh representa un procedimiento más simple, directo y menos engorroso, que el método algebraico, para minimizar las funciones de Boole. Se limita a funciones de hasta 5 variables como máximo.

El propósito de la simplificación de funciones de Boole es obtener una expresión algebraica que cuando se configure resulte en un circuito de bajo costo. Dado dos circuitos de bajo costo aquel que utilice menos compuertas es preferible debido a que cuesta menos.

Una suma de miniterminos será *mínima* si no existe otra suma de miniterminos con menor número de sumandos u otra con igual número de sumandos pero con menor cantidad de literales. Puede haber dos o más suma de productos igualmente mínimas.

Las restricciones, limitaciones y criterios para seleccionar una expresión algebraica particular son:

- Número mínimo de compuertas
- Número mínimo de entradas a las compuertas
- Tiempo de propagación mínima de una señal a través del circuito
- Número mínimo de interconexiones
- Limitaciones de la capacidad de accionamiento de cada compuerta

Con los circuitos integrados, no es la cantidad de compuertas lo que determina el costo, sino el número y el tipo de circuito integrado usado y el número de interconexiones externas necesarias para ejecutar una función dada.

3.2 CIRCUITOS COMBINACIONALES

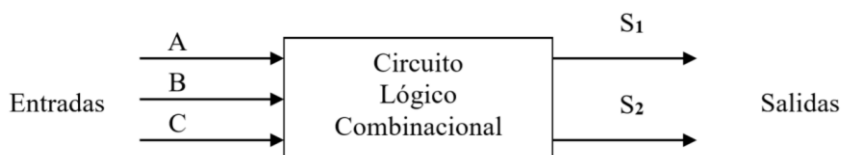
Un circuito combinacional está compuesto por compuertas lógicas cuyas salidas se determinan teniendo en cuenta la combinación presente de las entradas y sin considerar las previas. Un circuito combinacional realiza una operación de procesamiento de información específica completamente lógica por medio de un conjunto de funciones de Boole.

Entonces, un circuito combinacional consiste en variables de entrada, compuertas lógicas y variables de salida. Las compuertas lógicas aceptan las señales en la entrada y generan señales en las salidas. Este proceso transforma la información binaria de los datos de entrada dados a datos de salida requeridos.

En muchas aplicaciones la fuente y el destino son registros acumuladores, estos no deben influenciar el comportamiento del circuito lógico ya que si lo hace el sistema total se convierte en un circuito secuencial.

Los CC pueden describirse por una tabla de verdad y se emplean para:

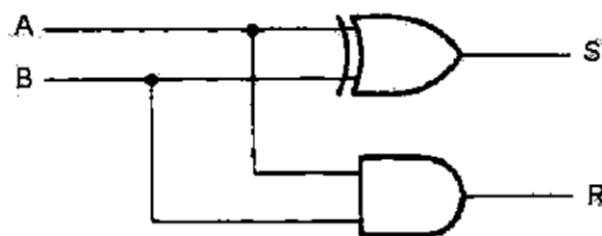
- Generar decisiones de control binarias
- Proporcionar componentes digitales requeridos para el procesamiento de datos



El Semisumador

Es un circuito que realiza la suma de dos dígitos binarios (bits). El resultado de la suma consta de dos dígitos binarios, uno de menor peso S y otro de mayor peso R. La salida S representa el bit menos significativo de la suma, y el bit R que representa el arrastre o acarreo es el bit más significativo de la suma.

Entradas		Suma	
A	B	R	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



$$S = A \oplus B$$

$$R = A \cdot B$$

Figura 4-2 Sumador Medio

Sumador Completo

Para realizar la suma de dos números binarios es preciso tener en cuenta en cada posición n , los dos dígitos de dicha posición y el arrastre de la posición $n-1$. Al operador que realiza la adición de 2 dígitos binarios y del arrastre eventual se le llama **etapa de sumador**.

El sumador completo es un circuito combinacional que consiste en 3 entradas y dos salidas. Las tres entradas serán A, B y R y las dos salidas S y R.

Entradas			Salidas	
A	B	R'	R	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

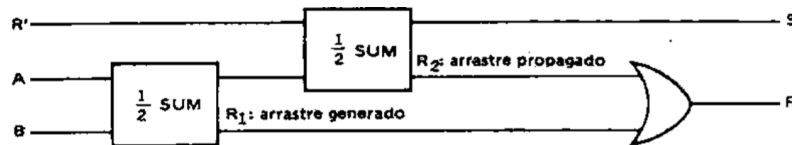
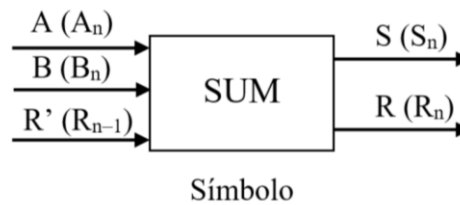
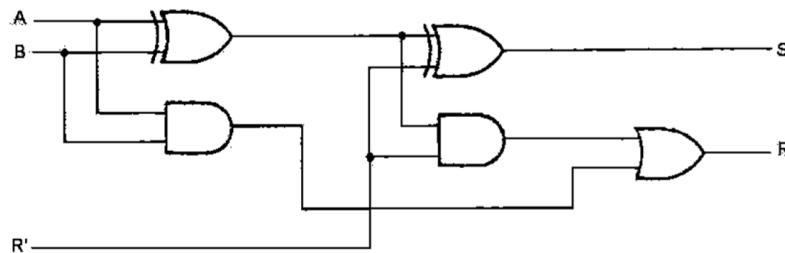


Fig. 62. Una etapa del sumador realizada con ayuda de semisumadores



El Semisustractor

El semisustractor es un circuito combinacional que resta dos bits, tiene dos salidas, uno genera el resultado de la diferencia D y la otra indica si se ha prestado un 1 (B - Borrow). Se designa el bit minuendo con X y el bit sustraendo con Y.

X	Y	B	D
0	0	0	0
0	1	1	1
1	0	0	1
1	1	0	0

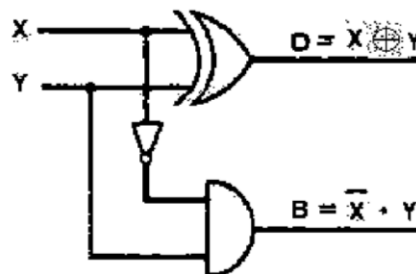
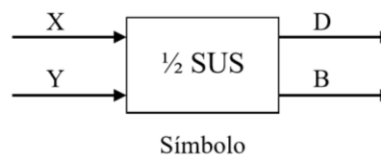


Fig. 65. Semisustractor

Sustractor Completo

El sustractor es un circuito combinacional que realiza una resta entre dos bits, tomando en consideración que se ha prestado un 1 de un estado menos significativo. Este circuito tiene 3 entradas X (minuendo), Y (sustraendo), Z (bit de arrastre o bit prestado respectivamente). Además, posee dos salidas: D (diferencia) y B (salida del bit prestado respectivamente).

Debido a sus grandes similitudes con el sumador completo, es posible convertir un sumador completo a un sustractor completo simplemente complementando la entrada X antes de su aplicación a las compuertas que forman el bit de arrastre de salida.

El sustractor puede obtenerse de dos maneras diferentes:

1. Se restan sucesivamente Y de X, después Z de la diferencia obtenida (D1).
2. Se suman Y y Z y el resultado se resta de X.

Diferencia: $D = (X \oplus Y) \oplus Z$

Prestado: $B = \overline{(X \oplus Y)} \cdot Z + X' \cdot Y$

X	Y	Z	B	D
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	1	0
1	0	0	0	1
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

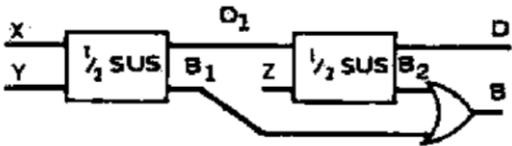
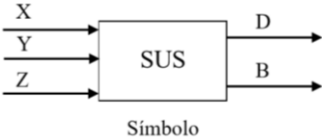


Fig. 66. Sustractor formado por dos semisustractores

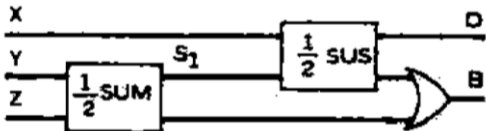


Fig. 67. Sustractor formado por un semisumador y un semisustractor

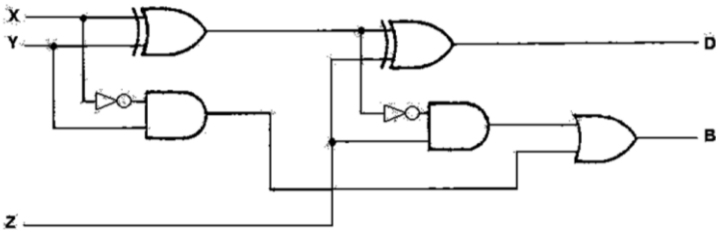


Figura Sustractor completo con dos semisustractores

Unidad 4

Circuitos Secuenciales

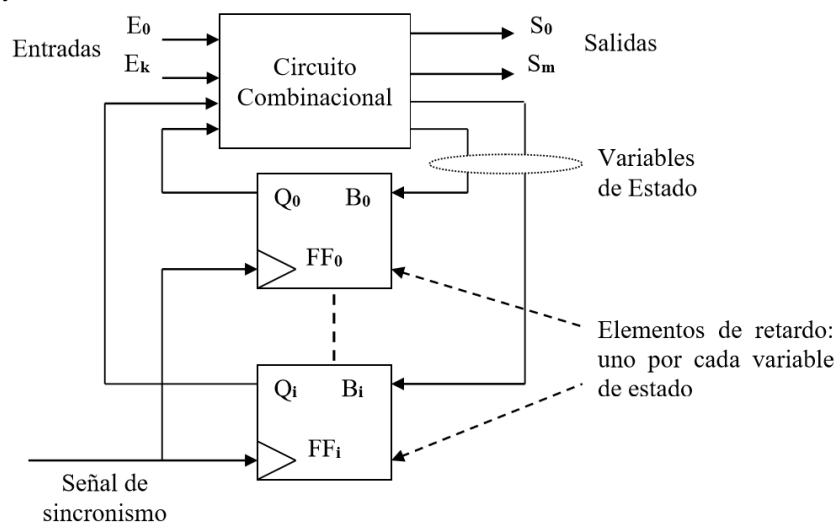
CIRCUITOS SECUENCIALES

Un circuito secuencial es una máquina de estados finitos, el cual toma una entrada y un estado actual para generar una salida y un nuevo estado. En estos circuitos, a diferencia de uno combinacional, la historia de las entradas del circuito secuencial tiene influencia sobre sus estados y sobre su salida.

Los circuitos secuenciales usan elementos de memoria (celdas binarias FF) además de las compuertas lógicas. Las salidas de un circuito secuencial están en función de las entradas externas como así también del estado presente de los elementos de memoria, pero el estado de los elementos de memoria es, a su vez, una función de las entradas previas. Por lo tanto, las salidas de un circuito secuencial dependen de las entradas presentes y las del pasado; y el comportamiento del circuito debe especificarse por una secuencia de tiempos de las entradas y estados internos. Un diagrama de bloques de un circuito secuencial consta de un circuito combinacional al cual se le conectan elementos de memoria (FF) para formar un camino de retroalimentación. La información binaria almacenada en los elementos de memoria en cualquier momento dado define el estado del circuito secuencial.

. Funcionamiento y Grafico

El modelo clásico de una máquina de estados finitos consiste en un circuito combinacional que toma las entradas desde las líneas $E_0 - E_k$ externas a la máquina de estados finitos, además de tomar las entradas desde las líneas de estado $Q_0 - Q_i$ (salidas de los FF) internas a ella. El circuito combinacional genera los nuevos bits de salida $S_0 - S_m$ y un nuevo conjunto de bits de estado. Los elementos de retardo mantienen el estado actual de la máquina de estados finitos hasta que una señal de sincronismo provoque la carga de los valores B_i en los elementos FF_i tras lo que aparecen en Q_i como los nuevos bits de estado.



. Clasificación: depende del temporizado de sus señales

1. *Circuito secuencial SINCRONICO*: es un sistema cuyo comportamiento puede definirse a partir del conocimiento de sus señales en instantes discretos de tiempos, es decir, sus señales pueden afectar los elementos de memoria en instantes discretos de tiempo.

La sincronización se logra por un dispositivo de tiempo llamado generador maestro de tiempo el cual genera un tren periódico de pulsos de reloj. Estos se distribuyen a través del sistema de tal manera que los elementos de memoria son afectados solamente con la llegada del pulso de sincronización.

2. *Circuito secuencial ASINCRONICO*: depende del orden en que cambien las señales de entrada y pueden ser afectadas en cualquier instante de tiempo dado. Los elementos de memoria comúnmente usados en los circuitos secuenciales asincrónicos son mecanismos retardadores de tiempo. Así, un circuito secuencial asincrónico puede tomarse como un circuito combinacional con realimentación.

TABLA DE ESTADOS

La secuencia de tiempo de las entradas, salidas y estados de los FF puede enumerarse en una tabla de estado. Ella consiste en 3 secciones:

- Estado Presente: designa los estados de los FF antes de la ocurrencia de un pulso de reloj
- Estado Siguiente: muestra los estados de los FF después de la aplicación de un pulso de reloj
- Salida: la selección de salida lista los valores de la variable de salida durante el presente estado

La deducción de la tabla de estados comienza a partir de un estado inicial asumido. Generalmente se define como el estado con cero en todos los FF.

Comúnmente un circuito secuencial con m FF y n variables de entrada tendrá 2^m filas, una para cada estado. Las secciones del siguiente estado y de la salida tendrán cada una 2^n columnas, una para cada combinación de entrada.

ECUACIONES DE ESTADOS

Una ecuación de estado es una expresión algebraica que especifica las condiciones para la transición de estado de un FF.

El lado izquierdo de la ecuación denota el estado siguiente del flip – flop y el lado derecho una función de Boole que especifica las condiciones del presente estado que hacen el siguiente estado igual a 1. La ecuación de estado se deriva directamente de la tabla de estado.

DIAGRAMAS DE ESTADOS

La información disponible en la tabla de estados puede representarse gráficamente en un diagrama de estados, es decir, suministran la misma información pero en diferentes formas de representar.

En este diagrama se representa un estado por un círculo y la transición entre estados se indica por líneas dirigidas que conectan los círculos. El número binario dentro de cada círculo identifica el estado representado por el círculo. Las líneas dirigidas se marcan con dos números binarios separados por /. El valor de la entrada que causa la transición de estado se marca primero; el número después de la / da el valor de salida durante el presente estado. Una línea dirigida que conecta un círculo a sí misma indica que no hay cambio de estado.

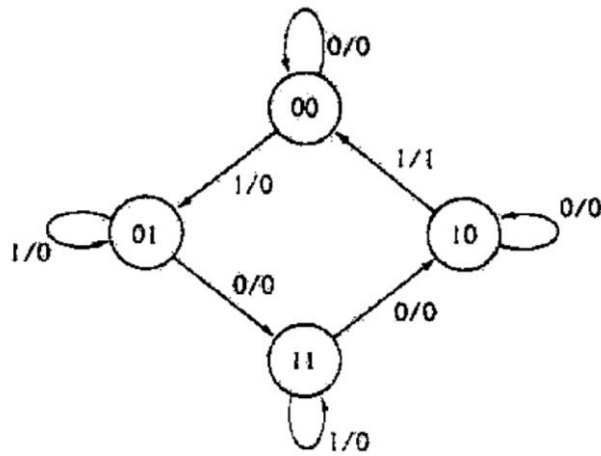


Figura 6-16 Diagrama de estado

TABLAS DE EXCITACIÓN

Cada tabla consiste en dos columnas, $Q(t)$ y $Q(t+1)$, y una columna para cada entrada para mostrar cómo se logra la transición requerida (o cambio de estado). Hay cuatro transiciones posibles del presente estado al siguiente. Las condiciones de entrada requeridas para cada una de las cuatro se derivan de la información disponible en la tabla característica (o tabla de funcionamiento). El símbolo X en las tablas representa la condición de no importa, es decir, no importa que la entrada sea 1 ó 0.

TABLAS DE FUNCIONAMIENTO

BIESTABLES

Son los elementos de memoria que se usan en los circuitos secuenciales temporizados. Estos circuitos son celdas binarias capaces de almacenar un bit de información. Un circuito FF tiene dos salidas, una para el valor normal y uno para el valor del complemento del bit almacenado en él. La *diferencia principal entre los diversos FF* está en el número de entradas que poseen y en la manera en la cual las entradas afectan al estado binario.

. Características

1. El FF es un sistema con dos estados estables (0 y 1). El estado es una situación interna del circuito. El estado de un circuito está determinado por lo valores que los biestables pueden estar en ese momento.
2. La salida de un FF queda determinada tanto por las entradas actuales como por las historias de las mismas.
3. Cada FF puede almacenar/recordar un bit de información binaria.
4. El FF tiene dos señales de salida, una es complemento de la otra.
5. El FF se mantiene en uno de los estados mientras no sea excitado por señal alguna.
6. La mayoría de los FF tiene entradas de impulso y salidas de nivel.

. Clasificación: según en qué instantes pueden actuar sus entradas y cambiar de estado

- a. Biestables SINCRONICOS: sus entradas pueden actuar sobre el estado del circuito solo en determinados instantes, cuando se activan los pulsos que llegan por otra entrada de sincronismo conocida como reloj o "clock".
- b. Biestables ASINCRONICOS: sus entradas pueden actuar en cualquier instante para cambiar el estado del circuito, y en consecuencia, cambiar el valor de sus salidas.

Flip - Flop	Asincrónico	Sincrónico
RS	SI	SI
JK	SI	SI
D	-	SI
T	SI	SI
MS	-	SI

FLIP – FLOP RS – ASINCRONICO

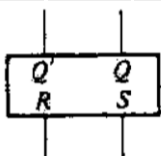
El biestable dispone de dos estados S(set) y R(reset) y de dos salidas Q y Q', donde una es el complemento de la otra, y se las trata como salidas normales y de complementos respectivamente.

Este circuito FF puede construirse con dos compuertas NOR o dos compuertas NAND. La conexión de acoplamiento inter cruzado de la salida de una compuerta a la entrada de la otra constituye un camino de retroalimentación. Por esta razón, los circuitos secuenciales se clasifican como circuitos secuenciales asincrónicos.

En este FF con compuertas NOR (funcionamiento normal) cuando se aplica un 1 solo a la entrada S el biestable se coloca en 1. Si se aplica un 1 solo por la entrada R el biestable se coloca a 0. Pero, si se aplica un 1 por ambas entradas ambas salidas Q y Q' van a ser 0, esta condición viola el hecho de que estas salidas deben ser complementos entre sí, por lo tanto, este estado es indefinido y debe evitarse.

Este circuito RS también puede realizarse con compuertas NAND pero trabaja inversa a la anterior. Un 0 en S pone el FF a 1; un 0 en R pone el FF a 0; y cuando las dos entradas son 0 el estado del FF es indefinido.

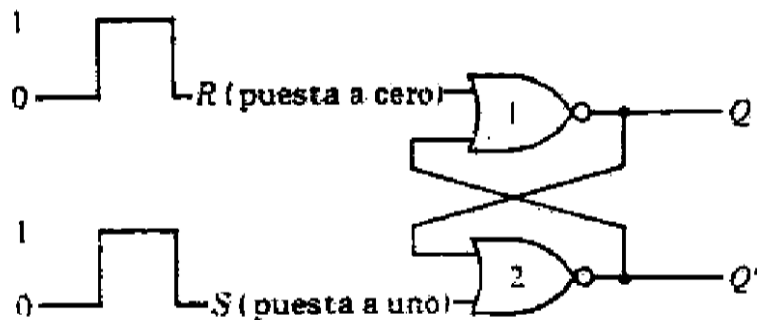
Q_t	Q_{t+1}	R	S
0	0	X	0
0	1	0	1
1	0	1	0
1	1	0	X



(b) Símbolo gráfico

Tabla de funcionamiento

R	S	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	?
1	1	1	?



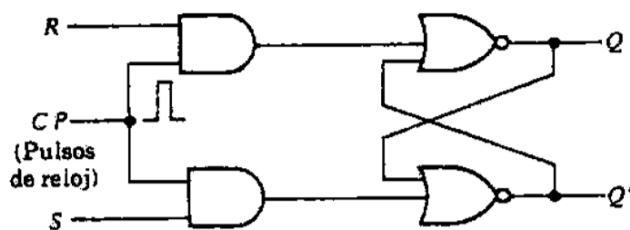
(a) Diagrama lógico

FLIP – FLOP RS – SINCRONICO

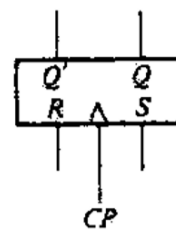
Con el objeto de lograr una sincronización controlada de los circuitos de lógica secuencial, se suele incorporar una señal de sincronismo o reloj, con la cual cada circuito se sincroniza a sí mismo para aceptar cambios de sus entradas en instantes determinados. Un circuito de reloj produce una secuencia continua de unos y ceros.

El FF RS con reloj consta de un FF básico RS-ASINCRONICO pero se le agrega dos compuertas AND. Las salidas de las dos compuertas AND permanecen en 0 en tanto que el pulso de reloj (CP) sea 0, sin importar los valores de la entrada de R y S. Cuando el CP va a 1, se permite que la información de las entradas S y R lleguen al FF básico.

Este FF tiene tres entradas S, R y CP; la entrada CP no se escribe sino que se simboliza con un triángulo, que denota el hecho de que el FF responde a una transición de reloj.



(a) Diagrama lógico

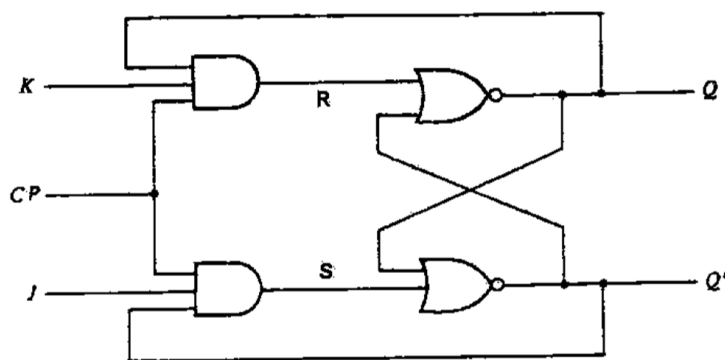


(b) Símbolo gráfico

FLIP FLOP – JK

Un FF- JK es un refinamiento del FF RS ya que el estado indeterminado del tipo RS se define en el tipo JK. Las entradas J y K se comportan como las entradas S y R, la entrada J se usa para la entrada de puesta a uno y la entrada K para la entrada de puesta a cero. Cuando ambas señales de entrada son 1, el FF – JK cambia su estado de complemento, o sea, si $Q=1$ cambia a $Q=0$ y viceversa.

La salida Q se aplica con K y CP a una compuerta AND de tal manera que el FF se ponga a cero durante un pulso de reloj solamente si Q fue 1 previamente.
 Cuando J y K sean 1, el pulso de reloj se transmite solamente a través de una y solo una compuerta AND, y ya sea que $Q=1$ o $Q'=1$, en cualquiera de los casos el estado de salida del FF se complementa cumpliendo con el funcionamiento.



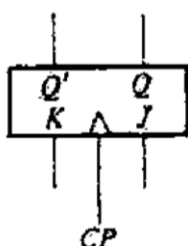
(a) Diagrama lógico

Q	J	K	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

(c) Tabla característica

Tabla de excitación

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0



(b) Símbolo gráfico

FLIP FLOP – D

Es una modificación del FF básico RS con reloj. La entrada D va en forma directa a la entrada S y su complemento a la entrada R. La diferencia que tiene este FF con el RS con reloj, es que el RS para almacenar un 0 o un 1, hace falta aplicar un 1 a una de sus entradas y un 0 a la otra, en cambio, en este FF como R y S están conectadas a través de un inversor, si $D=1$ se pone a uno y si $D=0$ se pone a cero. (Siempre y cuando $CP=1$).

Este FF recibe su denominación por la habilidad de transferir datos a su FF. El inversor agregado reduce el número de entradas de dos a uno.

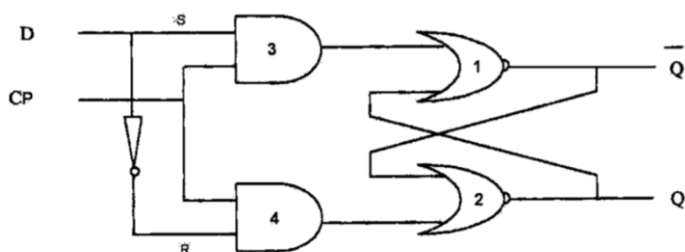
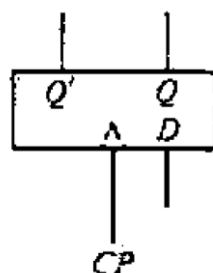


Tabla de funcionamiento del flip flop D Tabla de excitación del flip flop D

D	Q_t	Q_{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

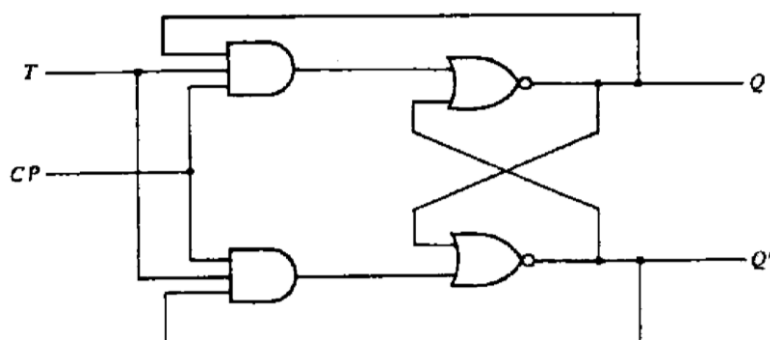
Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1



(b) Símbolo gráfico

FLIP FLOP – T

El FF-T es una versión de una sola entrada del FF – JK. El FF – T se obtiene de un tipo JK al cual se le unen las dos entradas. La denominación T proviene de la habilidad del FF para “conmutar” o cambiar de estado (toggle). Es decir, este biestable mantiene su estado si por la entrada ingresa un 0 y complementa el estado si por la entrada ingresa un 1.



(a) Diagrama lógico

Tabla de funcionamiento

T	Q_t	Q_{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

Tabla de excitación

Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

FLIP FLOP – MAESTRO ESCLAVO

Un FF maestro esclavo se construye con dos FF por separado. Este consiste en un FF maestro, un FF esclavo y un inversor.

Durante la subida del pulso de reloj se habilita el maestro y se deshabilita el esclavo. La información de entrada es transmitida hacia el FF maestro. Cuando el pulso baja nuevamente a cero se deshabilita el maestro lo cual evita que lo afecten las entradas externas y se habilita el esclavo. Entonces el esclavo pasa al mismo estado del maestro.

Lo mismo sucederá si el esclavo tiene una información binaria almacenada y el CP pasa a 0, entonces se habilitará el maestro y todas las entradas que afecten al biestable maestro no afectarán al esclavo mientras el CP se encuentre en 0.

Entonces, en un flip – flop maestro esclavo, es posible variar la salida y la información de entrada con el mismo pulso de reloj. Puede ocurrir por ejemplo que la entrada S venga de la salida de otro flip – flop maestro esclavo que fuera conmutado con el mismo pulso de reloj.

La combinación maestro esclavo puede construirse para cualquier tipo de flip-flop agregando un flip-flop RS temporizado con un reloj invertido para formar un esclavo.

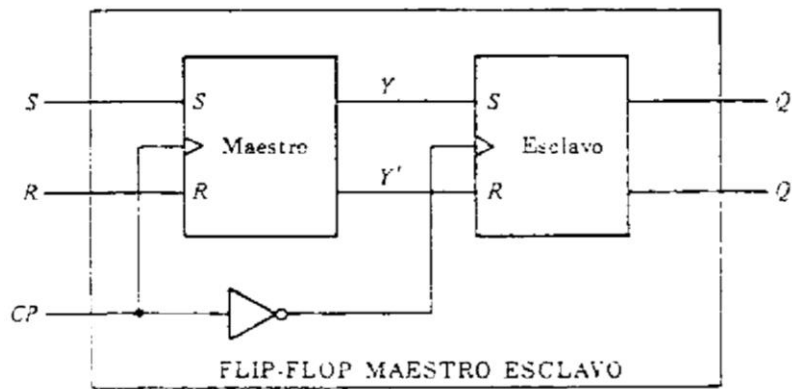


Figura 6-9 Diagrama lógico de un flip-flop maestro esclavo

MONOESTABLE

Es un sistema de 2 estados con solo una posición estable. Bascula bajo el efecto de un impulso de entrada y retorna seguidamente a su posición de equilibrio.

El estado estable es el estado 0, que bajo la influencia de un impulso S el biestable pasa a 1, lo que a su vez posiciona la entrada R a 1: el sistema vuelve al estado de equilibrio. La temporización θ permite ajustar la duración de la señal de salida

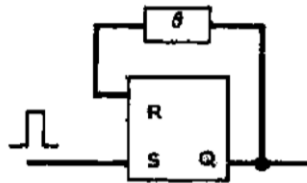


Fig. 47. Monoestable

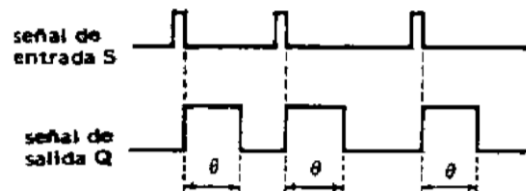


Fig. 48. Cronograma del monoestable

BASCULA

La báscula es un sistema de 2 estados, ninguno de ellos estable. Puede realizarse uniendo las 2 entradas de un biestable JK.

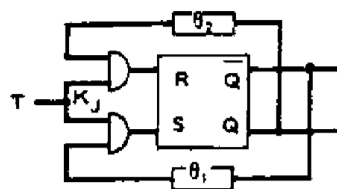


Fig. 49. Báscula

Mientras se tenga a 1 la entrada T, la báscula cambia constantemente de estado y el ritmo de cambio depende del tiempo de respuesta de los circuitos. Este ritmo es ajustable mediante las temporizaciones θ_1 y θ_2 . Pueden utilizarse las básculas para realizar relojes que dan un tren de impulsos regularmente espaciados

REGISTROS

Se denomina registro a un conjunto de biestables que funcionan como unidad, permitiendo almacenar información en el transcurso de un proceso. Son elementos de memoria de almacenamiento temporal de información, instrucciones y datos, durante el procesamiento. Un registro es un grupo de celdas de almacenamiento binario capaz de retener información binaria que actúan en conjunto para cumplir con cierta función. Como una celda almacena un bit de información, se desprende que un registro de n celdas puede almacenar cualquier cantidad discreta de información que contenga n bits. Un registro de n bits tiene un grupo de n flip – flops y tiene la capacidad de acumular cualquier información binaria que contiene n bits.

El estado de un registro es un número enésimo de unos o ceros con cada bit indicando el estado de una celda en el registro. Más claramente, un registro de n celdas puede estar en uno de los 2^n estados posibles. El contenido de un registro, es una función de la interpretación dada a la información almacenada en él.

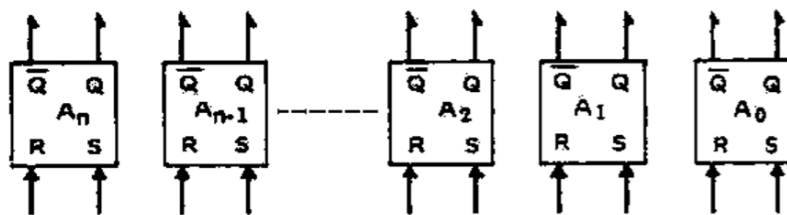


Fig. 68 Registro con biestables

Además de los flip – flops, un registro puede tener compuertas combinacionales que ejecutan ciertas tareas de procesamiento de datos, es decir, que las compuertas controlan cuando y como se transfiere la nueva información al registro.

La forma más común e representar un registro es por medio de un rectángulo con el nombre del registro dentro de él (Figura a). Las celdas individuales pueden ser distinguidas como en la Figura b, cada celda con su respectiva letra y número suscrito.

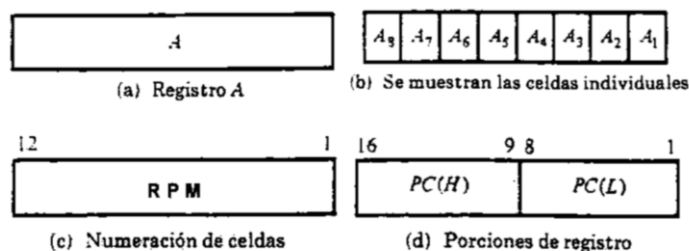
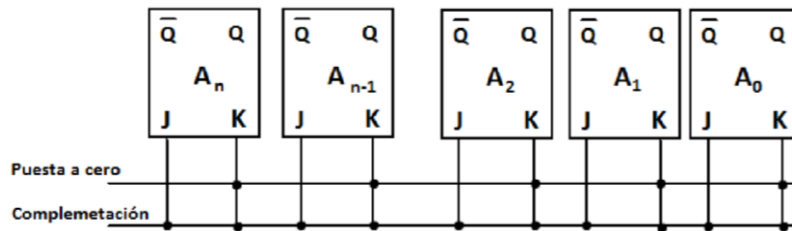


Figura 8-1 Diagrama de bloque de los registros

Operaciones elementales sobre los registros

1. Un registro puede limpiarse o ponerse a cero, enviándole un impulso por la entrada R de cada biestable (si son FF-RS).
2. El contenido de un registro puede complementarse, enviándole un impulso por la entrada C (de complemento) de cada biestable. Para complemento a la base -1 se toma los Q' .

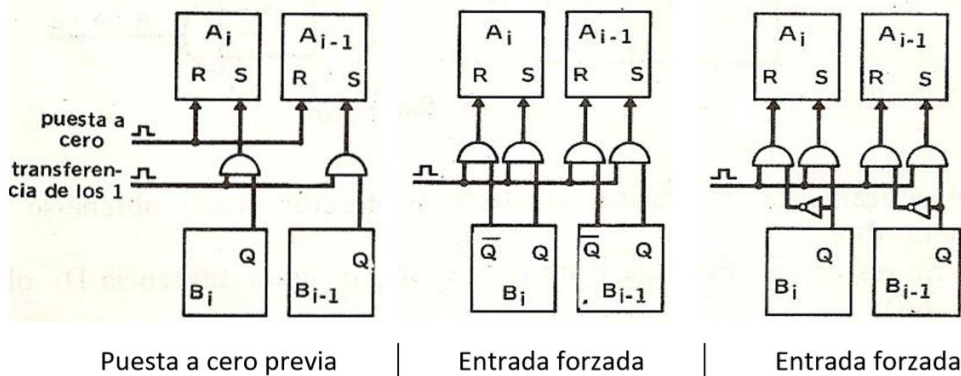


Transferencias en paralelo entre registros

Una operación de transferencia entre registros es una operación básica en sistemas digitales y consiste en la transferencia de la información almacenada de un registro a otro.

Puede transferirse la información de un registro B a otro registro A:

- Por dos impulsos sucesivos, uno de puesta a cero y el otro de transferencia de los 1.
- Por introducción forzada de los 0 y de los 1.



Registros de Desplazamiento

Cableando adecuadamente el entorno de un registro de biestables se consigue, en respuesta a un determinado número de impulsos, desplazar el contenido del registro de uno o varios dígitos a la izquierda o derecha.

Un *registro de desplazamiento* es un registro capaz de desplazar su información binaria hacia la izquierda o hacia la derecha.

La configuración lógica de un registro de desplazamiento consiste en una cadena de FF conectados en cascada, con la salida de un FF conectado a la entrada del siguiente. Todos los FF reciben un pulso de reloj común el cual causa el desplazamiento de un estado al siguiente. La salida Q de un FF dado, se conecta a la entrada S, mientras que la salida Q' a la entrada R, del FF a la derecha (en caso de desplazamiento a derecha). Cuando se trabaja con FF D, la salida Q va conectado a la entrada D, del FF de la derecha.

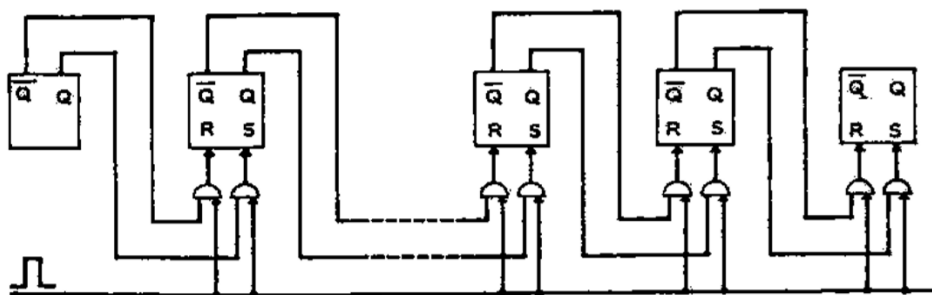


Fig. 74. Registro de desplazamientos a derecha

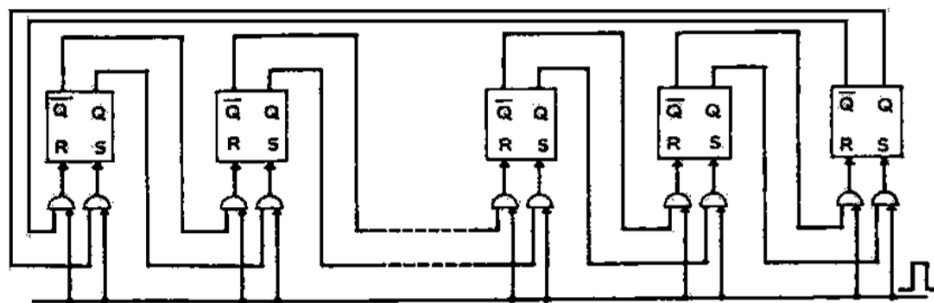


Fig. 74. Registro de desplazamientos a derecha cíclico

Registros contadores

Son registros cuyo contenido se incrementa en 1 cada vez que se presenta un impulso a la entrada de cuenta. Son capaces de contar el número de impulsos recibidos.

Trabaja de la siguiente manera: el impulso ataca al biestable de menor peso posicional. Si está a cero, debe pasar a 1; si está a 1, debe pasar a 0, pero debe propagar el impulso al biestable inmediato con objeto de tomar en cuenta el arrastre. Por consiguiente, el biestable debe ser complementado en todos los casos y debe propagarse el impulso solamente en aquellos casos en que el biestable estaba a 1 antes de la complementación (condición para que haya arrastre).

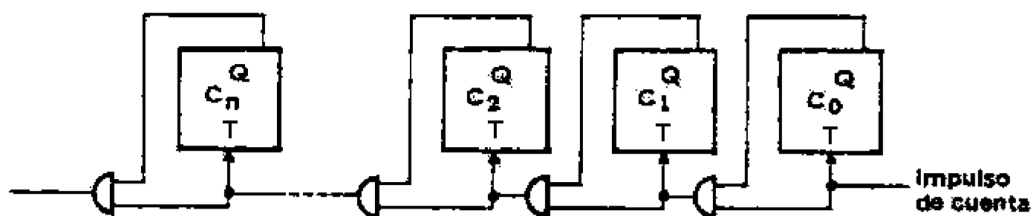


Fig. 76. Contador binario

Unidad 5

Las Memorias

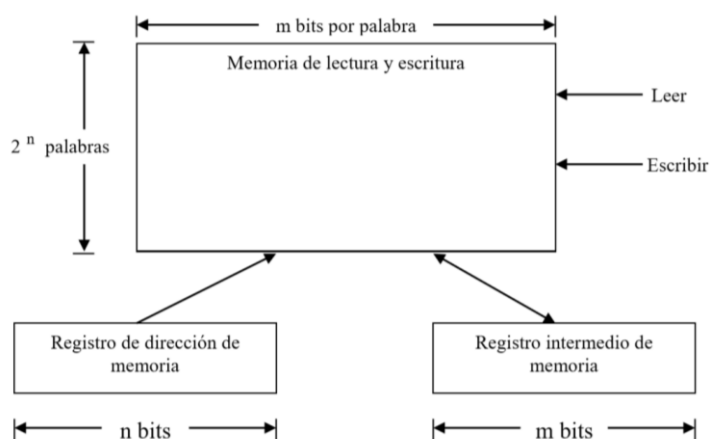
INTRODUCCIÓN

La memoria está formada por un conjunto de células o registros, capaces de almacenar información (en general, una palabra). La información guardada podrá ser un dato o una instrucción, sin embargo, las instrucciones están en una zona y los datos en otra. Los registros de almacenamiento en una unidad de memoria se llaman **registros de memoria**.

Cada célula de memoria tiene un número que la identifica llamada **dirección**, que es habilitada por la unidad de control. Esta habilitación nos permite leer o escribir información contenida en una dirección determinada.

Para realizar una lectura o escritura, la unidad de control proporciona la dirección de la célula implicada a un registro asociado a la memoria central, llamado **registro de dirección** de memoria, o también conocido como **registro de selección** de memoria. Este registro está formado por n elementos binarios (generalmente FF), en los que 2^n es el número de palabras que pueden ser almacenados en la memoria.

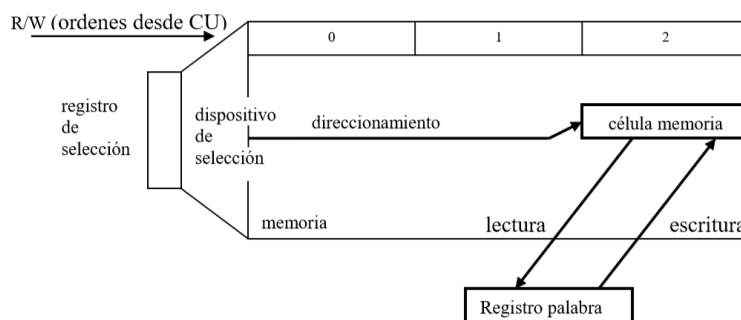
Figura 1 Memoria de lectura y escritura



El dispositivo de selección de memoria analiza el contenido del registro de dirección/selección y sensibiliza la célula implicada, bien para una lectura o para una escritura. Si se tratase de una lectura, primero se limpia registro de palabra, y luego la información almacenada en la célula sensibilizada será transferida a este registro. El registro de palabra está asociado a la memoria central, y también se conoce como registro de intercambio que tiene tantos elementos de

almacenamiento binario como bits en cada palabra de memoria (m bits). En el caso de una escritura, previamente habrá sido preciso cargar este mismo registro con la información que se quiera transferir a la célula en cuestión. Se dice a la memoria que se va a escribir, por medio de un 1 en la línea de escribir. La memoria, entonces almacenará el contenido del registro intermedio de memoria, en la posición especificada en el registro de dirección de memoria.

Figura 2 lectura y escritura en la memoria



DEFINICIÓN

Se denomina memoria a todo dispositivo electrónico capaz de almacenar informaciones, de tal manera que el órgano que se sirva de él pueda acceder a la información solicitada en cualquier momento. Este dispositivo forma parte integral de la máquina y está directamente controlado por ella.

La memoria debe ser de *lectura no destructiva*, por lo tanto una celda de memoria tendrá que poder ser leída todas las veces que sea necesaria sin que se altere su contenido. Solo deberá modificarse el contenido cuando escribimos una palabra.

. Punto de Memoria: se denomina punto de memoria al soporte físico del bit; que es la información más elemental registrada en la totalidad de las memorias (almacenamiento binario). Es decir, cada punto de memoria almacena un bit. El punto de memoria puede ser perfectamente definido e individualizado y constará, además del dispositivo de almacenamiento, de los de lectura y escritura: es el caso del biestable electrónico o del núcleo de ferrita.

CARACTERÍSTICAS DE MEMORIA

- **Volatilidad**: se dice que la información almacenada es volátil si corre el riesgo de verse alterada cuando se produce un fallo de la alimentación eléctrica y no volátil en caso contrario. Las memorias constituidas por biestables electrónicos son volátiles, en cambio, las magnéticas son no volátiles.
- **Lectura y Escritura**: son las operaciones básicas de las memorias. Se realiza *escritura* cuando se registran informaciones en la memoria y *lectura* cuando se extraen informaciones previamente registradas.

La lectura puede ser **destructiva** (la información leída se borra de la memoria), por ejemplo, en los núcleos magnéticos o en la RAM Dinámica; en estas se debe re-grabar la información sistemáticamente luego de cada lectura. En caso contrario, la lectura es **no destructiva**.

La escritura puede exigir o no exigir un borrado previo. Las memorias de semiconductores son de lectura no destructiva y no obligan forzosamente a un borrado previo a la escritura, mientras que en las memorias de núcleos la lectura es destructiva y la escritura exige un borrado previo.

- **Direccionamiento:** Para localizar una información dentro de una memoria, la computadora debe saber donde la ha colocado anteriormente. Es la función del direccionamiento.
- **Modo de Acceso:**
 - *Aleatorio, directo o selectivo:* en las memorias con punto de memoria electrónico (biestables) se tiene acceso directamente a cualquier información, cuya dirección sea previamente conocida, en una cantidad de tiempo que es independiente a la posición de la información en la estructura de la memoria.
 - *Secuencial:* en las informaciones guardadas en cinta magnéticas, es necesario desenrollar completamente la cinta, leyendo registro por registro hasta encontrar el buscado. En este tipo de memorias, el tiempo de acceso al registro buscado dependerá fundamentalmente de la posición de este en la cinta. En los discos magnéticos el acceso a la información es selectivo (directo) en cuanto a la elección de la pista, y secuencial en el interior de la pista.
- **Tiempo de Acceso:** Tiempo que transcurre entre el instante en que se lanza una operación de lectura en memoria y el instante en que se dispone, en el RPM, la primera información buscada.
- **Tiempo de Escritura:** Es el tiempo que transcurre entre el momento en que se proveen, en los registros asociados correspondientes de la memoria, la información a guardar y su dirección, y el instante en que la información queda realmente almacenada.
- **Velocidad de Transferencia:** Es la velocidad a la que se pueden transferir datos a, o desde, una unidad de memoria. Para memorias de acceso aleatorio coincide con el inverso del tiempo de ciclo.
- **Caudal:** así se llama al número máximo de informaciones leídas o escritas por unidad de tiempo. Para las memorias de acceso por bloques se supone que las informaciones están situadas en lugares sucesivos de memoria. Se expresa en kilo-informaciones o mega-informaciones por segundo.
- **Tiempo de Ciclo de Memoria:** incluye el tiempo de lectura y reescritura.
- **Capacidad:** Es el número de informaciones que la memoria puede almacenar. Teniendo en cuenta el direccionamiento binario, las capacidades de la memoria se expresan habitualmente como potencia de dos.

CLASIFICACIÓN TECNOLÓGICA

	Bipolares	MOS Estática	MOS Dinámica	CMOS	SOS	IIL
Velocidad	Rápidas	+ Lentas	++ Lentas	+ Rápidas que MOS	+ Rápida que CMOS	= Bipolares
Costo	Mayor	Menor	++ Menor	+ que MOS	+ que CMOS	Intermedio
Potencia		Consumen menos potencia		+ que MOS		Menor que la bipolar
Densidad de Empaque		Mayor	++ Mayor			= MOS
Área de la Celda		$\frac{1}{2}$ - $\frac{1}{4}$ de una celda bipolar				
Tamaño			+ simple y pequeña que la estática			
	Registros y Caché	Usan FF como elemento de almacenamiento del punto de memoria	Se usa la carga depositada en un condensador fabricado en circuitos integrados. La presencia o ausencia de esta carga determina el estado de la celda. Preferible para memorias grandes.		Formada sobre una superficie aisladora de safiro. Reduce	

TIPOS DE MEMORIA

Tipo de memoria	Clase	Borrado	Mecanismo de Escritura	Volatilidad	Uso Típico
Memoria de acceso aleatorio (RAM)	Lectura/Escritura	Eléctricamente por Bytes	Eléctricamente	Volátil	SRAM: Caché DRAM: memoria principal
Memoria de solo lectura (ROM)	Solo lectura	No disponible	Mediante Mascaras	No volátil	Aparatos en vol. grandes
ROM Programable (PROM)					Equipos en vol. pequeños
ROM Borrable (EPROM)	Sobre todo lectura	Luz ultravioleta, chip completo	Eléctricamente		Prototipos de dispositivos
PROM Borrable Eléctricamente (EEPROM)		Eléctricamente por Bytes			Prototipos de dispositivos
Memoria Flash	Lectura/Escritura	Eléctricamente por bloques			

Las **memorias SRAM** (Static) de semiconductores son memorias vivas (lectura/escritura), volátiles y no estáticas. Cada celda es un elemento biestable diseñado con compuertas, que puede modificarse solo con un proceso de escritura, si no, permanece inalterable siempre que esté conectada a una fuente de suministro.

Las **memorias DRAM** (Dynamic) son memorias vivas, volátiles y dinámicas. El termino dinámica hace referencia a que estas memorias degradan su información con el transcurso del tiempo, aun cuando están conectadas a la fuente de suministro. Cada celda almacena un “1” que se representa con la carga de un condensador o transistor. Este conserva su carga con cierto nivel de deterioro durante un periodo preestablecido; por lo tanto, antes de que la información se pierda hay que restablecer la carga. Este proceso se denomina **ciclo de refresco**. Son memorias más lentas que las SRAM pero tienen mayor capacidad. Una celda binaria de tipo estático está constituido por seis transistores, entonces el costo por bit de almacenamiento es mucho menor en las DRAM que en las SRAM, debido a que se gana mayor densidad de almacenamiento de bits en el mismo espacio.

JERARQUIA DE MEMORIAS

. Definición: “Es una sinopsis del conocimiento específico/particular de cada uno de los niveles que integran la jerarquía.”

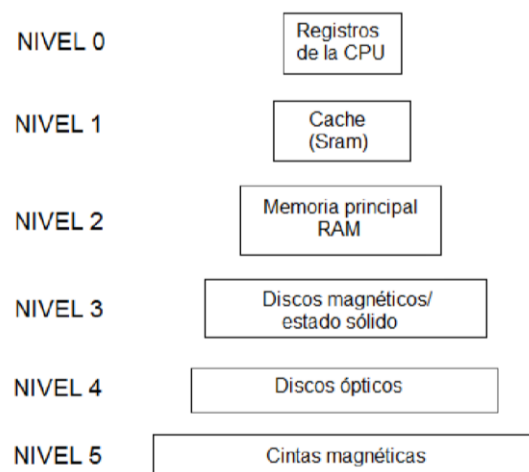
“Es una organización que agrupa tipos de memoria en función a 3 parámetros que la identifican: capacidad, tiempo de acceso y costo. De esta manera, cada nivel establece el área de aplicación dentro de la arquitectura, donde se desempeñarán estas memorias con similares características.”

“Es una clasificación de todas las memorias por niveles teniendo en cuenta similares tiempos de acceso, similares costos y similares capacidades. Cada nivel deberá agrupar a su vez, a grupos de memoria clasificándolas por su campo de aplicación dentro de la arquitectura.”

“Cuando hay una duda de donde colocar una memoria debido a su capacidad, costo y tiempo de acceso no concuerda totalmente con un nivel se hace principal énfasis en su campo de aplicación. Cada nivel define lo que la arquitectura espera de esos dispositivos, por ejemplo, el disco rígido respecto a un grabador de cinta de backup puede tener similar costo por unidad de información, similar capacidad y hasta quizás similar tiempo de acceso; pero el nivel de disco rígido aplicado a la arquitectura no podría incluir una grabadora de backup porque tiene otra función en la arquitectura.”

En un ordenador hay una jerarquía de memorias atendiendo al tiempo de acceso y a la capacidad que normalmente son factores contrapuestos por razones económicas y en muchos casos también físicas. Comenzando desde el procesador al exterior, es decir en orden creciente de tiempo de acceso y capacidad.

“La capacidad a medida que bajamos es mayor, los dispositivos almacenan más. El costo es menor hacia abajo. El tiempo de acceso disminuye hacia abajo.” (Hacia donde apunta la flecha significa que aumenta).



*costo = costo por unidad de información

Nivel 0: Registros del Procesador/CPU

Estos registros interaccionan continuamente con la CPU (porque forman parte de ella). Los registros tienen un tiempo de acceso muy pequeño y una capacidad mínima, normalmente igual a la palabra del procesador (1 a 8 bytes).

Los registros intermedios constituyen un paso intermedio entre el procesador y la memoria, tienen un tiempo de acceso muy breve y muy poca capacidad.

Son registros que se utilizan como registros de almacenamiento de operandos o registros de cálculo, registros para almacenar información de control y estado del sistema y registros para almacenar temporalmente identificadores de posiciones en la memoria principal.

“Estos dispositivos tienen que ser muy veloz, el campo de aplicación es para el diseño lógico de la máquina Von Neumann.”

“Estos registros son los que interaccionan con la unidad central de proceso. Los registros de la CPU son:

- *Registro P*
- *Registro de selección de memoria*
- *Registro M*
- *Acumulador*
- *Contador de programa*
- *Registro de instrucción.*

Sus características es que son pequeñas capacidades.”

“Se habla de costo no porque estos registros se compren, sino que son un diseño industrial son parte de una placa. Se habla de costo por unidad de información pero relativo.”

Nivel 1: Memorias Caché (SRAM)

Es una tecnología de semiconductor de tipo estático, cuya velocidad de respuesta se ajusta de manera muy favorable a los tiempos del procesador.

Son memorias de pequeña capacidad, normalmente una pequeña fracción de la memoria principal, y pequeño tiempo de acceso. Este nivel de memorias se coloca entre la CPU y la memoria central. Hace algunos años este nivel era exclusivo de los ordenadores grandes pero actualmente todos los ordenadores lo incorporan. Dentro de la memoria caché puede haber a su vez, dos niveles denominados **caché on chip**, memoria caché dentro del circuito integrado, y **caché on board**, memoria caché en la placa del circuito impreso pero fuera del circuito integrado, evidentemente, por razones físicas, la primera es mucho más rápida que la segunda. Existe también una técnica, denominada arquitectura Harvard, en cierto modo contrapuesta a la idea de Von Neumann, que utiliza memorias caché separadas para códigos y datos. Esto tiene algunas ventajas como se verá posteriormente.

“Un caché es un dispositivo que permite amortiguar las diferencias de velocidad que hay en la transmisión de datos entre un dispositivo y otro. Es una memoria intermedia que permite mejorar la diferencia de velocidad entre un dispositivo que transmite y otro que recibe.

Por ejemplo, cuando la memoria RAM (Tiempo de acceso=2 nanosegundos) quiere transmitir a una impresora (TA=0.05 segundos), entonces cuando la memoria transmite se queda ociosa, para solucionar esto se agrega la memoria caché entre ellos.”

“La CPU se comunica con la RAM por medio del bus M.”

“La cache se ubica entre la CPU y la memoria principal que es la RAM. Si la CPU necesita una información le consulta al caché y si no tiene, la CPU le pide a la RAM, si la RAM tiene la información le pasa al caché y luego el caché se encarga de pasarle a la CPU. Mientras el caché tenga la información buscada se comunican entre la CPU y el caché dejando libre la RAM.”

CPU

CACHE

RAM

Nivel 2: Memorias Principal/RAM (DRAM)

En este nivel residen los programas y los datos. La CPU lee y escribe datos en él y aunque con menos frecuencia que en los niveles anteriores. Tiene un tiempo de acceso relativamente rápido y gran capacidad. Las extensiones de memoria central son memorias de la misma naturaleza que la memoria central que amplían su capacidad de forma modular.

El tiempo es similar, a lo sumo un poco mayor, al de la memoria central y su capacidad puede ser algunas veces mayor.

“Su campo de aplicación es: está afuera de la CPU, es intercambiable, son módulos que van afuera. Tiene una amplia conectividad con la arquitectura a través del bus M (mover instrucciones y datos), el bus S (enviar direcciones y poder ubicar palabras adentro).”

Nivel 3: Discos magnéticos/ SSD

Son memorias que residen en dispositivos externos al ordenador, en ellas se archivan programas y datos para su uso posterior. También se usan estas memorias para apoyo de la memoria central en caso de que esta sea insuficiente (memoria virtual).

Estas memorias suelen tener gran capacidad, pero pueden llegar a tener un tiempo de acceso muy lento. Dentro de ellas también se pueden establecer varios niveles de jerarquía. Permiten acceso aleatorio a la información.

Las memorias auxiliares no pueden tener una relación directa con el procesador, lo hacen por medio de la memoria principal. Se incluyen en toda computadora debido a su carácter no volátil.

“Estas memorias suelen tener un tiempo de acceso notablemente más lento respecto a la RAM.”

“Su objetivo es acceder velozmente a campos de programa y aplicación.”

Nivel 4: Discos Ópticos

Un disco óptico es básicamente un pedazo en plástico lleno de marcas o huecos que un rayo láser puede leer. Cuando más marcas podamos hacer (y leer) en su superficie, mayor será la capacidad de datos que podremos almacenar en él. (CD o DVD)

Nivel 5: Cintas Magnéticas

“Para backup es lo mejor. La cinta magnética tiene una durabilidad superior a cualquier otro dispositivo, ni siquiera el pendrive tiene la fiabilidad ni el costo tan bajo que tiene una cinta magnética. No podemos negar que este nivel se creó para aquellos dispositivos que son para backup. No es un nivel para sufrir, su campo de aplicación es el almacenamiento en masa, backup.”

ORGANIZACIÓN DE MEMORIA

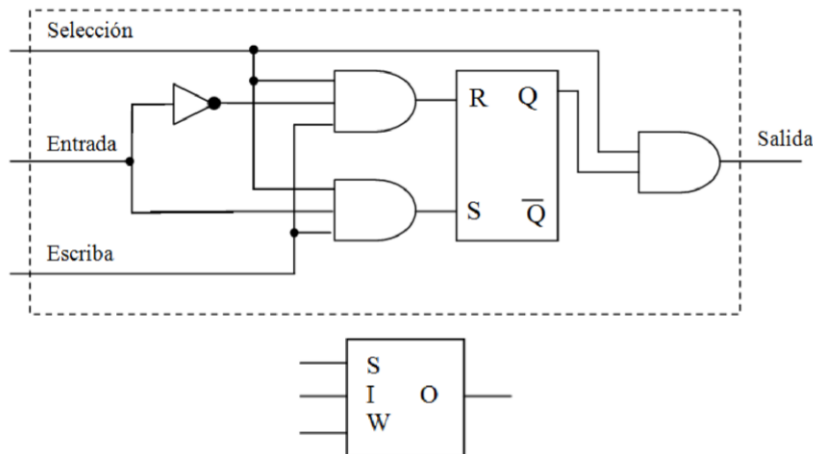
“La organización de memoria es un esquema digital que permite vincular en un único circuito a un conjunto de puntos organizados como palabra de memoria, para que estas permitan mantener información binaria referida a datos y programas, de manera organizada. Deberá incluir un diseño combinacional que permita seleccionar una y solo una palabra, dada una dirección de memoria y las capacidades básicas de lectura y escritura, de la información proveniente del bus M.”

Memoria 2D

En cualquier memoria debe existir un punto de memoria, formada por el elemento de almacenamiento (flip-flop) y los circuitos de control. Para que esta celda sea útil, deberá poder ser seleccionada mediante el registro de dirección de memoria y se debe tener un método para controlar cuando las celdas seleccionadas deban ser leídas o escritas.

Punto de Memoria

Es el soporte físico del bit. El objetivo del Flip-Flop es almacenar un 0 o 1, que va a ingresar por la entrada "I".



El punto de memoria 2D está compuesto por: un flip-flop de tipo RS, tres compuertas AND y una compuerta NOT. Posee tres entradas:

- S (selección): tiene una doble función, seleccionar el punto y también producir la lectura de su contenido a través de la salida O (output).
- I (input - entrada): permite el ingreso del bit que se va a almacenar.
- W (write - escritura): indica el momento en que se grabará la información de I en el flip-flop.

La lectura del flip-flop saldrá por O (output) siempre que S (selección) = 1.

Dispositivo de Selección

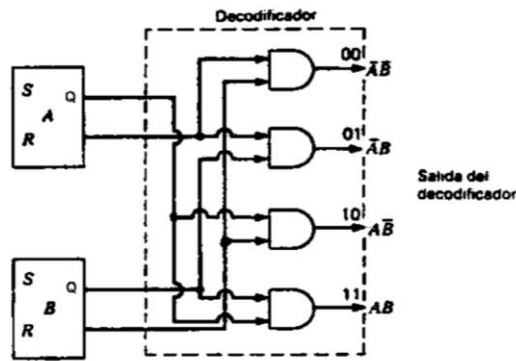
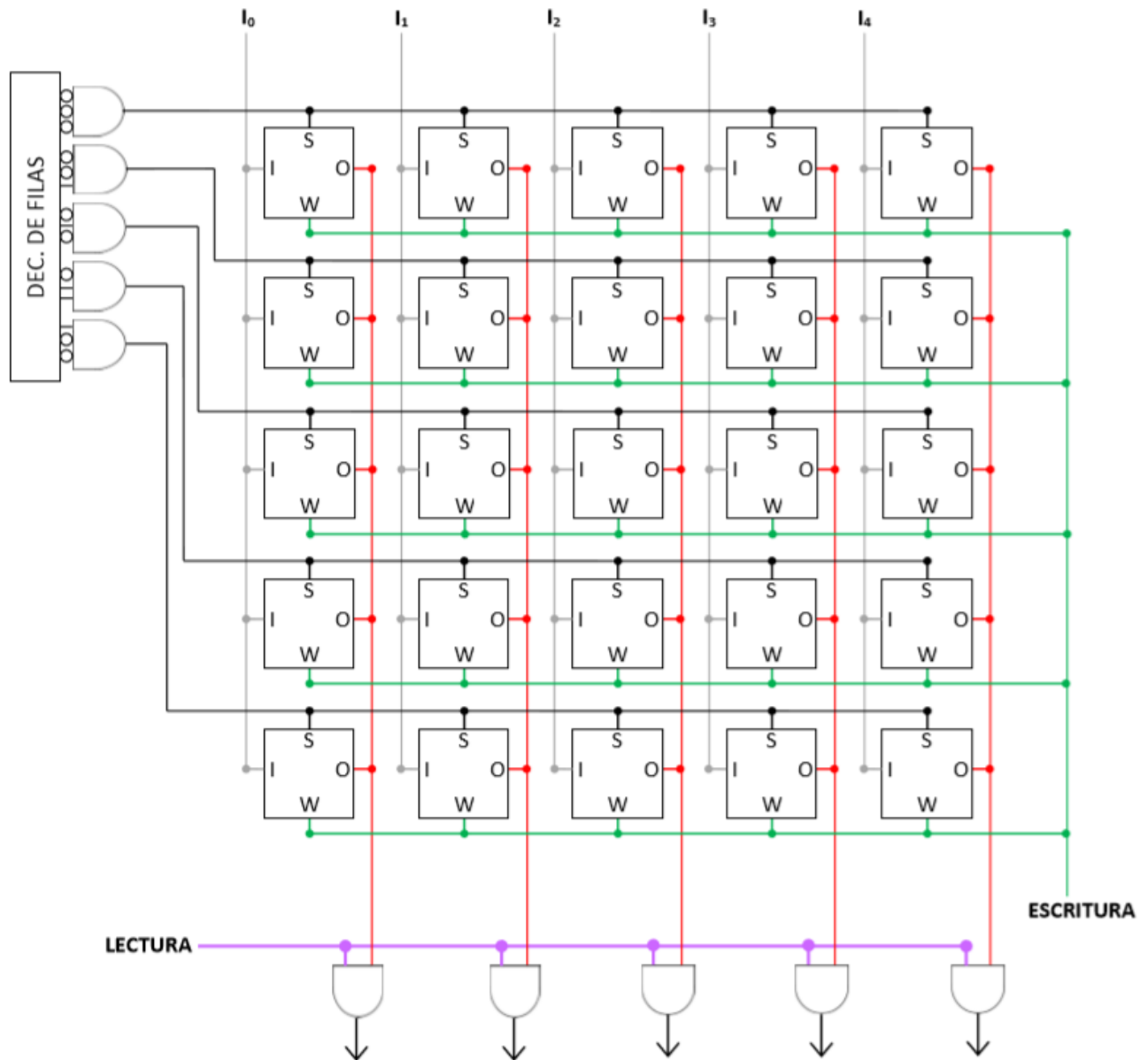


Fig. 7.5. (a) Decodificador de cuatro salidas.

El esquema de organización tiene 4 palabras de 3 bits por palabra (por lo tanto tendremos 3 puntos de memorias por cada palabra almacenada). Por lo tanto podemos decir que cada fila tiene 3 celdas que componen una palabra. El registro de dirección de memoria selecciona la celda de memoria (flip-flop) para ser leída o escrita, por medio de un decodificador que selecciona tres flip-flops para cada una de las direcciones que puede tener el registro de dirección de memoria. El esquema muestra que el decodificador tiene una entrada para cada flip-flop (bit) que va a ser decodificado. Si hay dos bits de entrada, se tendrán 4 líneas de salida, una para cada estado (valor) que pueda tomar el registro de entrada. Por ejemplo, si el RDM tiene 0 en los dos flip-flops (es decir, $Q = 0$ de cada flip-flop), la línea superior de salida del decodificador será 1 y las tres restantes permanecerán en 0. De igual forma, si las dos celdas de memoria tienen un 1, la línea de salida más baja será 1, y las tres restantes serán 0. Razonamientos similares mostrarán que habrá una línea de salida en 1, para cada uno de los posibles estados de entrada, mientras que las restantes permanecen en 0.

MEMORIA 2D DE 5x5



“Vamos a tener tantas compuertas AND para la lectura como cantidad de bits tenga la palabra. Generalmente, como mínimo una palabra en memoria tiene 8 bits.”

Para **escribir** una determinada configuración binaria en una palabra de memoria se envía un 1 para seleccionar (por medio del decodificador) la fila correspondiente, la información por los hilos de bit (I_1 , I_2 , I_3) y un 1 por el hilo de escritura que alcanza a todos los biestables. Únicamente los biestable sensibilizados por el hilo de palabra almacenarán la información.

La operación de **lectura** consiste en enviar un 1 por el hilo de selección que se desea leer y un 1 por el hilo de habilitación de lectura.

Ventajas y Desventajas

“La **ventaja** de la memoria 2D es la sencillez con la cual selecciona, graba y lee información.

La **desventaja** es un crecimiento excesivo a nivel vertical cuando se intenta incrementar la capacidad (nivel diseño).

Esto quiere decir, que los diseños de la arquitectura de computadores se intentan achicar aumentando la capacidad, en este esquema la única posibilidad de achicar es conseguir elementos más chicos que ya no existen.

A mayor cantidad de palabras se produce un crecimiento desmedido.

Aumenta el tiempo de acceso cada vez que se agranda el decodificar de direccionamiento.”

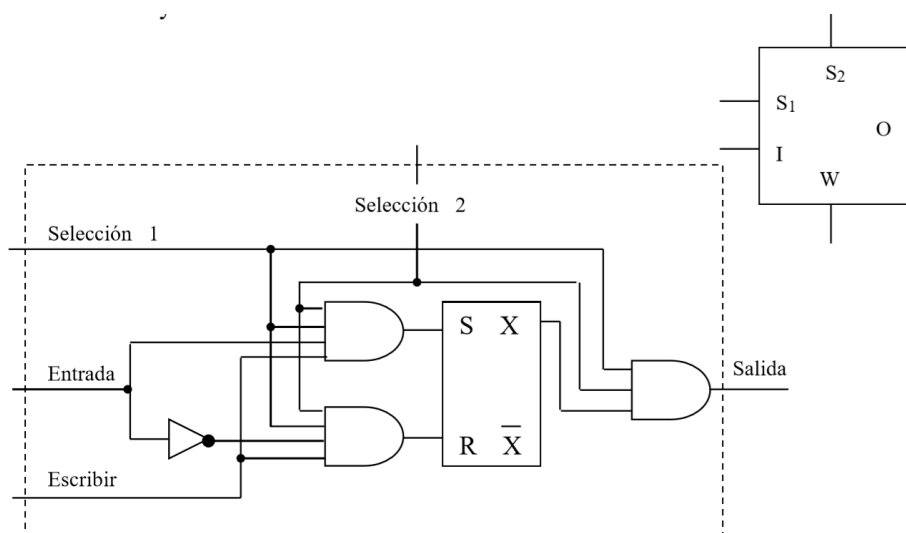
Memoria 3D

Punto de Memoria

“Agrega una entrada de selección más, es decir, una selección del tipo fila-columnas. Un decodificador para filas y otro para columnas.

Se soluciona el problema de la memoria 2D que el decodificador era excesivamente grande y largo, ahora tenemos dos entonces dividimos el problema.”

“Ahora las compuertas AND tienen 4 entradas y se eleva el costo.”

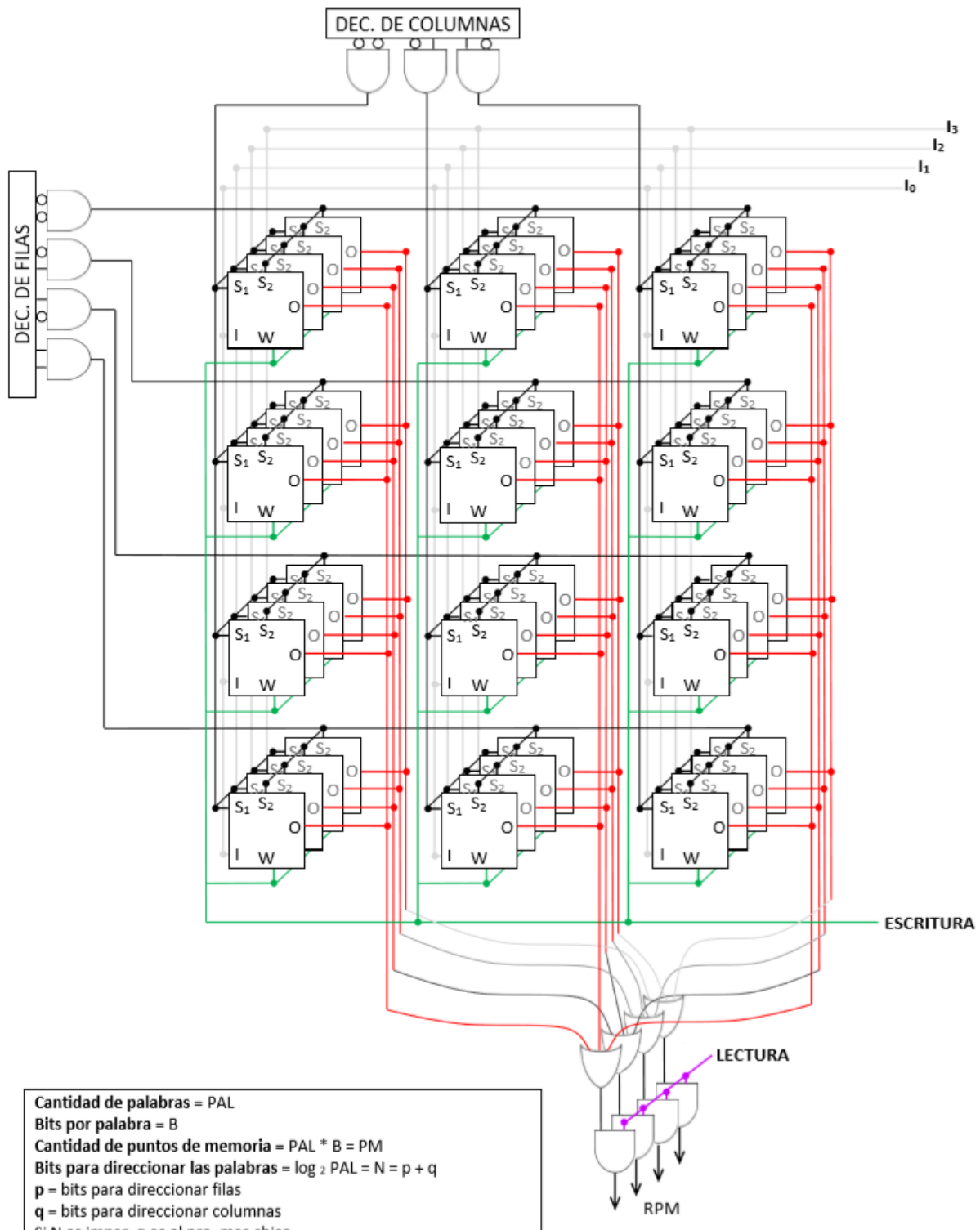


A la celda básica de la memoria 2D le agregamos otra entrada de selección (S1 y S2), ambas deben estar en 1 para seleccionar una celda determinada. Se precisan dos decodificadores para esta memoria, que tiene 16 palabras de 1 bit por palabra. El RDM tiene 4 bits y, por lo tanto, 16 estados. Dos de las entradas del RDM van a un decodificador y dos al otro.

La memoria se divide en m matrices de 2^n biestables, donde m es el número de bits de la palabra de memoria (los distintos planos que componen a la memoria) y n el número de bits del RSM.

- **Aclaración:** En este caso cada punto de memoria representado en el gráfico corresponde a una palabra, debido a que la memoria 3D está compuesta por varios planos, al seleccionar un punto de memoria por ejemplo, el “0000” estamos haciendo referencia a la palabra de memoria que se encuentra en esa dirección y que cada bit de información va a estar en un plano distinto. En cambio, en la memoria 2D toda una fila representaba una palabra; acá la palabra está representada por los distintos planos de una misma dirección.

MEMORIA 3D DE 4x3x4 (12 PALABRAS DE 4 BITS)



Ventajas y Desventajas

1. *La **ventaja** es la longitud de la palabra ya no depende del ancho sino de la profundidad*
2. *Otra **ventaja** es la disminución de la complejidad del decodificador, ya que se divide en dos*
3. *Otra **ventaja** es que mejora el tiempo de acceso debido a que la decodificación de la dirección se distribuye en dos*

La **desventaja** es que aumenta el costo debido a que por cada punto de memoria tenemos 3 entradas más que la memoria 2D, entonces al multiplicar por todos los puntos de memoria que se posee en la memoria aumenta notablemente el costo.

Otra **desventaja** es que aumenta la complejidad de la “calibración” para cuidar que el nivel de integración espacial produzca la mínima temperatura operativa y garantice el normal funcionamiento.”

Memoria 2 y ½ D

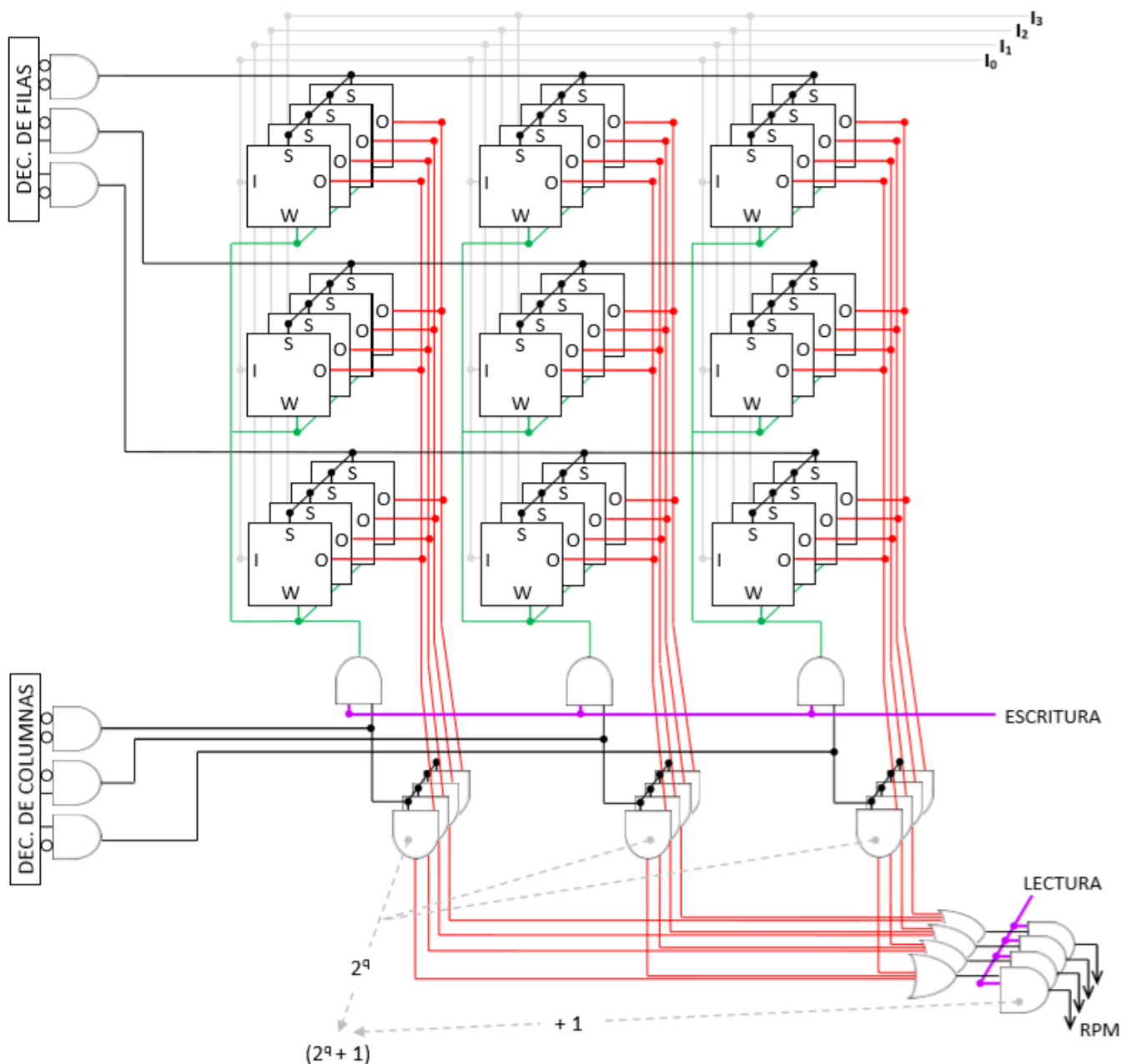
Estas memorias se caracterizan por poseer puntos de memoria 2D y organización 3D (usa 2 decodificadores). El esquema de selección usa un control en las entradas LEER y ESCRIBIR para así lograr la organización 3D. Es también una selección por corrientes coincidentes, con la única diferencia respecto a la selección 3D que la habilitación para escritura no se sitúa a nivel de los biestables, sino al de las líneas de selección de columna.

“Las **ventajas** son:

1. *Aprovecha las ventajas de los diseños anteriores. Es decir, integra puntos de memoria económicos 2D dentro de una organización espacial de arquitectura.*
2. *Levanta menos temperatura que la 3D, sobre todo porque no tiene tantas entradas.*
3. *Es económico.*
4. *Solo tiene 3 entradas por compuerta y una sola compuerta not.*

Una posible **desventaja** es que tiene que agregar compuertas AND, pero no se compara con todas las ventajas que proporciona.”

MEMORIA 2½D DE 3x3x4 (9 PALABRAS DE 4 BITS)



MEMORIA DE NÚCLEOS: Anillo de Ferrita

La ferrita es un material que contiene partículas de óxido de hierro y que tiene una particularidad muy interesante, se puede magnetizar conservando ese estado durante bastante tiempo.

Además, si al elemento de ferrita se le da forma de anillo, algunas de sus características mejoran.

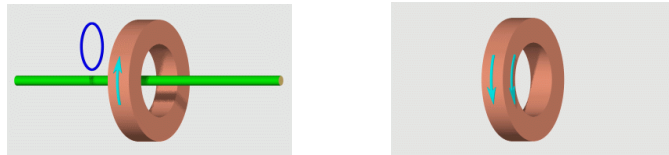
“Antes de los anillos estaban las válvulas, eran extremadamente caras, costosas, levantaban temperatura, tenían problema de consumo. Los anillos de ferrita fueron la salvación. Fueron evolucionando. Por ejemplo, el apolo 11 implementó esto”

“Al ser redondo mantiene mucho tiempo esa carga magnética.”

Escritura

Para actuar sobre la magnetización de un anillo de ferrita se le enhebra un hilo conductor y se hace pasar por él una corriente eléctrica. Al aplicar una intensidad suficiente, el anillo se magnetiza en el sentido del campo magnético inducido y, al suprimir dicha corriente inductora, la magnetización permanece.

“Por el hilo conductor deberíamos aplicar una corriente eléctrica tal que produzca una resistencia sobre el campo magnético que tiene el anillo, de manera tal de orientar todas sus partículas hacia el norte o hacia el sur magnetizándolo positivamente o negativamente.” Dependiendo del sentido para donde gire la corriente en el anillo será un 0 o 1.



Lectura

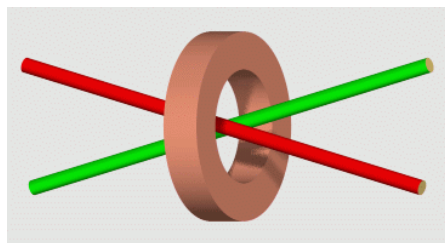
Para averiguar el estado o sentido de magnetización del anillo se enhebra un segundo hilo (que no debe ser paralelo al hilo de escritura para evitar inducciones indirectas entre ambos) que hará las veces de hilo de lectura (“porque tiene otras funciones”).

Para obtener el estado, se aplica una corriente eléctrica que fuerce la magnetización del anillo al estado 0. Si dicho anillo estaba en el estado 1, al pasar al 0 su magnetización cambia bruscamente. Si el anillo estaba en 0, este cambio induce una corriente en el hilo de lectura totalmente apreciable, aunque menor que la necesaria para producir una conmutación (como haría el hilo de escritura). Si el anillo estaba ya en el estado 0, la magnetización apenas cambia y la corriente inducida es mucho menor. Midiendo esta corriente inducida puedo averiguar cual era el estado del anillo.

Este mecanismo de lectura es destructivo, porque al magnetizarlo para obtener su estado siempre produce su puesta a 0. La manera de solucionar esto es aplicando un nuevo pulso de escritura cuando el estado inicial era 1. Así pues, al haber cambiado este estado a 0, lo volvemos a poner en 1 y rápidamente lo dejamos como estaba. “Por esto tiene dos ciclos, el primero lee y el segundo regenera lo que leyó, solamente cuando era 1 sino no hace falta.”

“Para la lectura agregamos un segundo hilo (de lectura) que no tiene que ser paralelo al de escritura para que solamente tenga un punto de inducción. Aplicamos por el hilo de escritura una carga lo suficientemente alta pero negativa para forzar que el núcleo/anillo que estaba en 1 se ponga en 0. Entonces, si el anillo estaba en 1 se ve forzado por la corriente negativa a ponerse en 0, en esa bajada de campo magnético se induce una corriente eléctrica por el hilo de lectura. Si estaba en 1 este baja a 0 pero se resiste, y en esa resistencia magnética sobre el anillo se produce un campo, ese campo produce sobre el hilo de lectura una pequeña corriente inducida que es leída. Al mandar una corriente negativa y el anillo pasar de 1 a 0 tiene que resistirse y esa pequeña tensión que genera es leída por el hilo de lectura. El otro caso es que en el anillo haya un 0 pero al pasar la corriente negativa el anillo va a pasar a tener por ejemplo, -3 y por lo tanto no va a haber una corriente inducida.”

“Si el anillo está cargado con un 1, aplicaría un corriente eléctrica contraria/negativa por el hilo de escritura tratando de llevar el 1 a 0. Por eso se dice que la memoria en la lectura es destructiva, el primer ciclo lee y el segundo tiene que regenerar.”



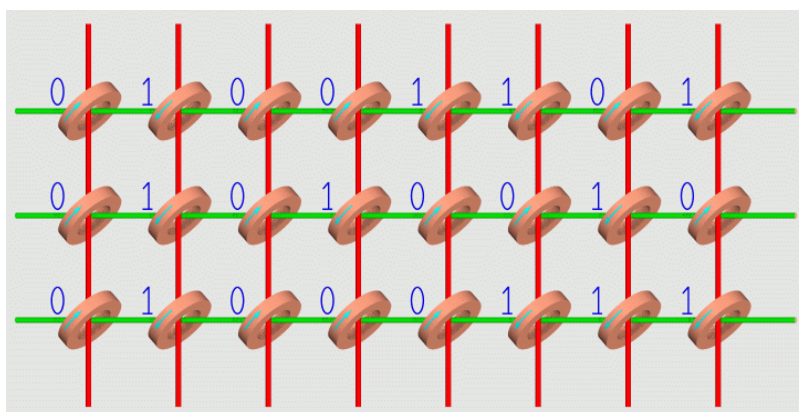
“Problema que se presenta: se necesita un hilo conductor y un hilo de lectura por anillo, es decir, cada anillo necesita dos hilos.” Se necesitan muchos anillos y aún mucho más hilos para realizar una memoria. Una memoria de 1 KB con palabras de 8 bits tendría 8.192 anillos, y con dos hilos por cada anillo necesitaríamos 16.384 hilos con sus 16.384 circuitos para controlarlos.

MEMORIA 2D

“Solución: teoría de las corrientes coincidentes. Vamos a tener dos tipos de hilos, hilos verticales por cada columna e hilos horizontales por cada fila.” (Memoria 2D) Esto reduce notablemente la cantidad de hilos. Con la misma memoria mencionada anteriormente se pasa de 16.384 hilos a 1032 hilos.

En aquellos anillos donde se quiera colocar un 1 se debe mandar una corriente de $\frac{1}{2}$ por toda su fila y además mandar una corriente de $\frac{1}{2}$ por su columna, entonces en aquellos anillos donde coincidan ambas corrientes de $\frac{1}{2}$ se pondrán en 1. Es importante destacar que en aquellos anillos donde solo tendrán una carga de $\frac{1}{2}$ no se llegaran a magnetizar.

El hilo que pasa por todos los anillos de una misma posición (hilo horizontal) se llama **hilo de palabra**, el hilo que pasa por todos los anillos homólogos (hilo vertical) se llama **hilo de bit**. Para una memoria de 1 KB se necesitarían 1024 hilos de palabra y 8 hilos de bit, mucho menos que los 16384.



. Lectura 2D

Se hace pasar una corriente negativa por el hilo de la palabra elegida, que lleve a todos los anillos al estado “0” (lectura destructiva). Los que tienen un 0 siguen igual, y los que tienen un 1 pasan a 0 induciendo una corriente apreciable en su hilo de bit. Pero como esta corriente es menor que la necesaria para forzar una conmutación no afecta el estado de los demás anillos atravesados por el mismo hilo de bit. “Esto quiere decir que cuando por un hilo baja la corriente no es lo suficientemente fuerte como para magnetizar a los demás anillos de la columna sino solamente a aquellos donde coincide la fila y la columna que se mandó la señal.”

“Por el hilo de palabra mandas la corriente para ponerlo a 0; aquellos que se resisten generan una corriente inducida sobre el hilo de bit. Esa corriente inducida sobre el hilo de bit no es lo suficientemente alta como para afectar a los compañeros que están sobre el mismo hilo porque no alcanza para hacerlos conmutar, es una pequeña variación de tensión que indica que hay un 1”.

. Escritura 2D

Primero se pone la palabra en cero. Luego se hace pasar una corriente eléctrica positiva por el hilo de palabra y, simultáneamente, otras iguales por los hilos de bits cuyos anillos hay que poner a "1".

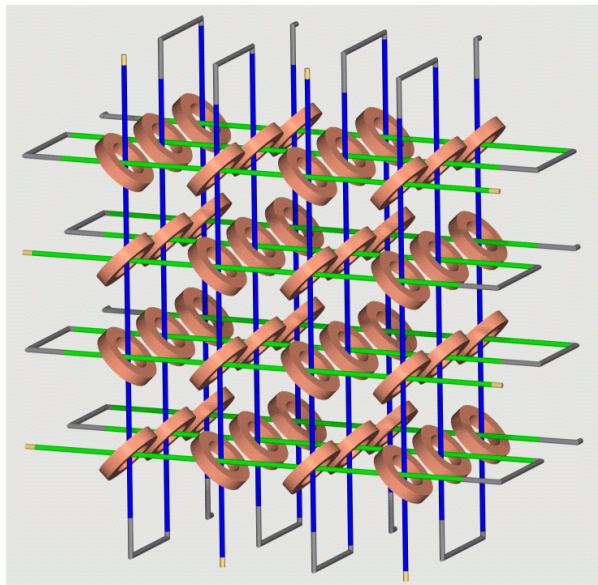
La intensidad de esas corrientes está establecida de forma que una, por si sola, no es capaz de hacer que los anillos que atraviesa se pongan a "1", pero si dos juntas. Los anillos que reciben el efecto combinado de la corriente en el hilo de palabra con la del hilo de bit se pondrán a "1".

"Esta técnica se conoce como transferencia con previa puesta a cero. Para la escritura hay que hacer una previa puesta a cero porque solo se magnetizan los 1 con corrientes coincidentes, no se pueden escribir un cero individual. Esto es porque escribir los ceros llevaría al proceso a ser mucho más complicado, trabajando con corrientes negativas y generar resistencias inversas. Por esto, se colocan todos a cero y se graban solo los uno."

MEMORIA 3D

Con el fin de disminuir aún más la cantidad de hilos la organización 3D plantea superponer planos colocándolos alternados, uno al derecho y otro al revés.

Finalmente, se conectan en serie los hilos de selección homólogos uniéndolos en lados alternos, de modo que se forme una especie de zig-zag con cada hilo. Un decodificador de direcciones X (horizontal) e Y (vertical) logran a través de corrientes coincidentes, seleccionar una palabra espacial. (Ej: tres primeros planos en memoria de 4 bits).



"La palabra está constituida por todos los anillos de la misma posición fila columna y con el mismo ángulo de inclinación. Es necesario que los anillos se encuentren con un ángulo de inclinación porque primero, para enhebrarlos es necesario tenerlos así conmutado y segundo, es necesario para que no haya una corriente inducida entre un plano y otro. Si estuviesen paralelos, podría una palabra inducir a otra en el momento de su conmutación. Entonces, se colocaban a 45 grados opuestos cada plano de manera tal que cada palabra no induzca sobre otra palabra, no hay inducción o disminuye porque no están en el mismo ángulo. "

ORGANIZACIÓN Y BÚSQUEDA DE LA INFORMACIÓN EN MEMORIA

Existen 3 organizaciones o métodos distintos:

1. **Vector, lista y puntero**: La organización más clásica de las informaciones en memoria consiste en almacenarlas sucesivamente, una detrás de otra. Muy generalmente este es el caso de las instrucciones sucesivas de un programa. Necesitará entonces, para buscar las informaciones sucesivas disponer de un **puntero**, cuyo valor sea igual sucesivamente a las direcciones de las informaciones almacenadas en secuencia. En lo que concierne a las instrucciones éste es, evidentemente, el papel del contador de programa. Un conjunto de informaciones almacenadas en secuencia se denomina **vector**.

El método anterior no es muy útil para manipulaciones con datos, como extracción e inserción, porque exigen desplazamiento de la información en memoria. Por esto, se recurre a una **estructura de lista** en que cada dato va acompañado de un puntero, que contiene la dirección del próximo dato. Las operaciones de inserción, combinación, extracción, etc. se llevan a cabo por manejo de punteros.

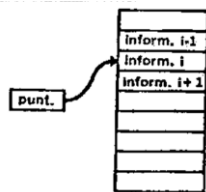


Fig. 11. Estructura de vector

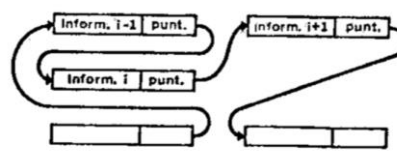


Fig. 12. Estructura de lista

2. **Tabla**: Una tabla establece una correspondencia entre 2 tipos de información: la información significativa que se busca, y la información de entrada que permite localizarla. Esta última se llama también dirección **asociativa o etiqueta**.

La memorización de una tabla puede reducirse a la constitución de un simple vector, si la posición de la información significativa en el vector es calculable a partir de la información de entrada.

La búsqueda en tabla exige sucesivas comparaciones de la información de entrada con todas las posibles informaciones de entrada, hasta llegar a la coincidencia.

3. **Pila y Cola de espera**: Estos dos conceptos designan organizaciones particulares de datos en la memoria, en las que el orden de utilización de las informaciones depende del orden en que han sido introducidas.

La **cola de espera** funciona según un principio análogo al que conocemos en los negocios:

“primero en llegar, primero en salir”. Se necesitan dos punteros para definir el estado de una cola de espera, uno apuntando a la última palabra introducida, el otro a la próxima palabra por extraer.

En el mecanismo de **pila** se accede a la última información almacenada con el esquema: “ultimo en entrar, primero en salir”. El estado de la pila se consigue con un solo puntero, pues basta con conocer el emplazamiento de la cima de la pila.

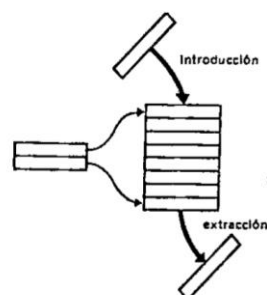


Fig. 13. Mecanismo de la cola de espera:
primero en entrar, primero en salir

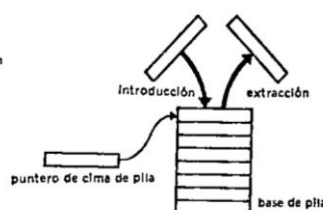
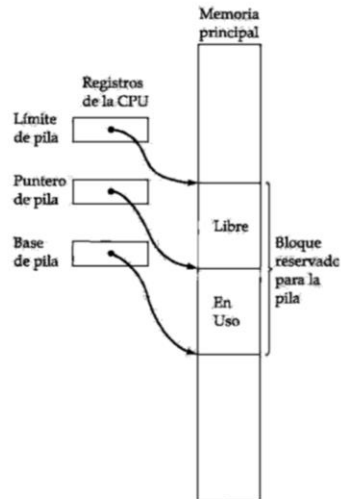


Fig. 14. Mecanismo de la pila:
ultimo en entrar, primero en salir

Para un funcionamiento correcto de la pila se necesitan 3 direcciones, normalmente memorizadas en registros de la CPU: puntero de la pila, base de la pila y límite de la pila.



(a) Toda la pila en memoria

FIGURA 9.14 Organizaciones de pila usuales

DECODIFICADOR

Un decodificador es un circuito combinacional que convierte la información binaria de n entradas a un máximo de 2^n líneas de salida., de las cuales solo una de estas 2^n líneas de salida será seleccionada por el decodificador (es decir tendrá el valor 1). Este circuito particular se llama decodificador de varios a uno, matriz decodificadora, o simplemente decodificador.

El siguiente esquema muestra un decodificador que es completamente paralelo en su construcción y diseñado para decodificar 3 entradas (flip-flops). Tiene $2^3 = 8$ líneas de salidas, y se seleccionara una sola línea de salida para cada uno de los 8 estados que pueden tomar las tres entradas (flip-flop), dicho esto este decodificador opera de la misma forma que el de 2 entradas, explicado en memoria 2D.

El número de compuertas AND es igual al número de líneas de salida, que es igual a 2^n , siendo n el número de FF de entrada.

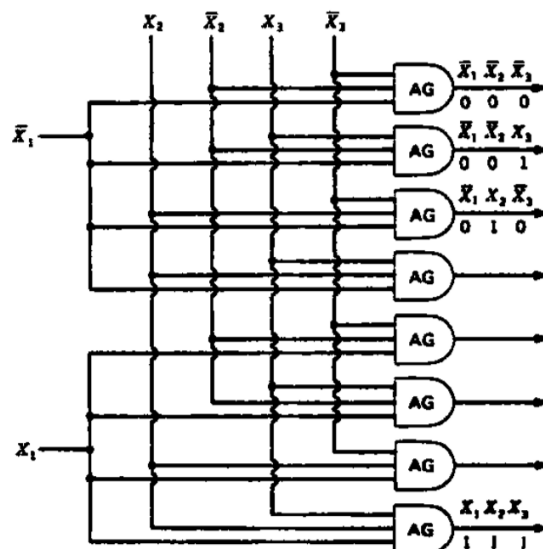


Fig. 7.8. (b) Decodificador paralelo.

BUSES

Un bus es un conjunto de hilos o alambres, en donde las informaciones binarias son mantenidas bajo forma de tensión por uno de los registros fuente. El bus puede alimentar independientemente o simultáneamente a varios registros destinatarios.

Los buses también llamados líneas ómnibus, se utilizan para interconectar varios registros, unos considerados como registros fuentes de información, otros como registros destinatarios de información.

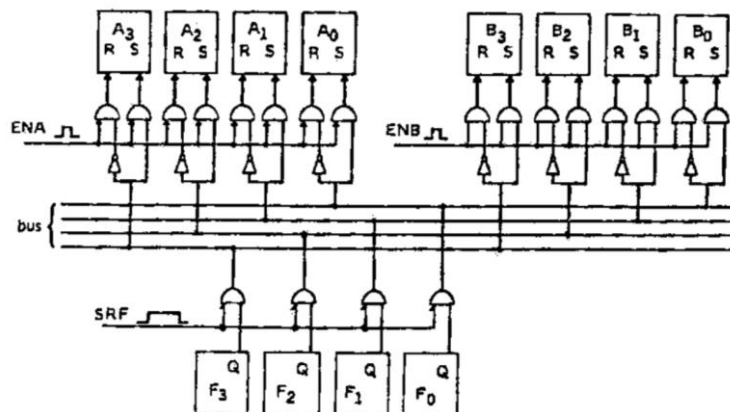


Fig. 71. Transferencia por medio de un bus

Observamos en la figura que una señal de nivel abre las puertas de salida del registro fuente F. En el bus se establecen los niveles de tensión correspondientes a los contenidos de los biestables del registro F y se mantienen mientras este activada la señal SRF. Una vez estabilizados dichos niveles en el bus, el impulso de señal ENB, abriendo las puertas de entrada a los biestables del registro B, fuerza a la introducción en este registro de la información mantenida en el bus por el registro F. Entonces la señal SRF puede volver al valor cero.

A la señal de gobierna EN (ENA, ENB, etc) que valida la información a la entrada del registro se la llama **señal de muestreo**.

Bus con multiplexores y decodificadores

Un sistema de bus común puede construirse con multiplexores y decodificadores. Los multiplexores seleccionan un registro fuente para el bus mientras que el decodificador selecciona un registro de destino para transferir la información desde el bus. La figura muestra un sistema de bus con cuatro registros, donde los 4 bits en la misma posición significativa de los registros pasan a través de un multiplexor de 4 a 1 línea para formar una línea de bus. Para registros de n bits, se necesitan n multiplexores para producir un bus de n líneas. Las n líneas en el bus se conectan a n entradas de todos los registros.

La transferencia en este caso se logra activando la señal de control de carga de ese registro. El control de carga particular activado se selecciona mediante las salidas del decodificador cuando se habilita, por lo tanto, si el decodificador no se habilita no se transferirá ninguna información, aunque los multiplexores coloquen el contenido de un registro fuente en el bus.

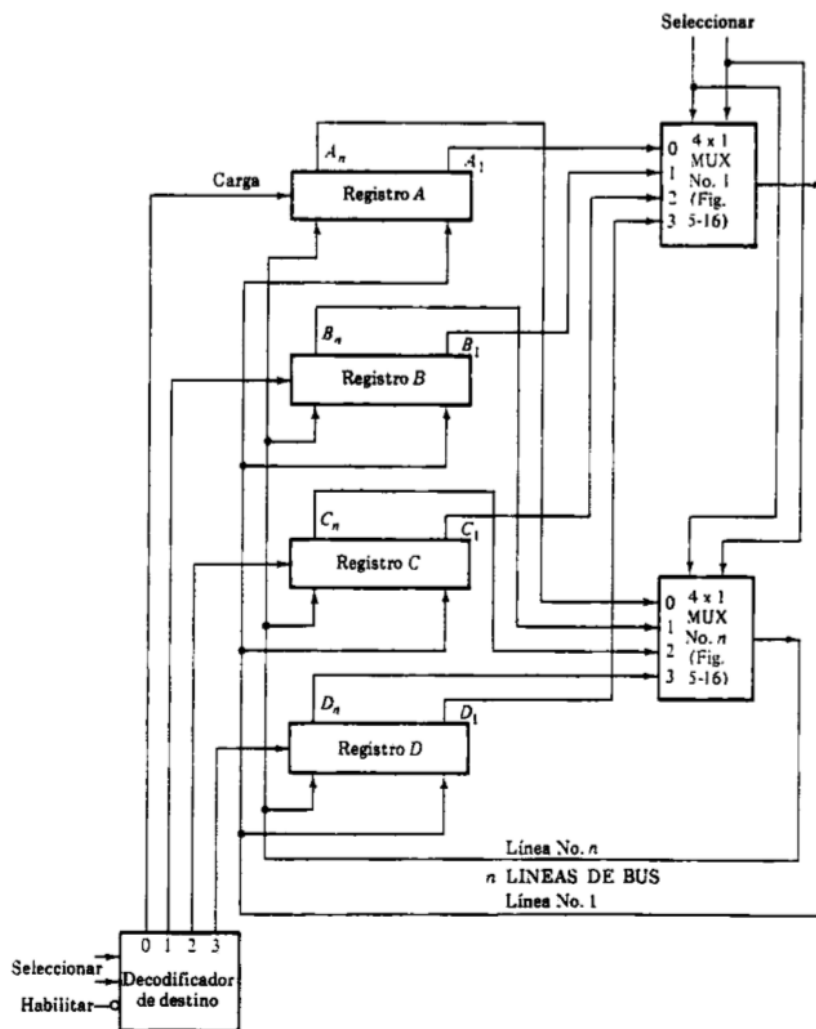


Figura 8-6 Sistema de bus para cuatro registros

Unidad 6

Unidad Aritmética Lógica

Es manejada por la UC, ejecuta las operaciones con los datos almacenados en la memoria principal.

Para pedir al ordenador una operación aritmética, por ejemplo una suma, la instrucción debe facilitarle las siguientes informaciones:

1. La clase de operación a realizar, se denomina **código de operación (CO)**.
2. La dirección de la palabra de memoria que contiene el primer dato, o primer operando.
3. La dirección de la palabra de memoria que contiene el segundo operando.
4. La dirección de la palabra de memoria donde quiere almacenarse el resultado.
5. La dirección de la palabra de memoria de la siguiente instrucción.

Existen 5 **formatos de instrucciones**:

a. Instrucciones de 4 direcciones

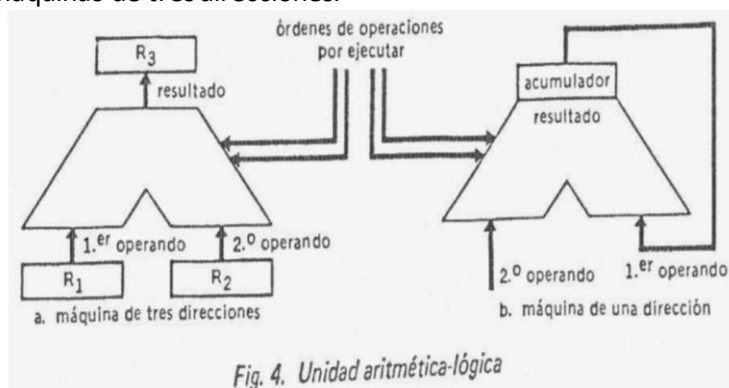
Código de Operación	Dirección 1 ^{er} Operando	Dirección 2 ^{do} Operando	Dirección Resultado	Dirección Siguiente Instrucción
---------------------	------------------------------------	------------------------------------	---------------------	---------------------------------

b. Instrucciones de 3 direcciones

Código de Operación	Dirección 1 ^{er} Operando	Dirección 2 ^{do} Operando	Dirección Resultado
---------------------	------------------------------------	------------------------------------	---------------------

Cuando se utiliza este formato de instrucción se supone que la siguiente instrucción se encuentra almacenada en la dirección de memoria siguiente a la que se está ejecutando.

La figura a representa una ALU capaz de ejecutar la operación anterior (de tres direcciones), la cual está rodeada de 3 registros donde se memorizan los dos operandos y el resultado. La instrucción de suma necesita cuatro accesos a la memoria central, que permiten sucesivamente buscar la instrucción, después el 1er operando, después el 2do operando y por último, almacenar el resultado. A las máquinas que utilizan este tipo de instrucción se les llama máquinas de tres direcciones.



c. Instrucciones de 2 direcciones

Código de Operación	Dirección 1 ^{er} Operando	Dirección 2 ^{do} Operando
---------------------	------------------------------------	------------------------------------

En las máquinas que utilizan este formato de instrucción el resultado generalmente se guarda en la dirección del primer operando. Es importante que el programador tenga en cuenta esto ya que ese operando se perderá.

d. Instrucciones de 1 dirección

Código de Operación	Dirección del Operando
---------------------	------------------------

Abacus es una máquina de una dirección. Su ALU posee un registro particular denominado Acumulador, que contiene tanto el 1er operando como el resultado. Esta característica permite instrucciones de una sola dirección: la del segundo operando.

En estas máquinas la operación de SUMA necesita 3 instrucciones para:

1. Cargar el primer operando en el Acumulador.
2. Sumar el segundo operando con el contenido del Acumulador.
3. Almacenar en memoria central el contenido del Acumulador.

e. Instrucciones sin dirección

Este caso se presenta fundamentalmente en las máquinas con MICROPROCESADOR en las cuales la longitud de la palabra es generalmente de 1 byte y se ocupa íntegramente para el código de operación, la dirección del operando se encuentra en la dirección o direcciones siguientes al de la instrucción.

CLASIFICACIÓN DE LOS OPERADORES ARITMÉTICOS

La ALU se compone de una unidad capaz de ejecutar todo el surtido de instrucciones del calculador o de varias unidades funcionales u operadores, cada uno especializado en la ejecución de una o varias clases de operaciones.

Los operadores pueden clasificarse en función del carácter combinacional o secuencial de su concepción. Los **operadores combinacionales** ejecutan las operaciones en una sola fase y por esto pueden ser gobernados por señales de nivel, que permanecen activas durante toda la operación. Los **operadores secuenciales** ejecutan las operaciones en varias fases gobernadas por impulsos distribuidos por un dispositivo de control (puede ser la UC o un órgano del operador). Los operadores constan entonces de dispositivos de memorización de los resultados parciales. Una solución intermedia consiste en disponer de un operador combinacional asociado a un registro acumulador o totalizador, que sirve para la memorización del primer operando durante la operación y para la memorización, después.

Habitualmente se asocian al operador aritmético indicadores que suministran informaciones acerca de la última operación realizada. Se encuentran indicadores de error: desbordamiento o sobrepasamiento de capacidad, división por cero, etc. e indicadores del estado del acumulador: positivo o negativo, igual a cero, etc. Estos indicadores constituyen lo que se ha dado en llamar código de condición, porque representan informaciones a verificar con ocasión de las instrucciones de salto condicional.

Todos los otros elementos del sistema computacional (UC, registros de memoria, E-S, etc.) llevan datos a la ALU para que esta los procese y luego devuelva los resultados. Está basada en el uso de

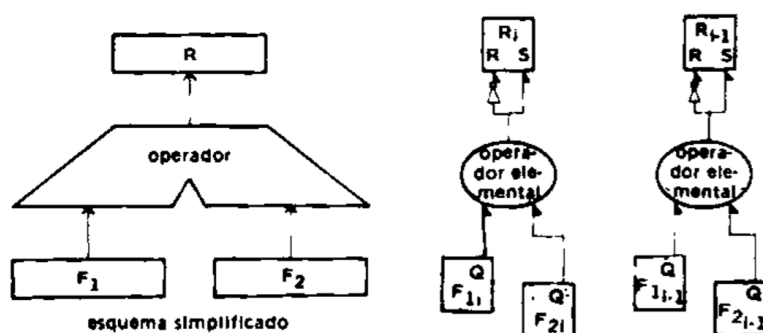
dispositivos lógicos digitales simples que pueden almacenar dígitos binarios y ejecutar operaciones de lógica simple de Boolean.

Los datos se presentan a la ALU en registros, y los resultados de una operación se almacenan en registros. Éstos son localidades de almacenamiento temporal dentro de la CPU que están conectados por vías de señal a la ALU. La ALU también pondrá banderas como resultado de una operación; los valores de las banderas se almacenan también en registros dentro de la CPU. La unidad de control proporciona señales que controlan la operación de la ALU, y el movimiento de los datos hacia dentro y hacia fuera de la misma.

UNIDAD ARITMETICO-LOGICA ELEMENTAL

Para el caso de las operaciones entre bits del mismo peso, se encuentran dos montajes posibles de los operadores:

1. **Operador Combinacional:** Los operadores se montan entre dos registros fuentes F1 y F2 (o entre dos buses fuentes) para los operandos y un registro R para el resultado (o un bus para el resultado). (Obsérvese que, sobre este esquema, el resultado solo es válido en tanto los operandos fuentes permanezcan posicionados).



Operador Combinacional

2. **Operador con Acumulador:** no exigen más que un registro fuente para almacenar uno de los operandos, ya que el otro es memorizado por el acumulador durante todo el tiempo de la operación. Cuando a la salida de los operadores elementales quedan establecidos los niveles lógicos, un impulso EAC introduce el resultado en el acumulador.

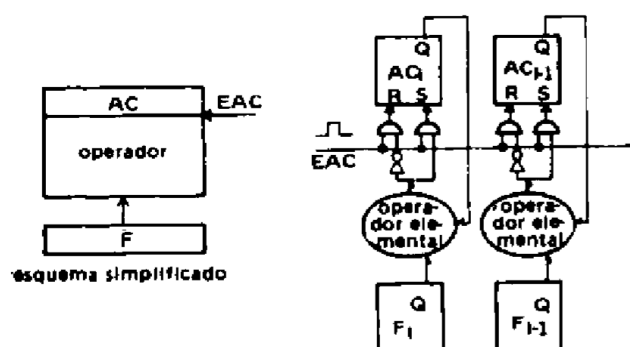


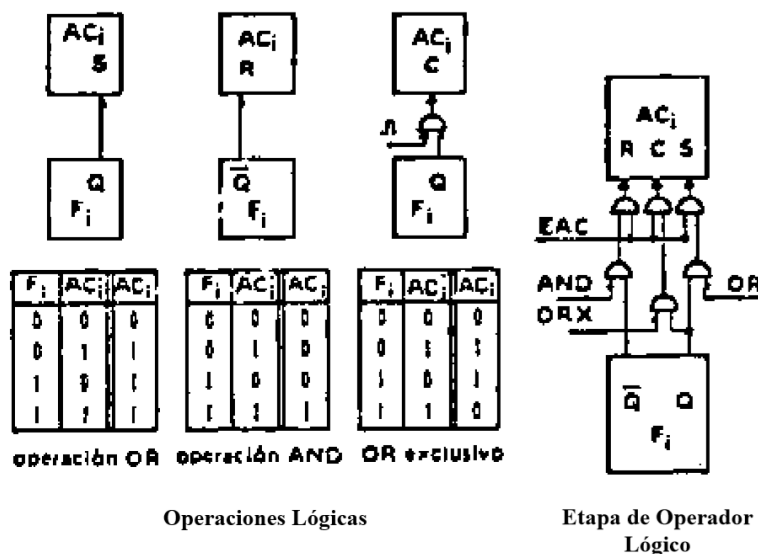
Fig. 2. Operador con acumulador

OPERACIONES LOGICAS

La complementación en el operador combinacional se consigue por circuitos NOT y en el caso del operador con acumulador con la complementación del acumulador; las demás operaciones se reducen a operaciones AND, OR y OR exclusivo.

1. **Operador Combinacional:** Utiliza el mismo esquema del operador combinacional visto en el tema anterior pero sustituyendo los operadores elementales por circuitos AND, si se trata de la operación intersección, OR si se trata de la operación de reunión, OR exclusivo si se trata de la operación OR exclusivo.

2. **Operador con Acumulador:** La organización presentada con la figura del operador con acumulador anterior puede simplificarse, si se parte de la idea siguiente: en vez de ejecutar la operación con los bits homólogos del registro fuente y del acumulador, se observa que basta, según el valor del bit fuente F_i , con dejar inalterado el bit del acumulador o forzarle a un valor no dependiente más que de F_i . Por ejemplo: para el AND, el bit acumulador continúa inalterado si $F_i = 1$, pero debe ponerse a cero si $F_i = 0$.



OPERACIONES DE DESPLAZAMIENTO

El operador de desplazamiento puede estar constituido únicamente por compuertas lógicas, en cuyo caso posee un carácter puramente combinacional.

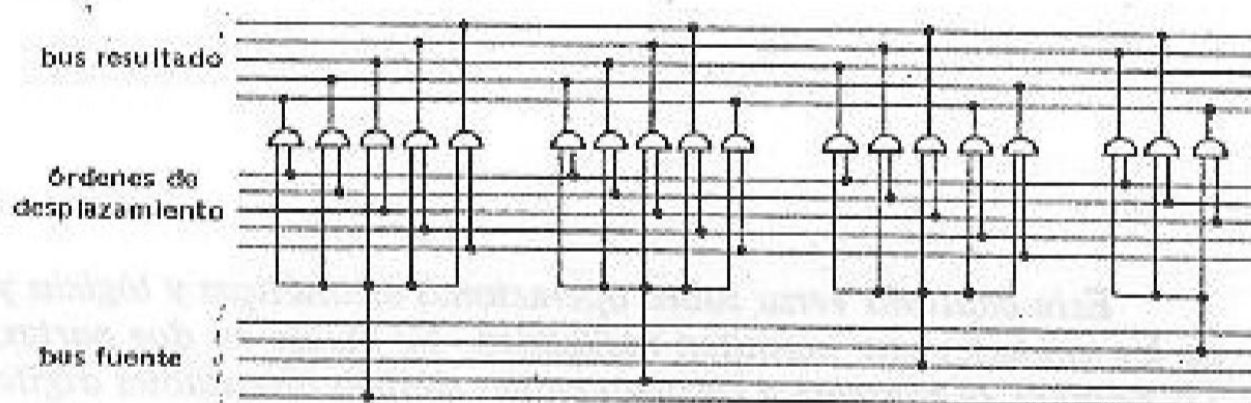
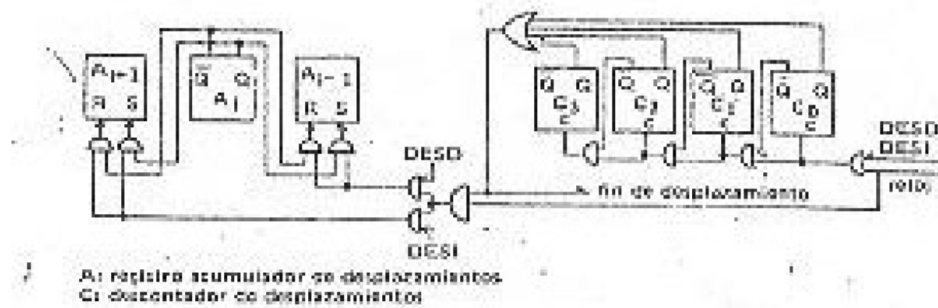


Fig. 5. Operador combinacional de desplazamiento

También puede montarse el acumulador como un registro de desplazamiento. En los operadores de las máquinas pequeñas no puede ejecutarse generalmente más que un desplazamiento elemental (una posición a la derecha o a izquierda) por impulso. A fin de no tener que utilizar una instrucción para cada desplazamiento elemental, se emplea un descontador de desplazamiento, cargado al principio de la operación con el número de desplazamientos por efectuar, número que aparece como operando inmediato en la instrucción. Este contador se ve descontado en una unidad a cada impulso de orden de desplazamiento; la operación se detiene cuando el descontador de desplazamientos llega a cero.

Es un primer ejemplo de un operador secuencial y de su dispositivo de control.



A: registro acumulador de desplazamientos
C: contador de desplazamientos

ADICION Y SUSTRACCION BINARIAS

Concepto de sumador binario en paralelo

Un sumador binario en paralelo está fundamentalmente constituido por una cadena de etapas de sumadores, en que la etapa de sumador de rango i transmite a la de rango $i+1$ el arrastre eventual. La etapa del rango 0 puede estar formada solamente por un semisumador. El arrastre de la etapa de mayor peso indica desbordamiento si está activada.

Sumador paralelo cableado entre un registro fuente y un acumulador:

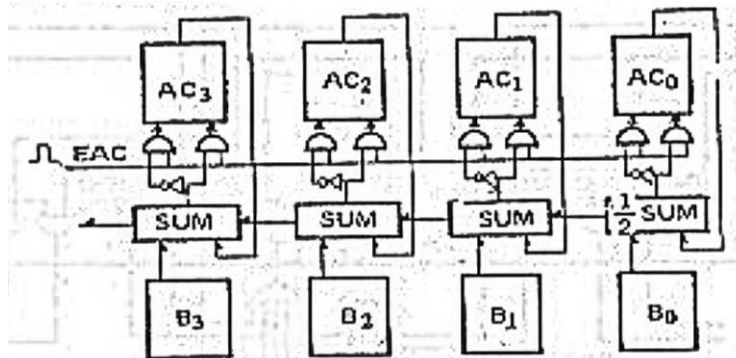


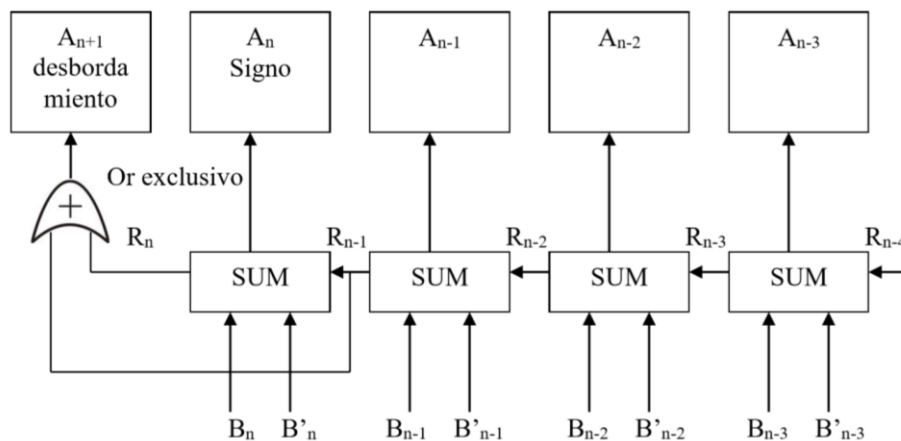
Fig. 7. Sumador paralelo

Adición y sustracción de números algebraicos binarios

Sustraer un número de otro equivale a sumarle su opuesto. Entonces, la sustracción se reduce a la suma algebraica siempre que se sepa calcular el opuesto de un número (generalmente se hace por complementación). Este método evita tener que situar un sustractor junto al sumador.

Limitaremos nuestro estudio de la suma de dos números algebraicos al caso de la representación por complemento a 2.

Supondremos que nuestra unidad aritmética procesa números representados por n bits, con el bit de signo situado en la primera posición a la izquierda. Asociaremos al acumulador un $(n+1)$ -simo bit, que actuará como indicador de desbordamiento.



Para estudiar la operación de suma algebraica, elaboraremos un cuadro a tres entradas correspondientes a las tres configuraciones posibles de signo de los operandos y considerando el arrastre n-1 procedente de la etapa inmediatamente a derecha del bit de signo. La presencia de este arrastre implica un desbordamiento en el caso de un resultado esperado positivo; su ausencia implica un desbordamiento en el caso de un resultado esperado negativo.

Se observa que el indicador de desbordamiento debe posicionarse solamente en dos casos:

- Cuando no hay arrastre de orden n-1 pero hay arrastre de orden n
- Cuando hay arrastre de orden n-1 pero no de orden n

Condiciones de Signo	2 operandos > 0; el resultado debe ser positivo	1 operando > 0; 1 operando < 0; No puede haber desbordamiento	2 operandos < 0; el resultado debe ser negativo
Representación inicial (registro 5 bits)	0.... 0....	1.... 0....	1.... 1....
Resultado si no hay arrastre procedente de la etapa n-1	0.... Valido: resultado positivo.	1.... Valido: resultado negativo.	10.... No valido puesto que resultado positivo: desbordamiento
Resultado si hay arrastre procedente de la etapa n-1	1.... No valido puesto que hay resultado negativo: desbordamiento.	10.... Valido olvidándose del arrastre: resultado positivo.	11.... Valido olvidándose del arrastre: resultado negativo.

ALU PARA ABACUS

La unidad aritmética de Abacus está montada entre un bus fuente M, donde se mantienen los niveles lógicos representativos del segundo operando, y un acumulador AC que mantiene los niveles lógicos correspondientes al primer operando hasta que la señal de muestreo EAC introduce el resultado en el acumulador.

Consta de un sumador en paralelo y de un conjunto de puestas para ejecutar las operaciones lógicas y distribuir las informaciones de acuerdo con las operaciones aritméticas a efectuar.

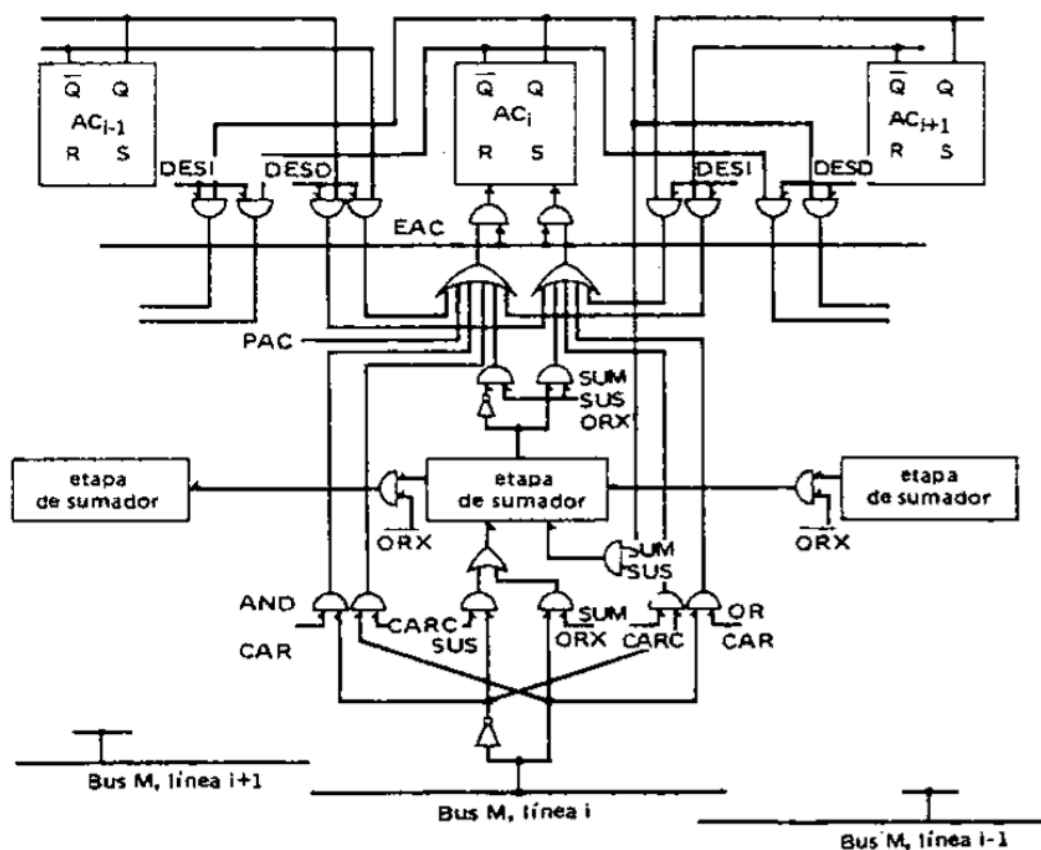
Además, el acumulador está montado como registro de desplazamiento.

Las diferentes operaciones están gobernadas por señales lógicas, procedente normalmente del generador central de secuencias del ordenador. Todas estas señales son de nivel, de tal suerte que al cabo de un cierto tiempo después del posicionamiento el resultado de la operación se estabiliza bajo la forma de niveles lógicos, que serán introducidos en el acumulador por el impulso EAC. En la figura siguiente se representa el i-simo elemento de esta unidad.

Las diferentes **operaciones** que permite son:

CAR	Carga: transferencia al acumulador de la información presente sobre el bus M.
CARC	Carga con complementación: el mismo proceso, pero después de complementar cada dígito.

SUM	Adición: suma al contenido del acumulador, la información presente sobre el bus M. Esta adición es operada por el sumador en paralelo y la duración de la misma es el tiempo de propagación de los arrastres.
SUS	Sustracción: se resta la información en el bus M del contenido del acumulador. Se realiza complementando la información del bus M y después sumándola en el sumador paralelo. La operación es correcta si Abacus opera en complemento a 1. Si no, se precisa posicionar un nivel 1 a la entrada de arrastre a la etapa de sumador de menor peso
AND	Intersección lógica: entre la información en el bus M y el contenido del acumulador.
OR	Reunión lógica: entre la información en el bus M y el contenido del acumulador. Estas dos últimas operaciones se realizan según el esquema de la etapa de operador lógico.
ORX	OR exclusivo: entre la información en el bus M y el contenido del acumulador. Esta operación se ejecuta aquí por parte del sumador, inhibiendo la propagación de los arrastres.
DESDI	Desplazamiento a izquierda: desplazamiento del contenido del acumulador una posición binaria a la izquierda.
DESD	Desplazamiento a derecha: desplazamiento del contenido del acumulador una posición binaria a la derecha.
DEB	Indicador de desbordamiento.
S	Indicador de signo.
CE	Indicador de cero. Puede cablearse mediante un NOR recibiendo todas las salidas Q de los biestables del acumulador, si Abacus trabaja en complemento a 2. (En complemento a 1 se precisaría un OR detrás de un NOR para todas las salidas Q y de un NOR para todas las salidas Q').



Elemento de la unidad lógica-aritmética de Abacus

MULTIPLICACIÓN Y DIVISIÓN BINARIAS

Multiplicador Secuencial por suma - Desplazamiento

Hay que mencionar que aquí nos limitaremos a las operaciones con números en valores absolutos, dejando aparte el tratamiento de los signos.

Consideremos el siguiente ejemplo numérico:

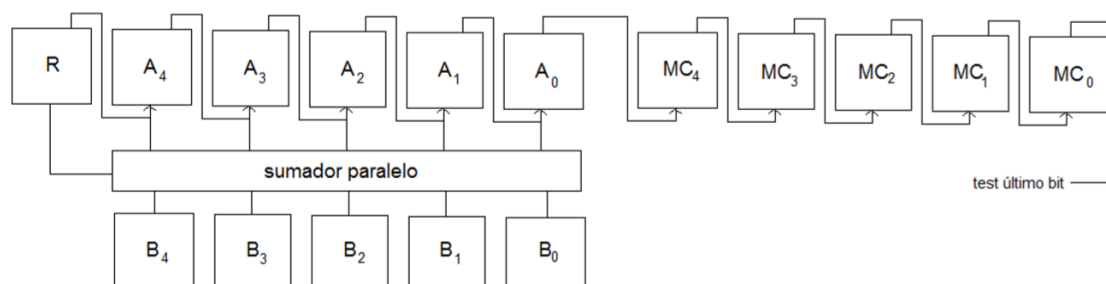
$$\begin{array}{r}
 11001 \\
 10101 \\
 \hline
 11001 \\
 00000 \\
 11001 \\
 00000 \\
 11001 \\
 \hline
 1000001101
 \end{array}$$

11001 ~ multiplicando
 10101 ~ multiplicador
 productos parciales
 producto

La multiplicación de enteros binarios sin signo se realiza igual que cuando utilizamos papel y lápiz. Se pueden hacer varias observaciones:

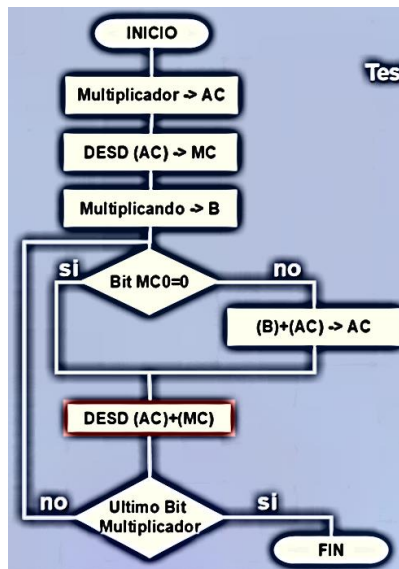
- La multiplicación implica la generación de productos parciales, uno para cada dígito del multiplicador. Estos productos parciales se suman después para producir el producto final.
- Los productos parciales se definen fácilmente. Cuando el bit del multiplicador es 0, el producto parcial es 0. Cuando el multiplicador es 1, el producto parcial es el multiplicando.
- El producto total se obtiene sumando los productos parciales.
- El producto de dos enteros binarios sin signo de n bits da como resultado un producto de $2n$ bits de longitud. Es decir, el resultado se obtiene en doble longitud.

Hay varias modificaciones que se le pueden hacer para efectuar la operación más eficientemente. En primer lugar, podemos realizar una suma progresiva de los productos parciales en lugar de esperar hasta el final. Esto evita la necesidad de almacenar todos los productos parciales, necesitándose menos registros. En segundo lugar, podemos ahorrar algún tiempo en la generación de los productos parciales. Para cada 1 del multiplicador se requiere un desplazamiento y una suma; pero por cada 0, solo se necesita desplazamiento.



La operación se realiza de la siguiente manera:

1. Carga del multiplicador en el acumulador.
2. Desplazamiento a derecha del contenido del conjunto formado por el acumulador y el registro MC, cuyo resultado es poner 0 en el acumulador y el multiplicador en MC.
3. Carga del multiplicando en B (que, eventualmente, pudiera ser sustituido por el RPM).



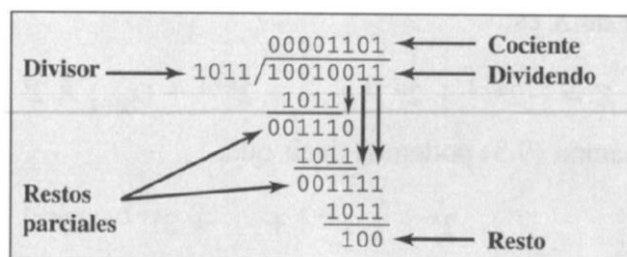
Técnicas de multiplicación rápida

Dos vías principales permiten aumentar, por medios lógicos, la velocidad de la multiplicación.

1. Basándose en la **técnica tradicional de suma-desplazamiento** y disminuyendo el número de operaciones parciales sucesivas:
 - a. Caso de sucesión 0
 - b. Caso de sucesión 1
2. Realizando un **operador celular para una multiplicación** casi paralela.

División por Sustracción y Desplazamiento

Para efectuar una división tomaremos de nuevo la unidad aritmética utilizada con la multiplicación (casi siempre es común la unidad para ambas operaciones) pero transformamos el sumador en sustractor, invertimos el sentido del desplazamiento y añadiremos el dispositivo de comparación. Siempre considerando que los números son sin signo.



Primero se examinan los bits del dividendo de izquierda a derecha hasta que el conjunto de bits examinados represente un número mayor o igual que el divisor; o, en otras palabras, hasta que el divisor sea capaz de dividir el número. Hasta que eso ocurre, se van colocando ceros en el cociente de izquierda a derecha. Cuando dicho evento ocurre, se coloca un 1 en el cociente, y se subtrae el divisor del dividendo parcial. Al resultado se le denomina resto parcial. Desde este punto en adelante, la división sigue un patrón cíclico. En cada ciclo, se añaden bits adicionales del dividendo al resto parcial hasta que el resultado sea mayor o igual que el divisor. Como antes, de este número se resta el divisor para producir un nuevo resto parcial. El proceso continúa hasta que se acaban los bits del dividendo.

Las **operaciones** consisten en:

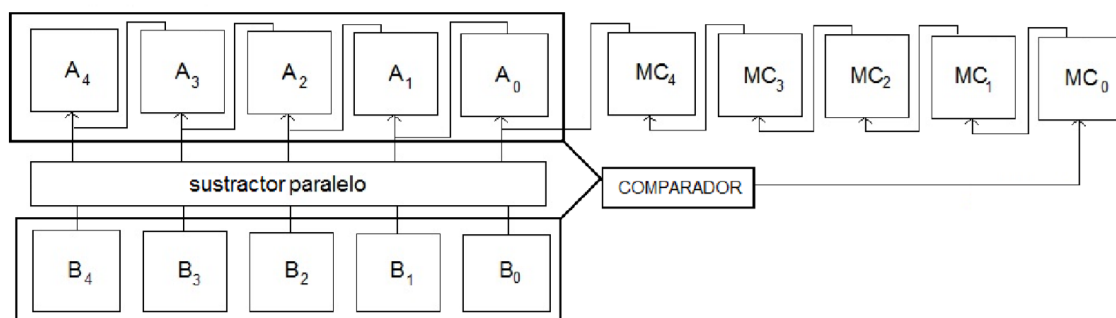
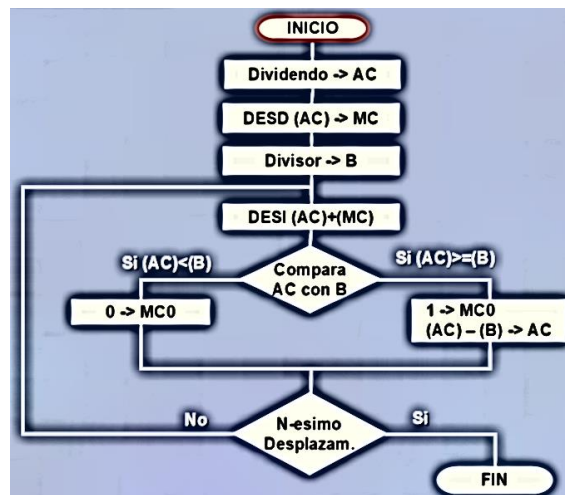
1. Carga del dividendo en el acumulador
2. Desplazamiento a derecha del contenido del conjunto acumulador + MC, con objeto de poner a cero el acumulador y cargar el dividendo en el MC.
3. Carga del divisor en B.

Luego, se realizará un desplazamiento a izquierda del conjunto acumulador + MC, lo que tiene por consecuencia poner el bit de mayor peso del dividendo en A0 y liberar el biestable MC0.

Se compara el contenido del acumulador con el contenido del registro B.

- Si $(AC) < (B)$, se pone MC0 a cero sin tocar el acumulador
- si $(AC) \geq (B)$, se resta del acumulador el divisor, se pone MC0 a 1, etc.

Al final de la operación, el registro multiplicador cociente alberga al cociente y el resto aparece en el acumulador.

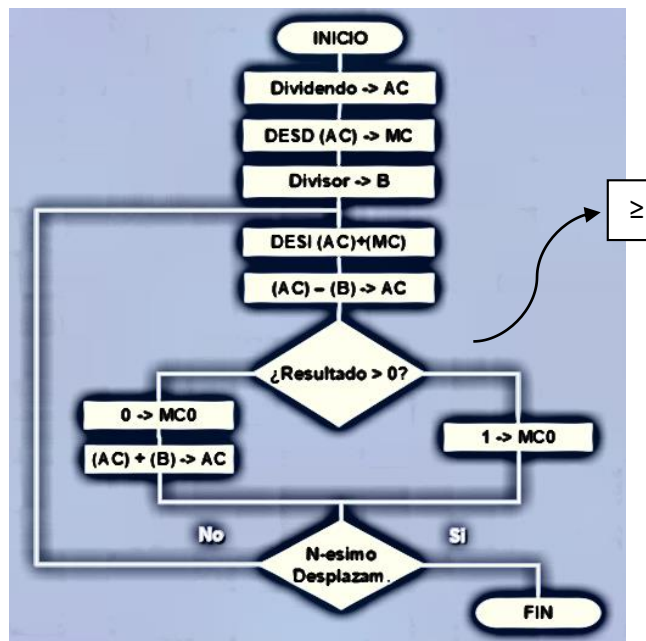


División con Restauración

En el punto anterior hemos dado por supuesto que contábamos con un dispositivo para comparar directamente los valores numéricos de los contenidos del acumulador y del registro B. En realidad, casi nunca se cuenta con él por razones de precio, por lo que puede hacerse la comparación intentando restar el divisor del dividendo y comprobando el signo del resultado. Este es, además, el procedimiento que empleamos cuando nos vemos obligados (desgraciadamente) a operar una división sin máquina ni regla de cálculo. Pero, al contrario de lo que sucede en el papel, la unidad aritmética pierde el antiguo valor del dividendo cuando el intento no surte efecto. En tal caso, es necesario restaurarlo, añadiendo el divisor al dividendo antes de hacer un nuevo intento, de ahí el algoritmo de la división con restauración.

En este caso lo que se hace es realizar la resta "a la fuerza", es decir, sin saber si es correcto o no realizarla. Pero en caso de que no fuera correcto se coloca un 0 en MC0 y se restaura todo. En caso contrario, como ya se realizó la resta y está bien se coloca un 1 en MC0 para indicar que se pudo.

La pregunta de si es negativo o no se realiza por una instrucción de máquina llamada SAN (Salto condicional por signo negativo). No hay que comparar nada es una instrucción que existe y realiza esto. Rompe la secuencia del programa, realiza un salto si es negativo.



División sin Restauración

Corresponde a un algoritmo que permite ahorrar la fase de restauración después de una sustracción con resultado negativo, lo que supone una ganancia de tiempo.

ARITMETICA BINARIA EN COMA FLOTANTE

“No existen ALU que trabajen con coma física. Porque el hecho de poner una coma física significa romper con la cadena numérica y además en las operaciones, hay que desplazar cada número hasta que queden las comas alineadas (suma, resta). Entonces la idea es trabajar con números enteros que tengan escondidos la posición donde iría la coma; y ese es el exponente en la notación científica.”

“Con un esquema del tipo signo |mantisa| exponente se puede indicar con el exponente donde debería estar la coma sin necesidad de ponerla físicamente. Para implementar esto, se trabaja con un acumulador de números enteros pero lo que se cambia es el exponente que indica donde va la coma.”

Esta representación consiste en representar los números bajo la forma: $SM \times B^E$ donde

- S: signo del numero
- M: mantisa del numero
- E: es el exponente del numero
- B: base del sistema de numeración. La base está implícita y no necesita memorizarse ya que es la misma para todos los números.

Por lo tanto, la representación de los números flotantes binarios en la maquina es:

S	Exponente	Mantisa
---	-----------	---------

Esta notación es corriente en la escritura cuando representamos 125.000.000 por 125×10^6 . Tal representación no es única, puesto que también puede escribirse: $12,5 \times 10^7$, 1250×10^5 , etc. De entre estas representaciones se selecciona una, llamada **normalizada**, donde su exponente está ajustado para que la mantisa tenga el mayor número de cifras significativas. Para normalizar un número se desplaza su mantisa hacia la izquierda o derecha según corresponda hasta que el primer dígito sea significativo, y se reduce o aumenta el exponente, respectivamente, en un número igual al número de desplazamientos que han sido necesarios.

Para simplificar los cálculos con números en coma flotante se requiere usualmente que estén normalizados. En un número normalizado, el dígito más significativo de la parte significativa es distinto de cero. Por lo tanto, para la representación en base 2, el bit más significativo de la parte significativa de un número debe ser uno. Por lo tanto, un **número normalizado en base dos** tiene la forma:

$$\pm 0,1bbb \dots b \times 2^{\pm E}$$

en donde cada b es un dígito binario (1 ó 0.)

Estándar IEEE – representación binaria coma flotante

La representación en coma flotante más importante es la definida en la norma o estándar 754 del IEEE, y adoptada en 1985. Este estándar se desarrolló para facilitar la portabilidad o transferencia de los programas de un procesador a otro y para alentar el desarrollo de programas numéricos sofisticados. Este estándar ha sido ampliamente adoptado y se utiliza prácticamente en todos los procesadores aritméticos actuales.

El estándar del IEEE define un formato simple con 32 bits con exponente de 8 bits, y un formato doble con 64 bits y exponente de once bits. La base implícita es dos.

Además define dos formatos ampliados, simple y doble, cuya forma exacta depende de la implementación (puede variar de un procesador a otro). Estos formatos ampliados incluyen bits adicionales en el exponente y en la mantisa. Los formatos ampliados se utilizan para cálculos intermedios. Con su mayor precisión, dichos formatos reducen la posibilidad de que el resultado final se vea deteriorado por un error excesivo de redondeo; con su mayor intervalo de variación también reduce la posibilidad de un desbordamiento intermedio que aborta un cálculo.

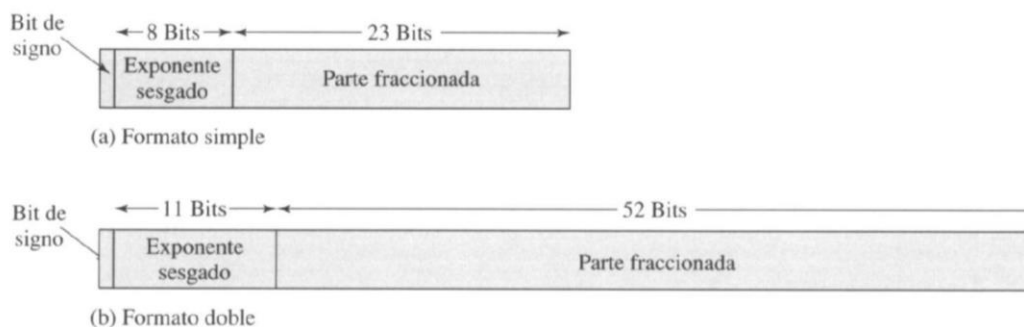


Figura 9.21. Formatos IEEE 754.

Suma y Sustracción en coma flotante

La **primera fase** consiste en *alinear las mantisas* con el objeto de enfrenar los bits del mismo peso. Supongamos a sumar los números $A = 12.000 \times 10^{-1}$ y $B = 80.000 \times 10^2$:

A								B						
+	0	0	1	2	0	0	y	+	2	8	0	0	0	0

No puede operarse directamente la suma de las mantisas, se comparan los exponentes y se detecta que el primero de los números es más pequeño, sobre este número se ejecuta una doble operación: incremento del exponente en una unidad y desplazamiento de la mantisa una posición a la derecha. De nuevo se comparan y se recomienza hasta obtener identidad de exponentes. En ese momento puede efectuarse la suma de las mantisas:

A	+	2	0	0	0	1	2
B	+	2	8	0	0	0	0

La **segunda fase** consiste en *ajustar los signos*. Consideremos la sustracción de $A - B$. En la representación en valor absoluto y signo, se deben primeramente comparar las mantisas A y B, comprobar que la de B es superior a la de A y deducir de ello que la operación pertinente es sustraer la mantisa de A de la de B y al resultado darle el signo menos.

A	+	2	0	0	0	1	2
- B	+	2	8	0	0	0	0

		M_B					
		8	0	0	0	0	0
		$-M_A$					
		0	0	0	1	2	
R	-	2	7	9	9	8	8

Debe tomarse así en cuenta en esta fase la clase de operación (suma o sustracción), el signo de los operandos y el resultado de la comparación de mantisas. El problema se simplifica si el conjunto signo-mantisa fuera representado en complementación cuando el número sea negativo.

La **tercera fase** es la *normalización del resultado*. Puede que el resultado bruto de la operación no esté normalizado por dos razones:

1. Hay desbordamiento de capacidad en la suma, en cuyo caso se deberá desplazar la mantisa una posición a la derecha, aumentando al mismo tiempo en una unidad el exponente.

1er operando:	+	2	8	0	0	0	0
2do operando:	+	2	8	0	0	0	0
Operación:	1	6	0	0	0	0	0
Normalización:	+	3	1	6	0	0	0

2. La diferencia de las mantisas implica un número de n ceros a la izquierda del resultado, en cuyo caso se deberá desplazar la mantisa n posiciones a la izquierda, restando simultáneamente n del exponente.

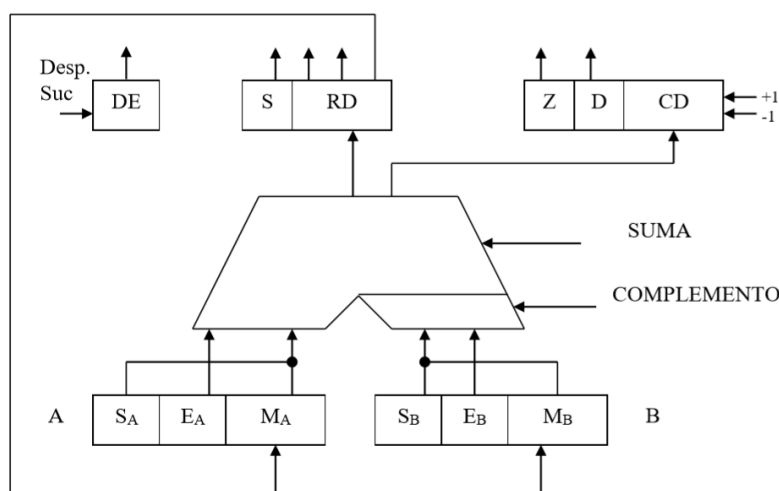
1er operando:	+	2	8	0	0	0	0
2do operando:	-	2	7	9	9	9	9
Operación:			0	0	0	0	1
Normalización:	+	-2	1	0	0	0	0

Funcionamiento de un operador de suma flotante

Nos proponemos describir el funcionamiento de dicho operador sobre operandos con la siguiente configuración:

S	E	M
---	---	---

- **S:** 1 bit (S=0 signo positivo, S=1 signo negativo)
- **E:** e bits del exponente, representa una potencia de 2 y está comprendido entre -2^{e-1} y $2^{e-1} - 1$; el exponente -2^{e-1} está representado por E=0 y el exponente $2^{e-1} - 1$ está representado por E= 2^{e-1}
- **M:** m bits de la mantisa. La mantisa de un operando negativo está representada en complemento a 2. Un operando normalizado se caracteriza por el hecho de que el bit de signo y el bit de mayor peso de la mantisa tienen valores diferentes.



El operador utilizado comprende dos registros A y B para los operandos y un sumador de m+1 bits que ejecuta las sustracciones por complementación previa del segundo operando. Se emplea, de una parte, para comparar los exponentes por sustracción, cuyo resultado va a un registro CD montado como contador/descontador binario con bit de desbordamiento, y de otra parte, para las operaciones con signo y mantisa, cuyo resultado va a un registro RD montado como registro de desplazamiento a derecha e izquierda con bit de desbordamiento.

La **operación** se **divide** en varias **fases**:

1. **Comparación de los exponentes:** Se realiza $E_a - E_b$ por medio de la suma $E_a + \underline{E_b}$ (resta por complemento a uno) y el resultado va a CD. Si el resultado es nulo, las dos mantisas están correctamente alineadas y se pasa inmediatamente a la fase de suma o sustracción de las mantisas. Si no, el contenido de CD permitirá contar los desplazamientos necesarios para alinear las mantisas.

2. **Alineamiento de las mantisas:** Esta operación se realiza sobre el operando cuyo exponente es menor. Si $EA < EB$ el resultado de la sustracción de los exponentes es negativo y se caracteriza por el valor 1 del bit de desbordamiento. El complemento del contenido de CD indica el número de desplazamientos por efectuar. CD se usará entonces en funciones de contador, y los desplazamientos se sucederán hasta un nuevo desbordamiento.

El desplazamiento de la mantisa se efectúa en RD, entonces se envían los contenidos de SA de MA yuxtapuestos a RD y se opera por desplazamiento aritmético hacia la derecha (lo que permite reproducir el signo en los bits no significativos y por lo tanto conservar la representación complementada).

En el caso $EA > EB$, el desplazamiento aritmético a la derecha afecta a SB y MB, pero en esta ocasión el registro CD se emplea como descontador, deteniéndose los desplazamientos cuando $CD=0$ (nótese que después de todo, pueden detenerse al cabo del m-ésimo desplazamiento ya que a partir de entonces la mantisa pierde todo su significado).

Esta fase finaliza preservando el mayor exponente en CD y devolviendo la mantisa contenida en RD al registro A o B que corresponda.

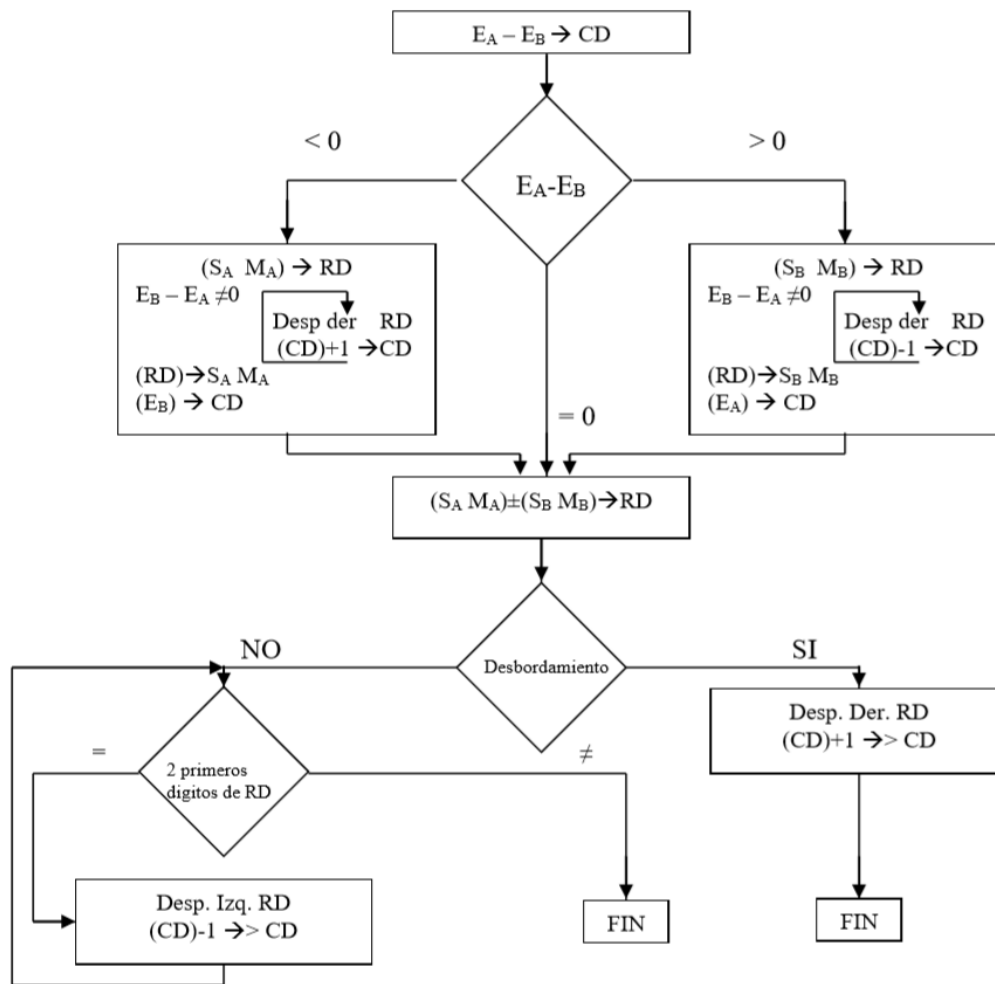
3. **Suma-sustracción de las mantisas:** Estas operaciones se ejecutan sobre el conjunto formado por el signo y mantisa, yuxtapuestos. La sustracción se obtiene sumando SA MA el complemento auténtico de SB MB. El resultado a RD, tomando en cuenta el signo y un eventual desbordamiento.

Las operaciones de normalización se realizan mediante desplazamientos sobre el registro RD, que contiene el signo y la mantisa del resultado, y sobre el registro CD, que contiene el exponente.

4. **Normalización en el caso de desbordamiento:**

Consiste en desplazar la mantisa del resultado en una posición a la derecha, siendo sustituido el bit de mayor peso por el de desbordamiento. El signo queda inalterado. El contenido del registro CD se ve aumentado en una unidad.

5. **Normalización en el caso que la mantisa del resultado contenga dígitos no significativos a su izquierda:** esta eventualidad es detectable haciendo un test sobre el signo y el dígito de mayor peso de la mantisa. Si son diferentes, el número ya estaba normalizado. Sino, es preciso realizar desplazamientos aritméticos de la mantisa hacia la izquierda hasta que sean diferentes. A cada desplazamiento se resta una unidad del contenido de CD. El proceso se detiene incondicionalmente al cabo de m desplazamientos.



Multiplicación y División en Coma Flotante

La multiplicación flotante es reducible a la multiplicación de las mantisas y a la suma de los exponentes, en tanto que la división se reduce a la división de las mantisas y a la sustracción de los exponentes. No hay operación preliminar de comparación de exponentes o de alineamiento de las mantisas. La normalización es sencilla si se compara con la suma. La multiplicación de dos números comprendidos entre $\frac{1}{2}$ y 1 da un resultado entre $\frac{1}{4}$ y 1, de tal suerte que la normalización supondrá, como máximo, un desplazamiento de una posición a la izquierda. La división dará un resultado entre 2 y $\frac{1}{2}$, de forma que su normalización implicará, como máximo, un desplazamiento de una posición a la derecha. Se ejecutan estas operaciones sobre las mantisas representadas en valor absoluto.

Unidad 7

Unidad de Control

Introducción a UC

Esta unidad se encarga de *extraer* y *analizar* las instrucciones de la memoria central, luego establece las conexiones eléctricas correspondientes dentro de la ALU y extrae los datos de la MC implicados por la instrucción. Por último, ordena a la ALU el tratamiento de los datos extraídos. Eventualmente, suele almacenar el resultado en la memoria central.

“Unidad de control: es el órgano encargado de gobernar la ruta de datos como respuesta a la ejecución de una instrucción. El gobierno de la ruta de datos se logra manejando las compuertas de gobierno a través de señales impulsionales o de nivel según corresponda.”

“La UC integra tres componentes lógicos:

- Registro de Instrucción: es quien indica al secuenciador (a través de un proceso de decodificación) cuál de los cronogramas deberá ejecutar en coincidencia con la ejecución en curso. La condición de direccionamiento también está vinculada al secuenciador.
- Secuenciador: órgano encargado de generar señales de gobierno que permitan gobernar la ruta de datos en coincidencia a la instrucción en curso.
- Estado de maquina: son un conjunto de biestables y registros conectados al secuenciador con el objeto de informar estados de maquina relevantes a la generación de los cronogramas.”

Está compuesta por dos registros:

- **Contador de Programa:** contiene la dirección de la próxima instrucción por ejecutar. Su nombre se debe a que, a excepción de las rupturas de secuencia, este registro va aumentando su contenido en una unidad para pasar al siguiente.
- **Registro de Instrucción:** contiene la instrucción extraída de la memoria. El registro de instrucción de abacus tiene 2 partes:
 - Una para el código de operación (CO), que define el tipo de instrucción a ejecutar (suma, multiplicación, salto, etc)
 - Otra para la dirección del operando (DO)

Además, la unidad de control posee también un órgano llamado **secuenciador**, que es el encargado de analizar el código de operación de la instrucción y en base a ello distribuir las microórdenes al conjunto de las unidades del ordenador (memoria, alu) para hacer ejecutar las distintas fases de la instrucción.

La UC y la ALU forman un todo en la mayoría de los ordenadores denominado CPU.

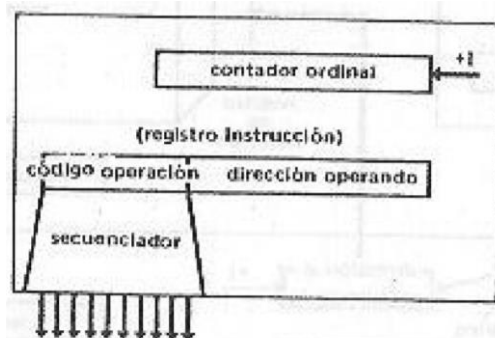


Fig. 5. Unidad de control

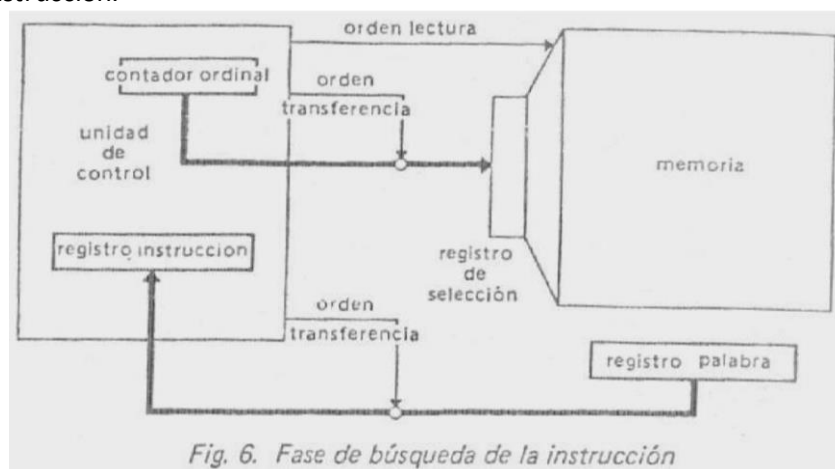
Desarrollo de una instrucción de procesamiento

El desarrollo de una instrucción de procesamiento en un computador de una dirección, como abacus, puede descomponerse en tres fases:

1. Búsqueda y análisis de la instrucción
2. Esta fase tiene dos posibilidades y en cada instrucción se selecciona solo una (excluyentes):
 - a. Búsqueda y procesamiento del Operando
 - b. Almacenamiento del resultado
3. Preparación para la próxima instrucción

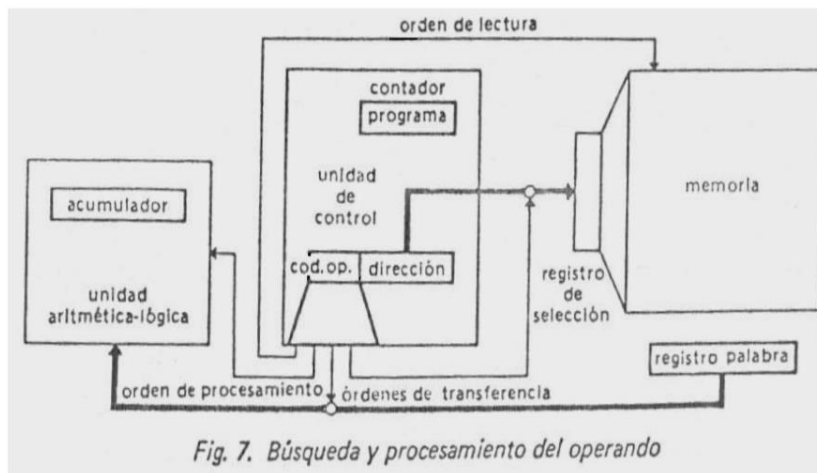
1. Búsqueda de la Instrucción

La unidad de control ordena la transferencia del contenido del contador de programa (es decir, la dirección de la instrucción por ejecutar) al registro de selección de memoria (S) y envía a la memoria una orden de lectura. Una vez terminada la operación de lectura, la instrucción queda disponible en el registro de palabra (M). Entonces la unidad de control ordena la transferencia del contenido de este registro al de instrucción. Los circuitos especializados de la unidad de control pueden ya analizar el código de operación de la instrucción. Esta primera fase es común a todos los tipos de instrucción.



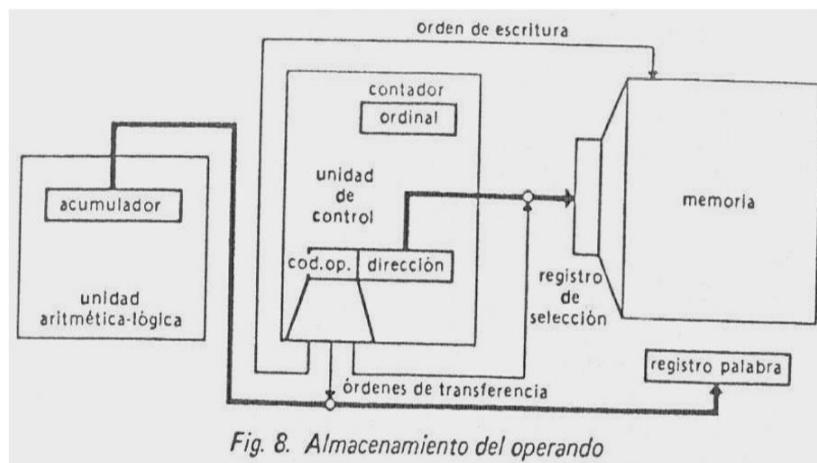
2.a Búsqueda y procesamiento del operando

Una vez analizado el código de operación de la instrucción, la unidad de control sabe que se trata de una instrucción de procesamiento, con búsqueda previa del operando. La dirección del operando se encuentra en la zona de dirección del registro de instrucción. La unidad de control ordena su transferencia al registro de selección de memoria y acto seguido ordena una operación de lectura en la memoria. Al finalizar dicha operación, el operando buscado queda disponible en el registro de palabra. La unidad de control posiciona los circuitos de la ALU para realizar el procesamiento solicitado por el código de operación y ordena la transferencia del operando a la ALU. El resultado del procesamiento queda almacenado en el acumulador. Obsérvese que un posible procesamiento pudiera ser simplemente una transferencia del operando al acumulador.



2.b Almacenamiento del Resultado

La dirección de almacenamiento del operando se encuentra en la zona de dirección del registro de instrucción: la unidad de control ordena su transferencia al registro de selección de la memoria. El operando por almacenar está en el acumulador: la unidad de control ordena su transferencia al registro de palabra de memoria. Luego de esto el secuenciador emite la orden de escritura y el resultado queda almacenado en memoria central.



3. Preparación para la próxima instrucción

Consiste en aumentar en una unidad el contenido del Contador Ordinal de la unidad de control, de manera que contenga la dirección de la próxima instrucción.

Instrucción de Ruptura de Secuencia

Este tipo de instrucción, también llamado instrucción de bifurcación o de salto, permite modificar el desarrollo secuencial del programa, haciendo que la instrucción de salto sea seguida, no por la instrucción almacenada en la siguiente dirección (secuencialmente, aumentando en 1 el CP), sino por una instrucción cuya dirección viene suministrada por la propia instrucción de salto (es decir, el CO será "salto" y dentro de la DO del registro de instrucción se encontrará la dirección a la cual se deberá saltar que será enviada al CP).

La ruptura de secuencia puede ser:

- **Condicional:** esta no tendrá efecto más que si se satisface una determinada condición, normalmente relacionada con el contenido del acumulador de la ALU, si la condición no se satisface el programa continuará en secuencia. Si la respuesta de la ALU es que la

condición se ve satisfecha, la UC ordena la transferencia de la dirección a la cual deberá saltar que se encuentra en la DO del Registro de Instrucción al CP e inhibe la suma de una unidad al contador ordinal, en caso contrario, se ordena en 1 el contador.

. **Aclaración:** Cuando se trata de una operación de salto condicional, de la UC pasa directamente a la ALU y se fija en el acumulador de la ALU si se satisface o no la condición, si se satisface le pasa la dirección que se encuentra en el registro de instrucción al contador de programa, sino, incrementa el contador de programa de la unidad de control en 1.

- **Incondicional:** la dirección de la siguiente instrucción a la cual deberá saltar, se encuentra en la DO del Registro de Instrucción al CP e inhibe la suma de una unidad al CP.

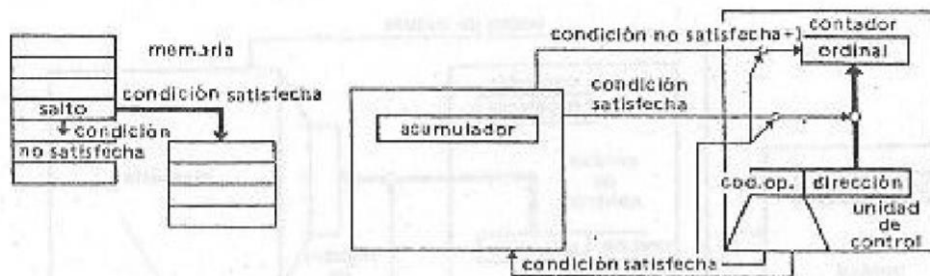


Fig. 9. Instrucción de salto condicional

Descripción de UC de Abacus

Las características generales de Abacus son las siguientes:

- Instrucciones de una sola dirección
- Transferencias en paralelo
- Máquina síncrona (implica que todos los elementos de la máquina están sincronizados con un mismo reloj)
- El reloj producirá un batido cada θ segundos
- El ciclo de base de la memoria será de 2θ intervalo correspondiente a dos batidos de reloj
- La longitud de la palabra será de n bits
- La longitud del Código de Operación será de p bits
- La longitud de la palabra es suficiente para que los m bits de la parte de dirección permitan direccionar toda la memoria

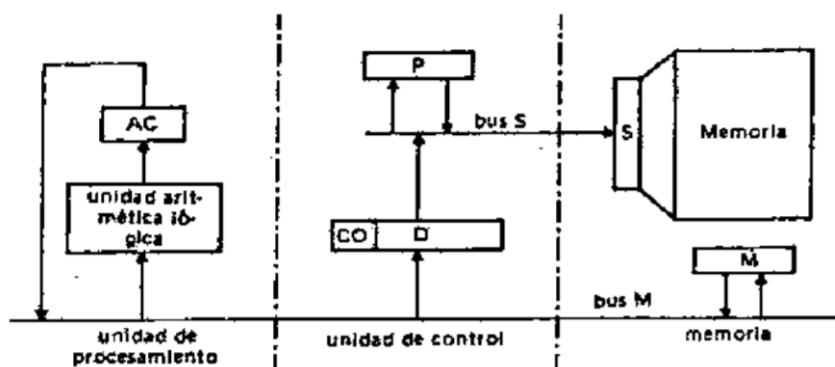


Fig. 6 Esquema simplificado de Abacus

Abacus es una máquina organizada en torno a dos buses, que son el bus de memoria (**bus M**) que transfiere las palabras leídas de, o escritas en memoria y el bus de selección (**bus S**) que transfiere las direcciones.

S	Registro de selección de memoria	m bits
P	Contador ordinal de programa	m bits
M	Registro de intercambio o de palabra	n bits
I	Registro de instrucción con dos partes: CO y D	p + m
CO	Código de operación	p bits
D	Dirección	m bits
AC	Acumulador	n bits

Con el efecto de profundizar el esquema simplificado de Abacus (Fig. 6) se explicaran tres instrucciones que permitirán resaltar todas las señales que gobiernan transferencias en Abacus.

- Suma
- Almacenamiento en memoria
- Salto Incondicional

Instrucción de suma en Abacus

SUM	DIRECCION
-----	-----------

Sumar el contenido del operando que se encuentra en la dirección de memoria dada por la DO, con el contenido del acumulador. Se descompone en cuatro fases:

Fase 1: Búsqueda de la Instrucción

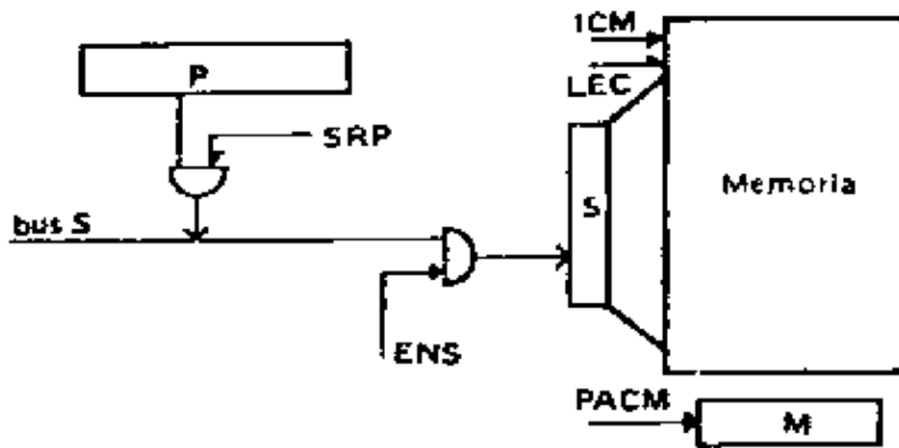
Suponiendo que la dirección de la instrucción por ejecutar se halle en el contador ordinal, es preciso:

- Transferir el contenido del Contador Ordinal al Registro de Selección de Memoria (RSM): (P) → S
- Lanzar un ciclo de memoria, de tal modo que sea transferido el contenido de la célula de memoria direccionada por el registro S al registro de palabra. ((S)) → M.

Análisis del Proceso:

(P) → S. Durante el último θ de la instrucción precedente, la señal SRP mantiene abierta la puerta de salida del registro P, por lo que la información de P está latente en el bus S bajo la forma de niveles lógicos. Bastará con validar esta información al primer batido de reloj de la instrucción por medio de la señal de muestreo (de impulso) ENS para introducirla en el registro S.

((S)) → M. Será necesario lanzar un ciclo de memoria mediante la señal impulsional ICM (inicio de ciclo de memoria), indicar que se trata de una lectura, manteniendo por ejemplo la señal LEC durante todo el intervalo del ciclo (y en cualquier caso hasta el inicio de la regeneración) y, si fuera preciso, poner a cero el registro M antes de la lectura de los núcleos (PACM).



Podemos trazar el cronograma de esta primera fase según la hipótesis indicada más arriba. Supondremos que la información buscada en la memoria está disponible en M antes del batido de reloj que sigue al inicio de ciclo de memoria.

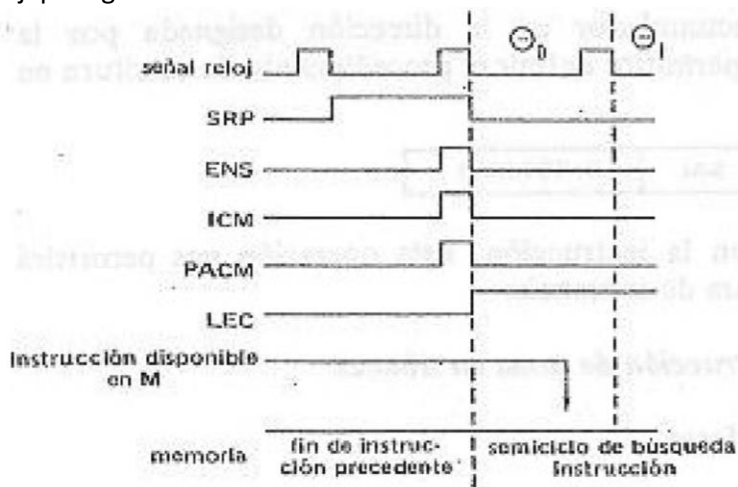


Fig. 8. Cronograma de la búsqueda de la instrucción

La fase de búsqueda de la instrucción puede resumirse así:

1. (P) → S SRP, ENS
2. ((S)) → M ICM, PACM, LEC

“Todo el tiempo entre el final de un batido y el principio del otro es el que necesita una señal de nivel para asegurar que se llene el bus. A continuación cuando el bus está lleno tenemos que impactar con una señal de gobierno del tipo ENS, haciendo ya ingresar el contenido del bus al registro de selección. Luego de esto podemos liberar al bus una vez que ya proveyó la dirección al registro de Selección.”

“En la fase 1 logramos que la instrucción ya está en el registro de palabra de memoria.”

Fase 2: Búsqueda del Operando

Debe enviarse la instrucción contenida en el registro M al registro I para proceder a su decodificación: (M) → I, la parte de dirección D de la instrucción, que especifica la dirección del operando ha de ser transferida al registro S: (D) → S, de manera que, tras un semiciclo de lectura, el operando se encuentre en el registro M: ((S)) → M.

Análisis paso a paso del proceso:

(M) → I. La instrucción se encuentra en el registro M antes del fin del primer θ del ciclo, se abre la puerta de salida del registro M al bus M durante este primer θ , para que, así mantenida la información en el bus M, pueda forzarse su introducción en I gracias a la señal de muestreo ENI, sincronizada con el segundo batido del reloj del ciclo de memoria.

(D) → S. El registro I se divide en dos partes: una parte de control, con el CO e informaciones complementarias, y una parte de dirección. La decodificación de la primera parte indica que esta dirección es la del operando que debe ahora localizarse en la memoria.

Por consiguiente, es preciso enviar la parte de dirección del registro I al registro S abriendo la puerta de nivel (señal SRD) para mantener la información en el bus S. Al siguiente batido se abrirá la puerta impulsional de entrada en S (señal ENS).

((S)) → M. Se borrará el registro M (señal PACM) simultáneamente con la introducción en S y se lanzará un ciclo de memoria por medio de un impulso ICM, indicando que se trata de una lectura gracias al mantenimiento durante todo el nuevo ciclo del nivel LEC.

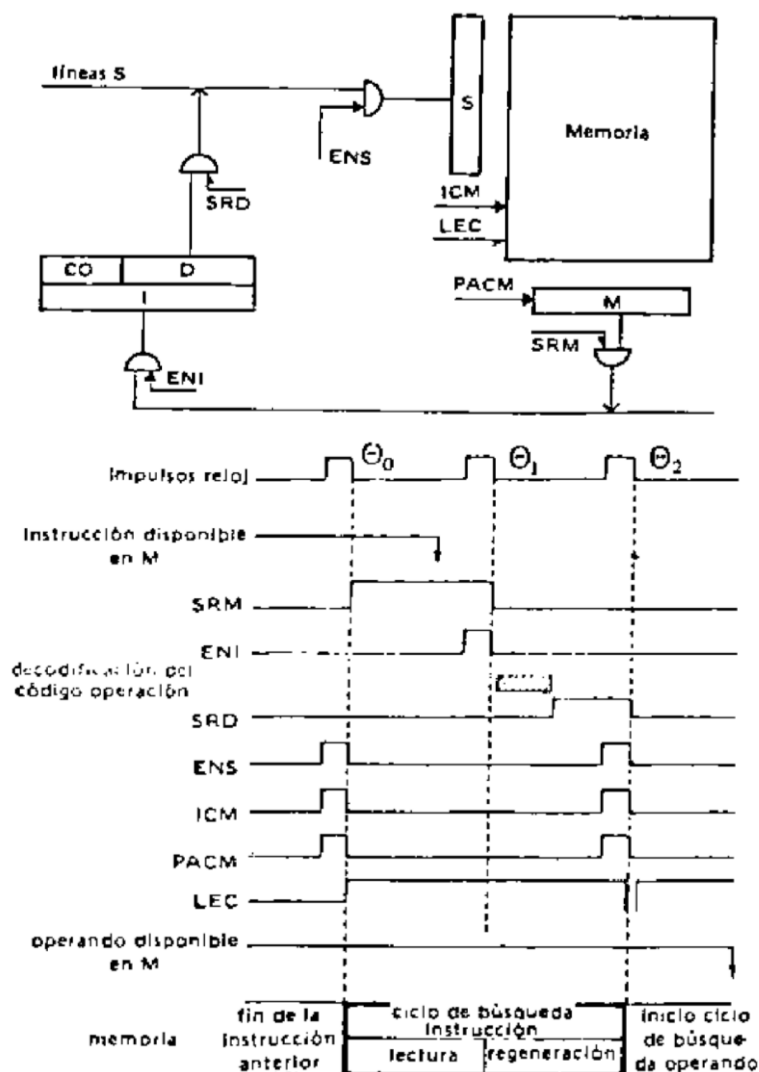


Fig. 10. Cronograma de búsqueda del operando

La fase de búsqueda del operando puede resumirse así:

1. $(M) \rightarrow I\ SRM, ENI$
2. $(D) \rightarrow S\ SRD, ENS$
3. $((S)) \rightarrow M\ ICM, PACM, LEC$

“Se podría decir que todas las salidas de registros hacia los buses son de **nivel**.”

“Una vez que se tiene el registro de instrucción cargado con el código de operación y la dirección del operando antes que vaya al registro de selección a buscar el operando tiene que saber de que instrucción se trata porque si la instrucción es un salto no tiene que ir a buscar ningún operando. Con esto vemos que hay familias de instrucción dentro de una máquina que requieren distinto tratamiento a nivel familiar. Todas las operaciones lógicas (and, or, sum, resta, etc) son con búsqueda de operando, estas instrucciones son del tipo IBO. ”

.Aclaración: Lo primero que se tiene que leer del registro de instrucción es el código de operación porque si se trata de un salto condicional no tiene que ir a buscar ningún operando. Con esto vemos que en las maquinas hay instrucciones que requieren distintos tratamientos.

Fase 3: Ejecución de Suma

Se trata de sumar el contenido de M con el contenido de AC: $(M) + (AC) \rightarrow AC$.

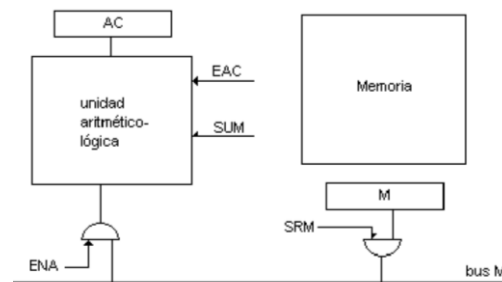


Fig. 11. Ejecución de la Suma

Los niveles lógicos correspondientes a los registros M y AC durante todo el tiempo de trabajo del sumador, consagrado esencialmente a la propagación del arrastre. Se consigue por las señales SRM, ENA, SUM. Terminada la suma, una señal EAC introduce el resultado en el AC.

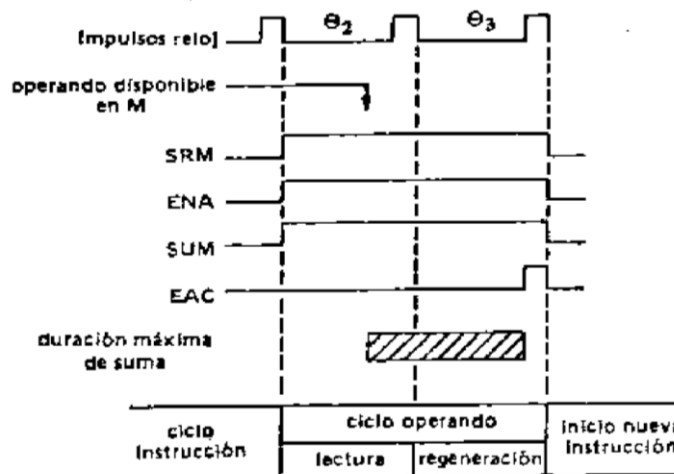


Fig. 12. Cronograma de la Ejecución de Suma

La fase de procesamiento del operando puede resumirse así:

1. $(M) + (AC) \rightarrow AC\ SRM, ENA, SUM, EAC$

Fase 4: Preparación para la próxima instrucción

Esta fase se dedica esencialmente a incrementar el contador de programa en 1: $(P) + 1 \rightarrow P$, enviándole un impulso de cuenta INCP para que al final de la instrucción contenga la dirección de la próxima instrucción. Esta última reanuda nuevamente el proceso: $(P) \rightarrow S$, ciclo de búsqueda de la instrucción, etc.

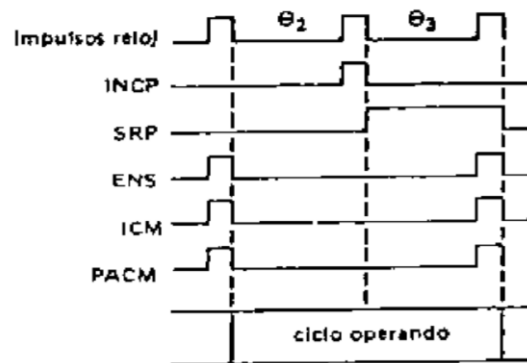


Fig. 13. Preparación de la siguiente instrucción

La fase de preparación de la próxima instrucción puede resumirse así:

1. $(P) + 1 \rightarrow P$ INCP
2. $(P) \rightarrow S$ SRP, ENS

Síntesis de la suma en Abacus:

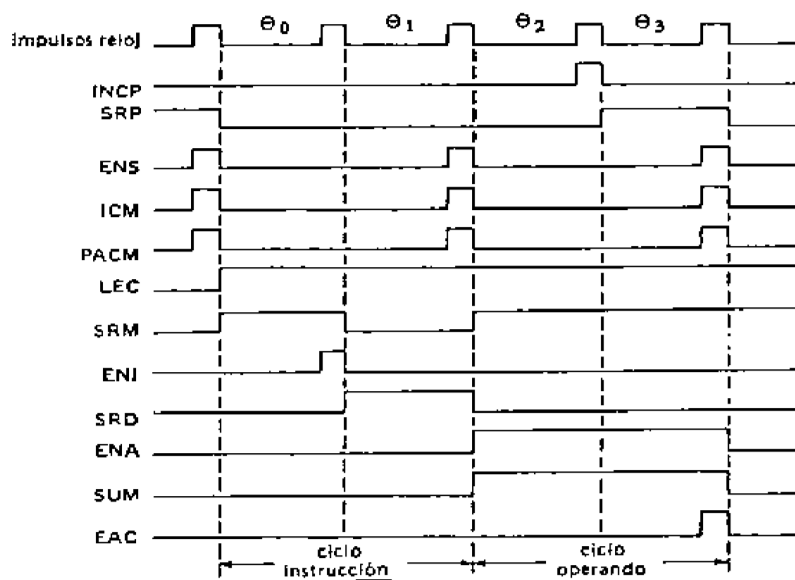


Fig. 14. Cronograma de la suma en abacus

1. $(P) \rightarrow S$ SRP, ENS
2. $((S)) \rightarrow M$ ICM, PACM, LEC
3. $(M) \rightarrow I$ SRM, ENI
4. $(D) \rightarrow S$ SRD, ENS
5. $((S)) \rightarrow M$ ICM, PACM, LEC
6. $(M) + (AC) \rightarrow AC$ SRM, ENA, SUM, EAC
7. $(P) + 1 \rightarrow P$ INCP
8. $(P) \rightarrow S$ SRP, ENS

“La lectura y escritura ocupan dos ciclos de memoria”.

Instrucción de almacenamiento en Abacus

El primer ciclo de la instrucción es idéntico al de la suma. El segundo ciclo consiste en almacenar el contenido del acumulador en memoria, en la dirección especificada por la instrucción. Entonces será preciso reproducir en M el contenido del acumulador, justo antes de la escritura en memoria.

- (D) → S SRD, $ENS^{\sim}$
- (AC) → M SAC, $ENM^{\sim}$
- (M) → (S) $ICM^{\sim}$, ESC

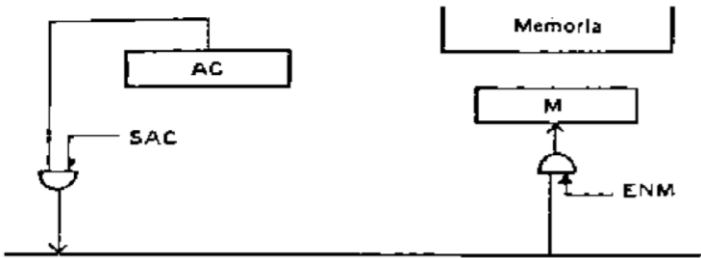


Fig. 15. Almacenamiento del Acumulador

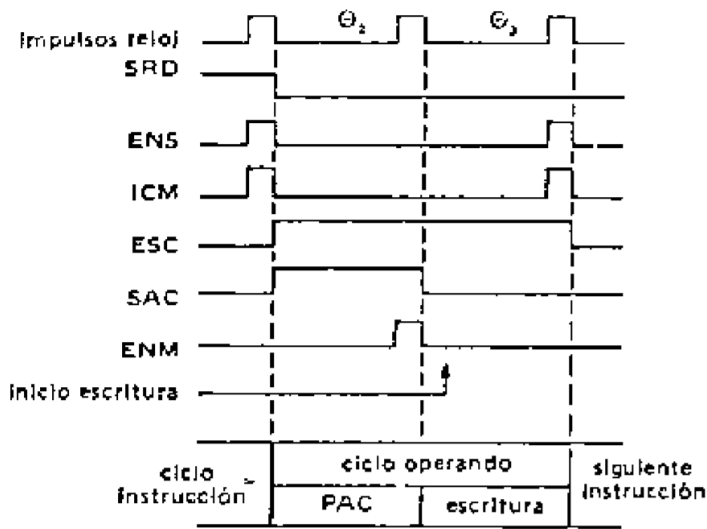


Fig. 16. Cronograma del Almacenamiento

Instrucción de salto condicional en Abacus

La información de la parte de dirección de la instrucción debe ser transferida simultáneamente a S para buscar la próxima instrucción y a P para actualizar el contador: (D) → S, (D) → P. Obsérvese que esta instrucción no dura más que un ciclo de memoria y que no hay que incrementar en 1 el contador como ocurre en las otras instrucciones.

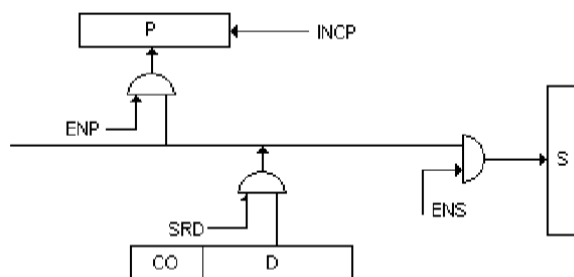


Fig. 17. Carga de la dirección de salto

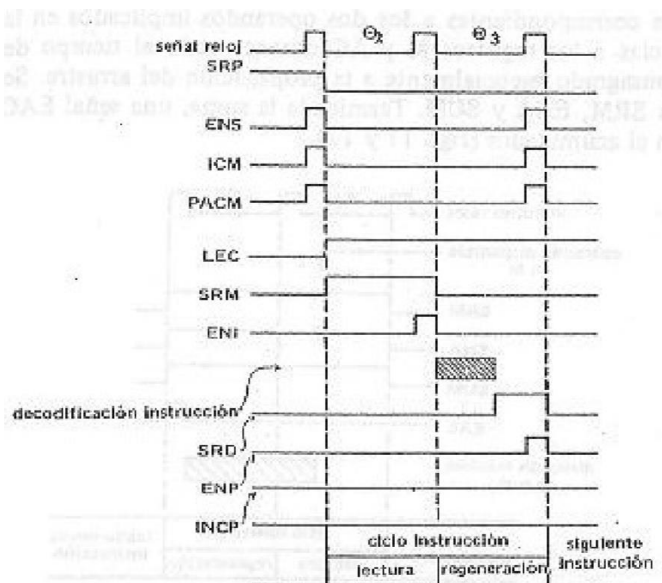


Fig. 18. Cronograma del salto incondicional

Resumen del desarrollo de la operación de salto incondicional:

(P) → S SRP, $ENS^{\sim}$
 ((S)) → M $ICM^{\sim}$, $PACM^{\sim}$, LEC
 (M) → I SRM, $ENI^{\sim}$
 (D) → P SRD, $ENS^{\sim}$
 (D) → S SRD, $ENP^{\sim}$, $ENS^{\sim}$

Señales de Gobierno de Abacus

Son todas las señales de impulsos o de nivel empleadas para gobernar las distintas operaciones en el desarrollo de una instrucción.

Señales de Transferencia			
	1. ENS	Impulso	(bus S) → S
	2. PACM	Impulso	0 → M
	3. SRM	Nivel	(M) → bus M
	4. ENM	Impulso	(bus M) → M
	5. ENI	Impulso	(bus M) → I
	6. ENA	Nivel	(bus M) → UAL
	7. SAC	Nivel	(AC) → bus M
	8. ENP	Impulso	(bus S) → P
	9. SRD	Nivel	(D) → bus S
	10. SRP	Nivel	(P) → bus S
	11. INCP	Impulso	(P) + 1 → P

Señales de gobierno de la memoria	12. ICM	Impulso	Inicio ciclo memoria
	13. LEC	Nivel	Ciclo de lectura
	14. ESC	Nivel	Ciclo de escritura
Señales de gobierno de la ALU	15. CAR	Nivel	Carga acumulador
	16. EAC	Impulso	Introducción del resultado en el A
	17. SUM	Nivel	Suma
	18. SUS	Nivel	Sustracción

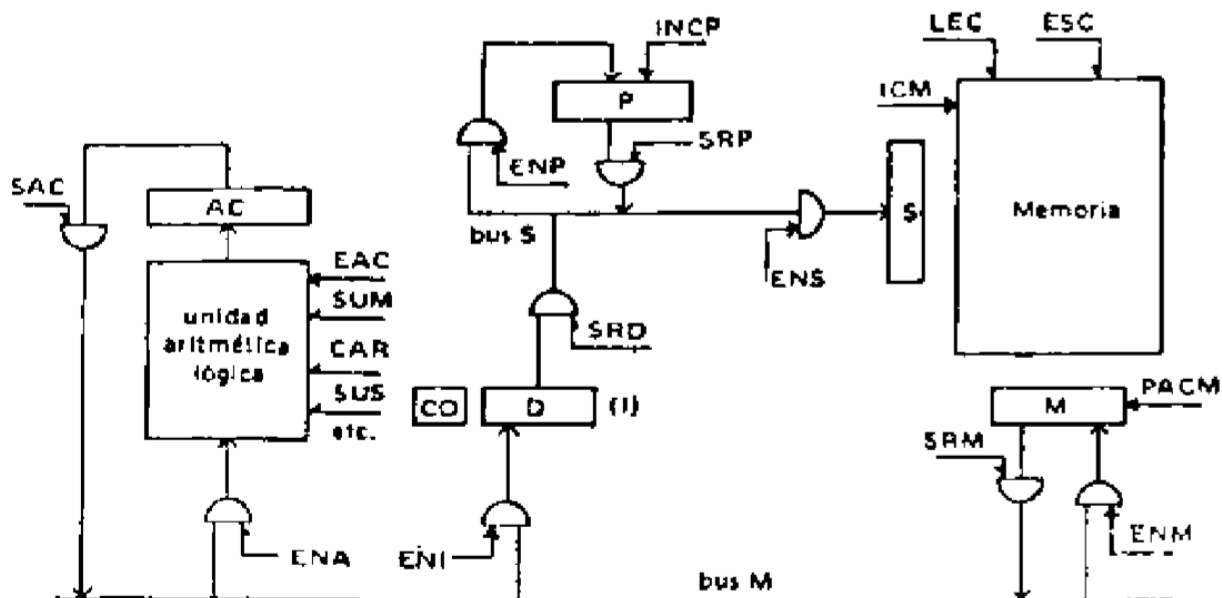


Fig. 19. Esquema de Abacus

DIRECCIONAMIENTO

Las diferentes técnicas de direccionamiento equivalen generalmente a una transformación de la parte de dirección de la instrucción; en la dirección que se transferirá finalmente al registro de selección en memoria para obtener la información deseada. Llamaremos a esta última **dirección efectiva**.

El tipo de procesamiento que debe sufrir el contenido de la zona de dirección viene especificado, o por el código de operación cuando este impone un tipo determinado, o por la configuración binaria de una parte de la instrucción, que contiene lo que convendremos en llamar las **condiciones de direccionamiento (CD)**.

La instrucción en un computador de una dirección constará de tres partes:

CO	CD	D
Código de Operación	Condiciones de Direccionamiento	Zona de Dirección de Memoria

“¿El direccionamiento está vinculado a la UC (específicamente al secuenciador)?

Si porque es parte de la responsabilidad del secuenciador también generar las señales de gobierno inherente a la instrucción e inherente a la manera que se va a leer de la memoria, no solo inherente a como resolver la instrucción en curso sino también de qué manera se va a leer el operando (directo, indirecto, etc).”

Los diferentes **tipos de direccionamiento** que existen son:

1. **Direccionamiento Inmediato:** No es propiamente un direccionamiento, puesto que la parte de dirección de la instrucción contiene ya el operando que se quiere procesar. Este tipo de direccionamiento no consume ciclo de memoria. La desventaja es que el tamaño del numero está restringido a la longitud del campo de direcciones que, en la mayoría de los repertorios de instrucciones, es pequeño comparado con la longitud de la palabra.

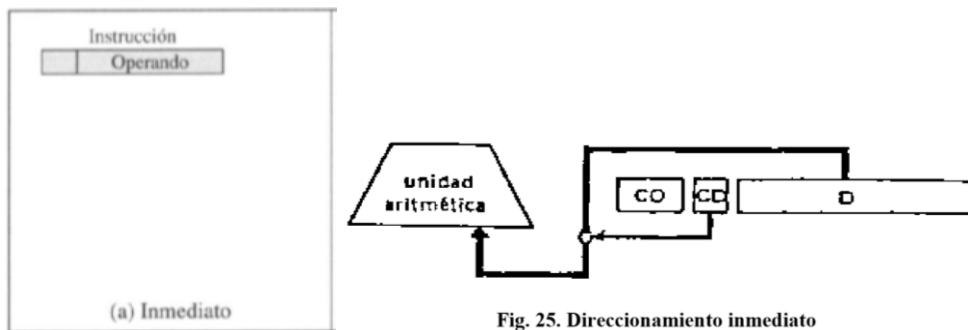


Fig. 25. Direccionamiento inmediato

2. **Direccionamiento Normal – Directo – Absoluto:** La zona de dirección de la instrucción contiene la dirección efectiva del operando. El direccionamiento normal exige un ciclo de memoria para obtener el operando. La limitación obvia es que proporciona un espacio de direcciones restringido.

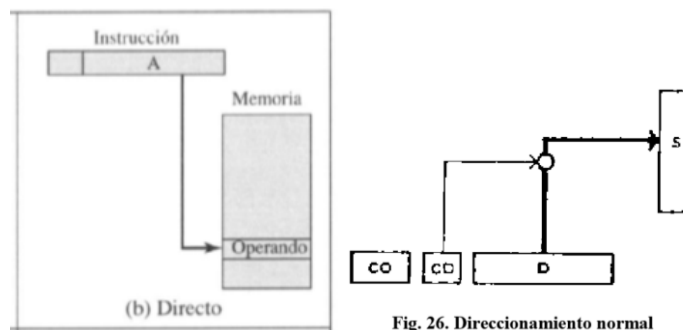


Fig. 26. Direccionamiento normal

3. **Direccionamiento indirecto:** La parte de dirección de la instrucción contiene, no la dirección de la información pedida, sino la dirección de una palabra de memoria donde se encontrará la dirección efectiva de la información. Por tanto, la localización de un operando direccionado indirectamente exigirá dos ciclos de memoria: un ciclo para buscar la dirección efectiva, otro ciclo para buscar el operando.

La ventaja obvia de esta aproximación es que para una longitud de palabra de N bits, se dispone ahora de un espacio de direcciones de 2^N . La desventaja es que la ejecución de la instrucción requiere dos ciclos de memoria para captar el operando.



Fig. 27. Direccionamiento indirecto

4. Direccionamiento Relativo: La dirección relativa no indica la posición de la información en la memoria en valor absoluto, sino que la sitúa en relación a una dirección de referencia. Esta, a su vez, está almacenada en un registro frecuentemente llamado registro de traslación. **La dirección efectiva se obtendrá sumando la dirección relativa con la dirección de referencia.** Las técnicas de direccionamiento relativo se emplean especialmente cuando la longitud de la palabra de memoria es insuficiente para permitir direccionar a toda la memoria.

Tres ejemplos de direccionamientos relativos:

4.1 Direccionamiento por base y desplazamiento: Un registro de la máquina, llamado registro de base, contiene la dirección de referencia (primera dirección de un programa o de una zona de datos, por ejemplo). A la información que alberga la parte de dirección se le llama desplazamiento. La dirección es la suma de la base y del desplazamiento. Algunos calculadores admiten varios registros de base. La instrucción debe especificar entonces la dirección de registro de base escogido.

“La dirección efectiva de la palabra surge como la suma o resta de un valor del registro base y el campo D.” Permite acceder a campos inalcanzables por el campo DIR.

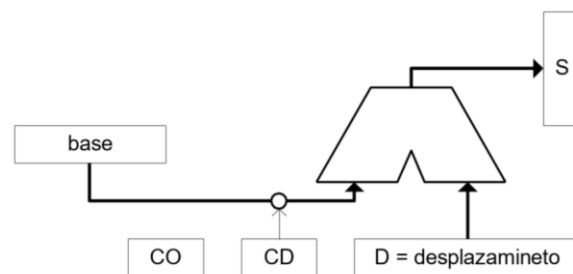


Fig. 28. Direccionamiento por base y desplazamiento

4.2 Direccionamiento por referencia al programa: El contenido del contador de programa sirve de dirección de referencia. Con este sistema es posible generalmente direccionar dos zonas de memoria a un lado y a otro de la instrucción en curso, según que la parte de dirección se sume o se reste con el contenido del contador.

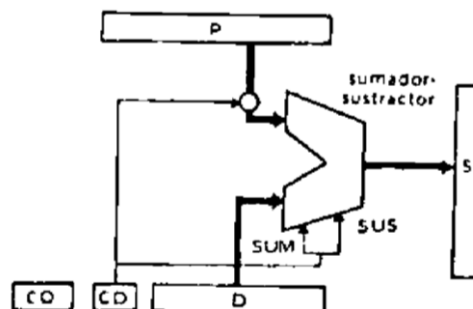


Fig. 29. Direccionamiento por referencia al programa

4.3 Direccionamiento por página (Yuxtaposición): Se considera a la memoria dividida en zonas de 2^n palabras llamadas páginas. En general la parte de dirección de la instrucción contiene n bits, por lo que no capacita a la máquina para direccionar más palabras que las que tiene una página. La dirección efectiva se obtendrá por yuxtaposición del número de

página (o dirección de página), supuesto conocido y de la parte de dirección de la instrucción, que suministra la dirección dentro de la página.

Las condiciones de direccionamiento, en la mayoría de los pequeños ordenadores organizados por páginas, poseen un BIT de direccionamiento que, según su valor, implica el direccionamiento absoluto, es decir dentro de la página cero, o el direccionamiento dentro de la página de la instrucción en curso por yuxtaposición de los bits de mayor peso del contador de programa y de la dirección dentro de la página. En este caso, el direccionamiento por yuxtaposición puede ser considerado como un direccionamiento por referencia al principio de la página en curso.

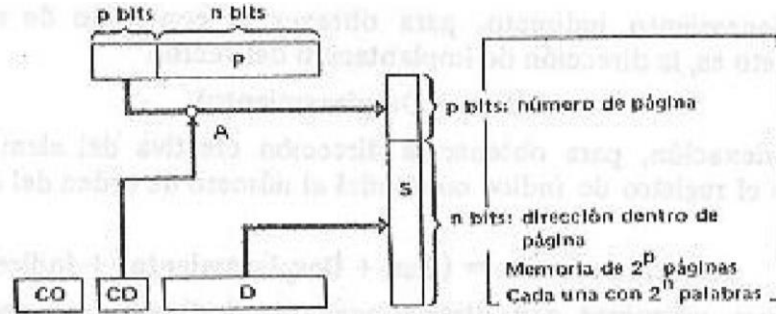


Fig. 31. Dirección en la página cero o en la página de la instrucción

5. Direccionamiento Indexado: Se obtiene la dirección efectiva sumando a la parte de dirección de la instrucción el contenido de un registro de la unidad central llamado registro índice; a menudo se llama índice al contenido. El registro de índice admite un cierto número de operaciones, como carga, lectura, incremento o decremento en 1, comparación. El programador lo utiliza para tratar, mediante una sola instrucción en un bucle de programa, datos almacenados vectorialmente en células sucesivas de la memoria. El direccionamiento correspondiente es indexado, lo que quiere decir que la dirección especificada en la instrucción es la primera célula del vector y a ella se suma el valor del índice, inicialmente puesto a cero e incrementado en 1 cada vez que se ejecute la instrucción de fin de bucle. Esta última compara el índice con el número de elementos del vector y origina un salto al principio del bucle mientras quede algún elemento por procesar.

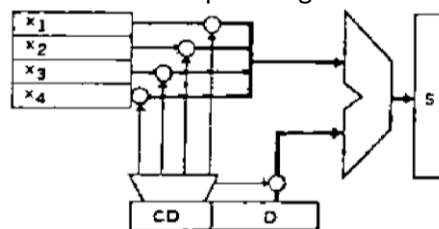


Fig. 31. Indexación

Unidad 8

El Secuenciamiento de las Instrucciones

Ruta de Datos: conjunto de órganos de la máquina que intervienen para transferir, memorizar y procesar las informaciones procedentes de la MC. (Instrucciones, direcciones y operandos).

Los órganos de la ruta de datos son:

- Buses de interconexión: para la transferencia de información
- Registros y Memorias: para su almacenamiento
- Unidades funcionales: para procesar

Microórdenes: Son señales de gobierno que están distribuidas por toda la ruta autorizando las diferentes operaciones. Son generadas por el secuenciador. A diferencia de las instrucciones, operandos y direcciones que son conocidas por el programador, las microórdenes no son conocidas por el programador.

“Una señal de gobierno, impulsional y/o de nivel.”

“Su función es gobernar las distintas compuertas de la ruta de datos”

*“Las señales de gobierno impulsionales y de nivel se las define como **microórdenes** en el entorno del secuenciador.”*

SECUENCIADOR CENTRAL

El secuenciador central de un ordenador es el órgano que genera las microórdenes, que son distribuidas a lo largo de la ruta de datos para activarla y gobernar sus diversos elementos constituyentes.

ENTRADAS Y SALIDAS DEL SECUENCIADOR

- Las **informaciones de salida** del secuenciador son las microórdenes que deben distribuirse por la ruta de datos según cronogramas precisos a causa de los tiempos respuesta de los órganos gobernados.
- Las **informaciones de entrada** al secuenciador son suministradas, de una parte, por la instrucción (código de operación, condiciones de direccionamiento y las direcciones de registros), de otra parte, por el estado de la máquina, que agrupa un cierto número de informaciones (indicadores de error, demandas de interrupción, indicadores del estado de la memoria y de las unidades funcionales, etc.).

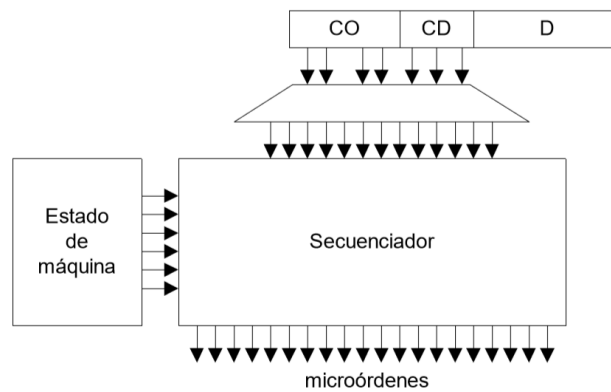


Fig. Entorno de un secuenciador

Estado de Máquina: “son un conjunto de biestables y registros conectados al secuenciador con el objeto de informar estados de máquina relevantes a la generación de los cronogramas. Por ejemplo: biestable signo del acumulador indicaría condición satisfecha o no para el caso de un salto condicional. Es decir, si el signo del acumulador es negativo hago salto a otra dirección.”

SECUENCIADOR SINCRONO Y ASINCRONO

. **Secuenciador Síncrono:** El secuenciador conoce los tiempos de respuesta de los diferentes órganos gobernados. Entonces hace intervenir los retardos apropiados entre las diferentes órdenes sucesivas de manera que tengan tiempo de ejecutarse las operaciones. Las microórdenes van generalmente sincronizadas con los impulsos de reloj. Así, si una operación necesita un poco más de dos batidos de reloj, el secuenciador esperará al tercer batido posterior al lanzamiento de la operación para enviar la siguiente microórden.

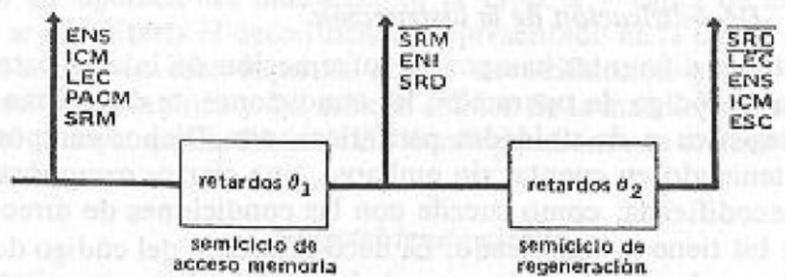
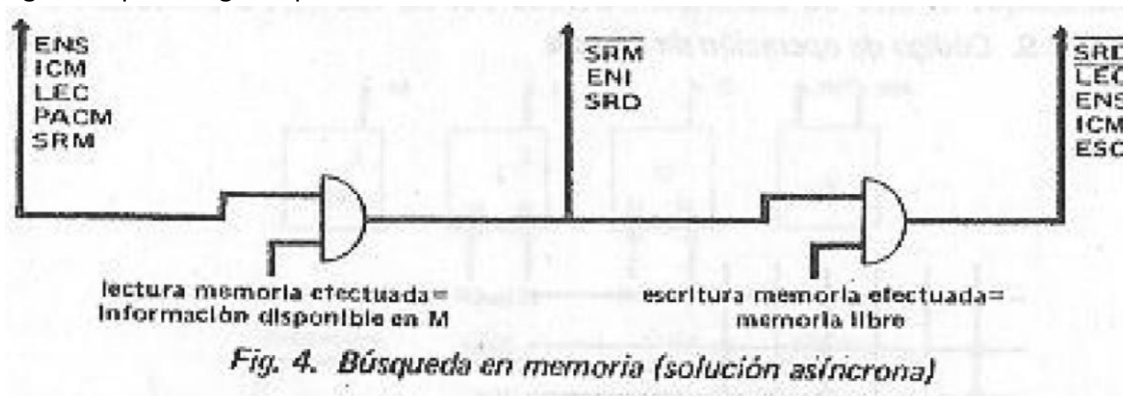


Fig. 3. Búsqueda en memoria (solución síncrona)

“En este caso, las señales de nivel, por ejemplo SRM, dura el retardo θ_1 porque luego se le niega porque tiene que dejar de funcionar en esa fase. Así funciona este esquema. Cuando se quiere desactivar una de señal de nivel se debe complementar la señal. Las señales en este esquema siempre están separadas un espacio de tiempo fijo, por eso se dice síncrono.”

“El esquema secuenciador cableado síncrono plantea la generación de microórdenes espaciada en el tiempo a través de retardos previamente definidos. Hay retardos que hacen que para pasar de una columna del cronograma a otra tiene que pasar el espacio de tiempo de retardo previamente establecido. Son de valores fijos. Son circuitos secuenciales porque es cableado, no hay memoria de control.”

. **Secuenciador Asíncrono:** El secuenciador recibe de los diferentes órganos del computador señales indicando que han terminado las operaciones ordenadas y que se liberan; aquel puede eventualmente comprobar su estado. No lanzará una nueva operación más que después de haber advertidos de que las operaciones procedentes han sido ejecutadas completamente y de haberse asegurado que el órgano que debe realizarla está libre.



“En el modelo asincrónico las compuertas de gobierno dejan pasar de una fase a otra cuando todas las señales de gobierno de una misma fase se cumplieron, entonces los retardos que hay entre un grupo y otro de señales son variables y depende de que todo el grupo de señales se haya cumplido. Es decir, en el modelo asincrónico se pasa a la siguiente fase solo cuando las señales del anterior se hubieran cumplido.”

“En realidad para la maquina estar controlado si todas las microinstrucciones se cumplieron es muy complejo, por lo tanto, el control que realiza es que, por ejemplo, en la primer fase sabe que debe pasar a la siguiente cuando la lectura de la memoria se haya efectuada y la información ya esté en M.”

SECUENCIADOR CABLEADO Y MICROPROGRAMADO

. **Secuenciador Cableado:** Realizados en forma de circuito secuencial electrónico.

“Los cableados responden a un diseño basado en la implementación de circuitos secuenciales encargados de generar las microórdenes para cada instrucción.”

“La ventaja del cableado es económico y muy veloz pero la economía puede verse afectada en función a la complejidad. La desventaja es que es muy rígido e imposible alterar su programación una vez implementado. Otro inconveniente es que ocupa gran espacio físico y alto consumo eléctrico por lo que es inviable su implementación en estos días.”

. **Secuenciador Microprogramado:** Corresponden a la introducción a la unidad de control de una memoria con un pequeño programa, llamado microprograma, para cada instrucción. El desarrollo de este microprograma genera las microórdenes que gobiernan la ejecución de la instrucción.

*“Los microprogramados reemplazan los circuitos por memorias especiales llamadas **memoria de control** y tienen almacenadas en ellas las señales de gobierno de cada cronograma. Ya están grabadas las microórdenes en la memoria y salen en función de la instrucción en curso.”*

“Tiene como desventaja un alto costo de implementación (memorias de nivel 5, de mucha capacidad para almacenar los cronogramas). La ventaja es que ocupa reducido espacio, consumen poca energía y son muy flexibles en cuanto a su microprogramación.”

“No hay circuitos para generar las microórdenes lo que están son las microórdenes grabadas en una memoria y van saliendo al compás que indica el clock.”

SECUENCIADORES CABLEADOS

Principio del Secuenciamiento

Las señales impulsiones tiene un ciclo de duración de acuerdo a lo que necesita la compuerta. Las de nivel no se sabe, para estas hay que mandar la señal inversa para desactivarlas.

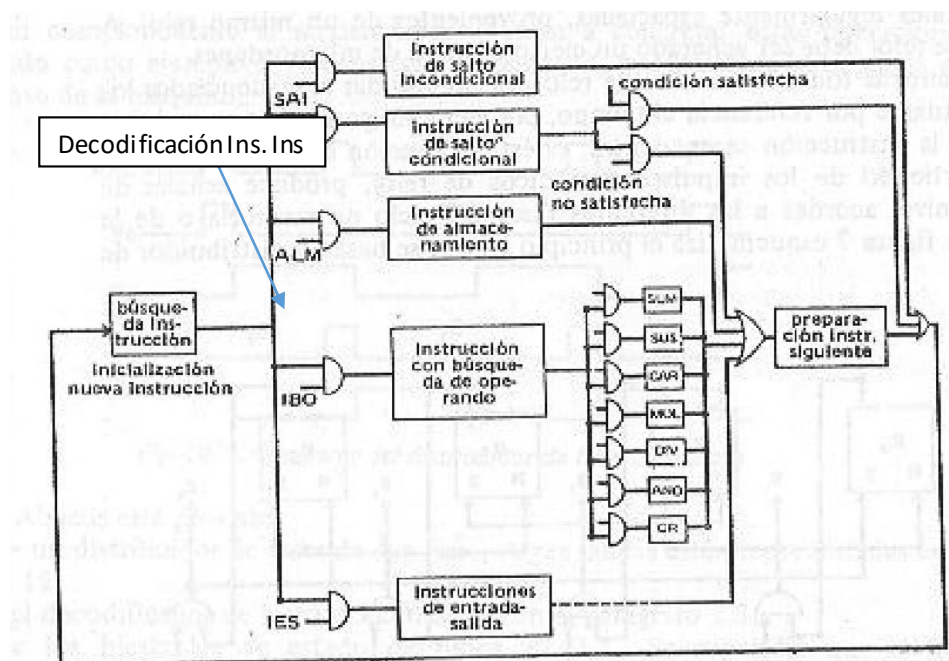


Fig. 5. Principio del secuenciamiento; tratamiento de las diferentes instrucciones

“Si la UC se encuentra con instrucción del tipo IBO son instrucciones con búsqueda del operando.”

“Las IES son instrucciones de entrada/salida. Es una familia de instrucciones (put, get, etc.) que indican movimiento de información entre la memoria y el periférico; también incluye otras instrucciones como prueba de estado del periférico o toma de gobierno del periférico.”

“SAC es un salto incondicional controlado por el signo del acumulador. Si el signo negativo es un salto incondicional. Si no lo es, va a preparar la siguiente instrucción. Es un cronograma con dos finales que lo determina el signo del acumulador en ese momento.”

¿Qué hace la instrucción stop en Abacus?

“Con la instrucción stop el secuenciador pone a 0 el biestable marcha de condición. El clock sigue funcionando pero no deja pasar al distribuidor de fases para que este haga andar al secuenciador. Si el distribuidor de fases se detiene el secuenciador se para donde estaba, el cronograma queda frenado. El que hace al andar los distintos cronogramas del secuenciador es el distribuidor de fases.”

“Toda máquina atraviesa por una fase de búsqueda de instrucción, posible búsqueda de operando, y la ejecución de la operación. Pero en el medio puede ocurrir dos cosas:

- Una interrupción que se denomina IT, que consiste en un teclado, mouse, táctil, pidiendo que la CPU le de tratamiento prioritario. En realidad la interrupción es que el programa en curso detenga su ciclo normal y desarrolle un pequeño programa que de solución al periférico que lo interrumpió.*
- Una petición de ciclo de memoria E/S o petición ciclo memoria E/S: es un pedido del canal a la CPU para realizar una escritura o lectura entre la memoria y un periférico que lo*

requiere. Debido a que los periféricos solo dialogan con el canal, es este quien pide el ciclo de memoria.”

Distribuidor de Fases

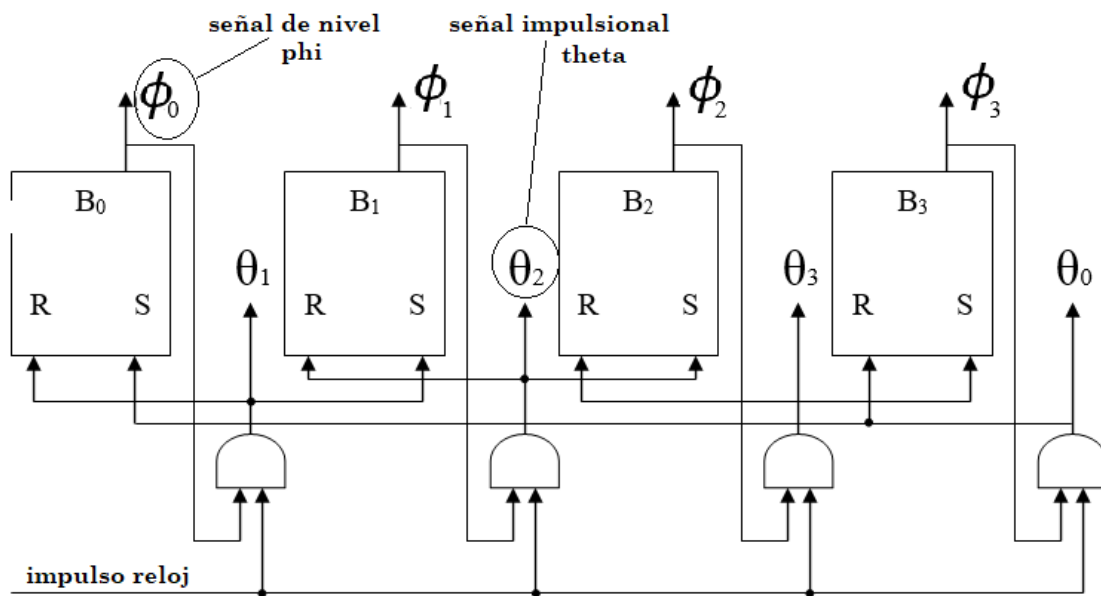
Su función es producir señales de impulso o de nivel (acordes a las diferentes fases del ciclo de memoria o de la instrucción) a partir de los impulsos periódicos del reloj.

“La cantidad de biestables depende de cuantas fases tenga que manejar. El biestable almacena la señal de nivel y la que sale de la compuerta es una señal impulsional. Este circuito es manejado por el impulso de reloj.”

“Ejemplo:

$$\phi = \text{phi}$$

$$\theta = \text{theta}$$

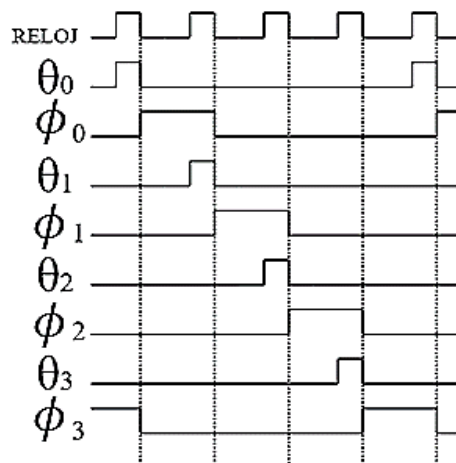


Distribuidor de cuatro fases

En un instante determinado, solamente uno de los biestables B0, B1, B2, B3 está posicionado a 1. El próximo impulso de reloj devolverá este biestable a 0, y posicionará el siguiente biestable a 1.

“Este circuito arranca siempre sobre la base de que un biestable tiene un 1 para comenzar.”

“Comienza con la señal impulsional θ_0 (theta 0) a la derecha de todo, poniendo en 0 el biestable B3 y en 1 el biestable B0. La salida de este biestable es una señal de nivel que permanece activa hasta que llega junto con la señal del clock a la compuerta que da como salida la señal impulsional θ_1 (theta 1), la cual pone el cero el biestable B0 y en uno el biestable siguiente (B1). El proceso se repite.”



"El distribuidor de fases genera una única señal impulsional y de nivel para todas las fases de los cronogramas de máquinas. Será entonces la tarea del secuenciador la de derivar estas señales (distribuir) a cada compuerta de gobierno de la ruta de datos en cumplimiento al cronograma de estado de la instrucción en curso."

"El distribuidor de fases debe tener tantas fases como para representar a todos los cronogramas. Tiene que abastecer al cronograma con mayor cantidad de fases. Si todos los cronogramas ocupan 4 fases y uno solo ocupa 32 fases entonces el distribuidor de fases debe tener 32 fases."

Decodificador de Instrucciones

"Es un circuito capaz de seleccionar uno y solo un cronograma en función al valor binario del campo CO. Los hilos intersectados equivalen a un 1 contrariamente es un 0."

NOMBRE NEMOTECNICO	CODIGO OPERACION	EFEECTO DE LA INSTRUCCION
PAC	000	puesta a cero del acumulador
SUM	001	suma con el acumulador
SUS	010	sustracción del acumulador
AND	011	intersección con el acumulador
OR	100	reunión con el acumulador
ALM	101	almacenamiento del acumulador
SAI	110	salto incondicional
SAP	111	salto si (AC) ≥ 0

Tab. 9. Código de operación de Abacus

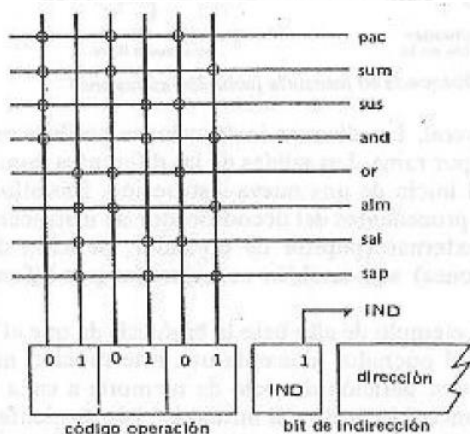


Fig. 10. Decodificación del código de operación de Abacus

Biestables de Estado

El contenido de los biestables de estado define el estado del calculador. Existen dos clases:

1. Los primeros memorizan lo que hemos denominado el estado de la máquina. Son posicionados por las instrucciones y los resultados de operaciones, o también por

acontecimientos exteriores, petición de ciclo de memoria, interrupción, etc. Sus salidas figuran entre las informaciones de entrada al secuenciador.

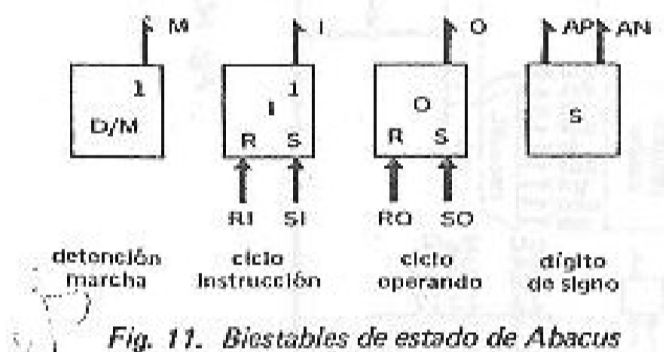
2. Los segundos constituyen parte integrante del secuenciador. Es éste quien los posiciona y quien utiliza su salida. Esencialmente sirven de marcas temporales. En cierto modo, los biestables del distribuidor de fases podrían incluirse dentro de esta categoría.

Abacus contiene los siguientes biestables:

- Detención/Marcha: "Si está en 0 pausa a la máquina. Si está en 1 continua con la ejecución."
- Ciclo Instrucción: "Si está en 1 indica que está en ciclo de instrucción,"
- Ciclo Operando: "Si está en 1 indica que está en ciclo de operando"

¿Por qué en Abacus no se tiene un solo biestable que indique si está en ciclo de instrucción o si está en ciclo de operando? Porque en el cronograma hay que sostener una señal de instrucción o sostener una señal de operando, todo el tiempo que le lleve hacer una cosa o la otra. Voy a necesitar un biestable que me esté dando una señal, entonces lo necesitas independiente. Cuando instrucción está en alto (1) ocupa dos batidos toda esa señal de nivel y baja cuando te pasas a operando pero necesitas otra que la levante que es la de operando. En el cronograma lo que se está representado es estas dos partes."

- Dígito de Signo: "Es un biestable del acumulador y sirve para determinar si es salto o no. El biestable de signo es el signo del acumulador y determina en la instrucción SAC si es satisfecha o no la condición."



Trazado de Cronogramas

Cronograma: Representación que nos permite conocer cómo se comportará el secuenciador ante una determinada instrucción.

Abacus está provisto por:

- a. Un distribuidor de fases de dos fases.
- b. Decodificador de instrucción (pac, sum, sus, and, or, alm, sai, sap).
- c. De los 4 biestables de estados (figura anterior).

Se supondrá que el funcionamiento del reloj está condicionado por la señal de detención-marcha, y que la puesta en marcha inicializa al distribuidor de fase.

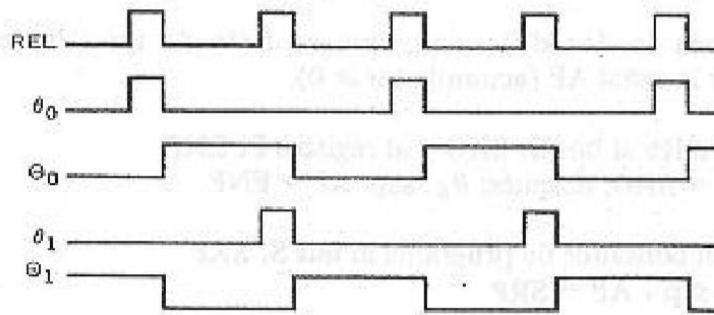


Fig. 12. Cronograma del distribuidor de fases de Abacus

Ecuaciones Lógicas: Conjunto de expresiones lógicas del ordenador relacionadas con las informaciones de entrada y de estado del secuenciador.

Instrucción de Salto: Ecuación Lógica - Cronograma

Cada uno de los cronogramas nos permite definir de forma analítica el comportamiento del secuenciador. Por ejemplo, para la instrucción SAP (salto si es positivo) es preciso efectuar las siguientes operaciones elementales:

En el instante θ_0 :

- Posicionar el biestable, si no lo estuviera ya, indicando que comienza un ciclo de instrucción.
- Lanzar un ciclo de memoria: ENS, ICM, PACM

Entonces esta etapa resulta: $\theta_0 \rightarrow SI, ENS, ICM, PACM$

Durante el intervalo ϕ_0 , sabiendo que se desarrolla un ciclo de instrucción:

- Mantener la señal LEC para indicar que se trata de una lectura
- Abrir la puerta de salida del registro M

$$I. \phi_0 \rightarrow LEC, SRM$$

$I. \phi_0$ Significa que durante todo el intervalo de tiempo ϕ_0 se desarrollará un ciclo de instrucción.

En el instante θ_1 , sabiendo que aún se mantiene el ciclo de instrucción:

- Muestrear a la entrada del registro de instrucción: ENI
- Incrementar el contador de programa: INCP

$$I. \theta_1 \rightarrow ENI, INCP$$

Durante el intervalo ϕ_1 , sabiendo que se desarrolla un ciclo de instrucción:

- Mantener la señal LEC para permitir la regeneración de información en memoria
- Sabiendo, tras la decodificación, que se trata de un salto si acumulador positivo, verificar la señal AP (Acumulador ≥ 0)

○ Si $AP=1$: Enviar la dirección de salto al bus S: SRD y al registro P: ENP
 $\phi_1.sap.AP \rightarrow SRD$, después $\theta_0.sap.AP \rightarrow ENP$

○ Si $AP=0$: Enviar el contenido del contador del programa al bus S: SRP

$$\phi_1.sap.AP \rightarrow SRP$$

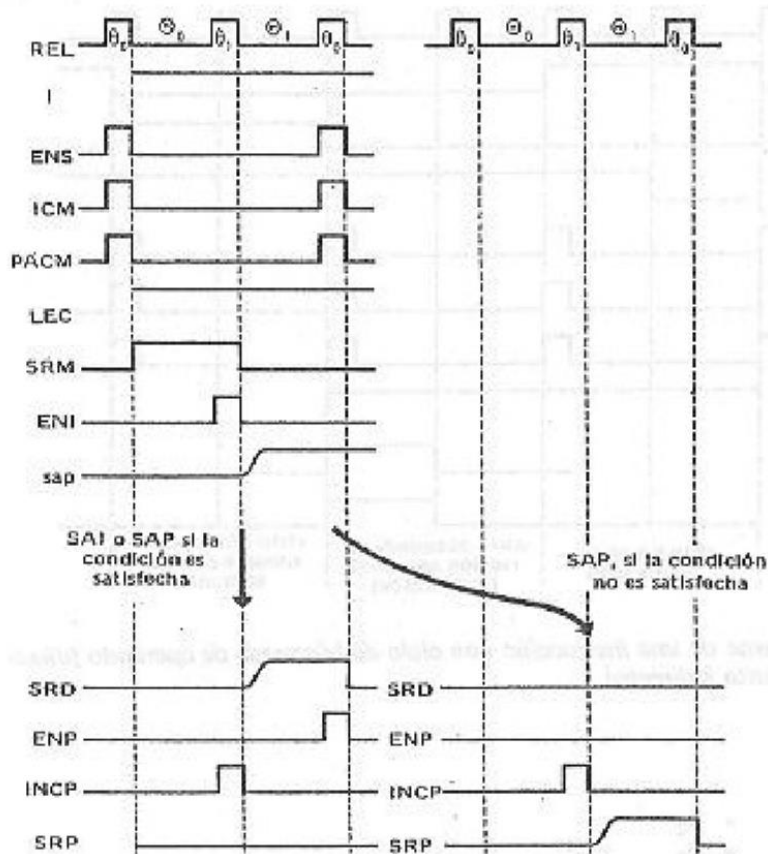
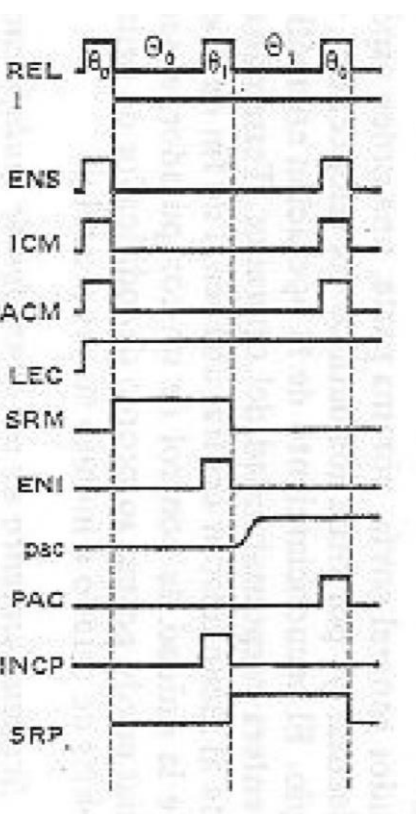


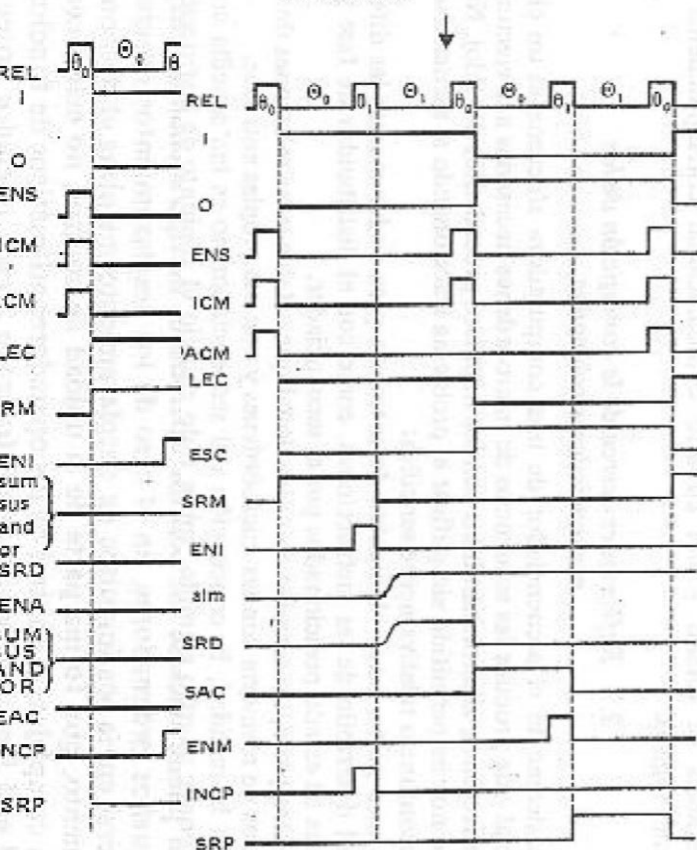
Fig. 18. Cronogramas de las instrucciones de salto

Cronograma de puesta a cero del acumulador



Cronograma de las instrucciones de Procesamiento (SUM, SUS, AND, OR)

Fig. 17. Cronograma del almacenamiento en memoria



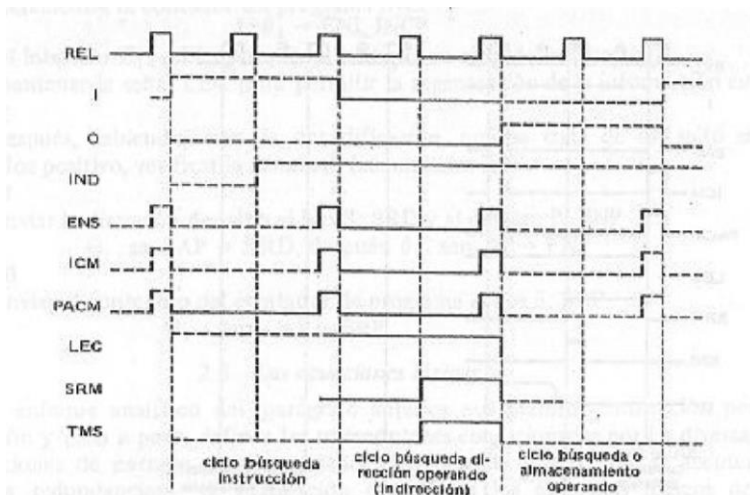


Fig. 19. Cronograma de una instrucción con ciclo de búsqueda de operando (direccionamiento indirecto)

$$\begin{aligned}
 \text{ENS} &= \theta_0 \\
 \text{ICM} &= \theta_0 \\
 \text{PACM} &= \theta_0 \\
 \text{LEC} &= I + O \cdot \text{alm} + (\bar{O} \cdot \bar{I}) \\
 \text{ESC} &= O \cdot \text{alm} \\
 \text{SRM} &= \theta_0 \cdot I + O \cdot \text{alm} + \text{IND} \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O}) \\
 \text{TMS} &= \text{IND} \cdot \bar{I} \cdot \bar{O} \cdot \theta_1 \\
 \text{ENI} &= \theta_1 \cdot I \\
 \text{SRD} &= I \cdot \theta_1 \cdot (\text{sum} + \text{sus} + \text{and} + \text{or} + \text{alm} + \text{sap} \cdot \text{AP} + \text{sal}) \\
 \text{SRP} &= \theta_1 \cdot \bar{I} \cdot (\text{pac} + \text{sap} \cdot \text{AN}) + \theta_1 \cdot O \\
 \text{INCP} &= I \cdot \theta_1 \\
 \text{ENP} &= \theta_0 \cdot (\text{sal} + \text{sap} \cdot \text{AP}) \\
 \text{SUM} &= \text{sum} \cdot O \\
 \text{SUS} &= \text{sus} \cdot O \\
 \text{AND} &= \text{and} \cdot O \\
 \text{OR} &= \text{or} \cdot O \\
 \text{ENA} &= O \cdot \text{alm} \\
 \text{EAC} &= O \cdot \theta_0 \cdot \text{alm} \\
 \text{PAC} &= \text{pac} \cdot \theta_0 \\
 \text{SAC} &= \theta_0 \cdot O \cdot \text{alm} \\
 \text{ENM} &= \theta_1 \cdot O \cdot \text{alm} \\
 \text{SI} &= \theta_0 \cdot O \\
 \text{RI} &= \theta_0 \cdot I \cdot (\text{sum} + \text{sus} + \text{or} + \text{and} + \text{alm}) \\
 \text{SO} &= \text{IND} \cdot \theta_0 \cdot I \cdot (\text{sum} + \text{sus} + \text{and} + \text{or} + \text{alm}) + \text{IND} \cdot \theta_0 \cdot \bar{I} \cdot \bar{O} \\
 \text{RO} &= \theta_0 \cdot O
 \end{aligned}$$

Fig. 20. Ecuaciones lógicas de Abacus

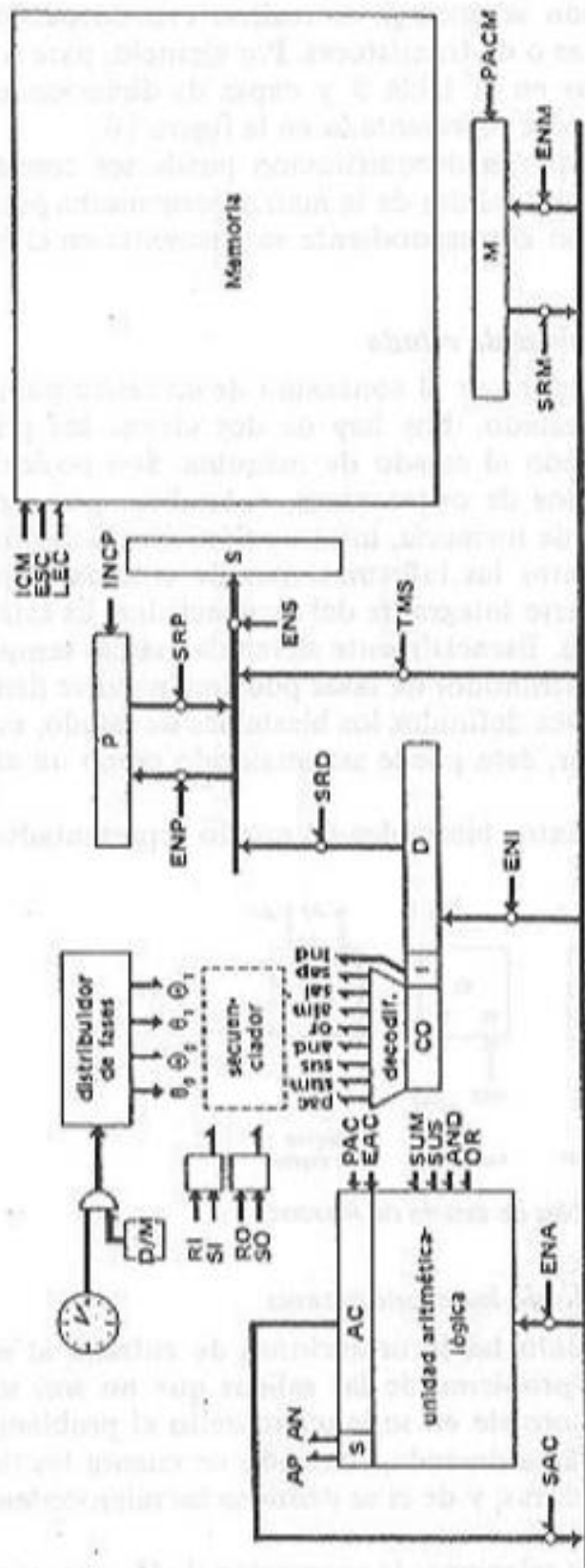


Fig. 14. Esquema general de Abacus

¿Qué pasa con TMS? ¿Y sus cronogramas? ¿Y qué pasa en las fases que se agregan con las señales I y O?

TMS se ocupa para cualquier instrucción en una modalidad que puede elegir el programador cuando quiera hacerlo por direccionamiento indirecto. Los cronogramas son idénticos a las operaciones pero agrega dos fases más para resolver la búsqueda del operando que no está en DIR

sino que está en M. En estas dos fases las señales I y O se encuentran muertas (0) porque no hay ni ciclo de instrucción ni ciclo de operando.

SECUENCIADORES MICROPROGRAMADOS

La microprogramación consiste en reemplazar el secuenciador cableado por un secuenciador programado. A cada instrucción del computador le corresponde un microprograma que se encuentra almacenada en una memoria especializada, llamada **memoria de control** o **memoria de gobierno**. El desarrollo de este microprograma genera las microórdenes que gobiernan la ejecución de la instrucción. Se llama micro-instrucciones a las instrucciones del microprograma. La ejecución de una micro-instrucción genera una o varias microórdenes.

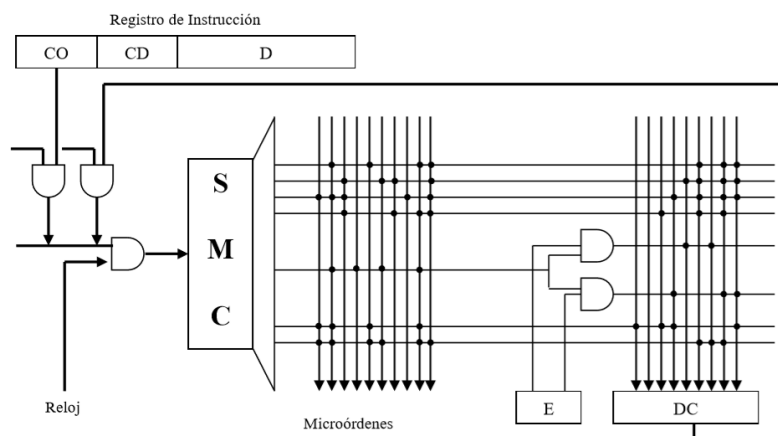
En los ordenadores microprogramados existe la posibilidad de modificar fácilmente el conjunto de instrucciones, puesto que cambiar o añadir un microprograma es mucho más sencillo que alterar el cableado.

Modelo de Wilkes

“El secuenciador multiprogramado a diferencia del cableado consiste una memoria de control no volátil, que es mucho más veloz que la central (que la RAM) la cual posee dos zonas: a la izquierda se almacenan las microinstrucciones y a la derecha la dirección de la próxima microinstrucción. La dirección de la primer microinstrucción (es lo mismo que palabra de control) es la que proviene del código de operación (CO), luego cada microinstrucción estará relacionada con una palabra que contenga la dirección de la próxima a ejecutar; terminando el proceso cuando la dirección de la próxima instrucción dirija a una vacía.

Podrá existir la posibilidad de bifurcación incrementando un biestable de estado (E) con dos compuertas AND que relaciona excepcionalmente a una microinstrucción con dos direcciones distintas de próximas microinstrucciones basado en el valor del biestable de estado.

Por ejemplo, SAP, si se cumple E=1 indicando una dirección de salto distinta a la de E=0 (condición no cumplida).”



. Funcionamiento:

Aclaración de términos utilizados:

- Operación = Instrucción. Por ejemplo SUM
- Microinstrucción = palabra de control. Por ejemplo: búsqueda del operando
- Microórden: señales de gobierno. ENS, SRM, etc.

Del CO viene la operación a realizar, esta pasa al SMC, es decodificada y se convierte en una dirección de palabra de control (microinstrucción) en memoria de control. A esta le corresponderá siempre un mismo hilo horizontal, por medio del cual, en el primer grupo de hilos verticales se sabrá que microórdenes se ejecutarán y siguiendo el hilo horizontal al encontrarse con el segundo grupo de hilos verticales se sabrá cual es la próxima microinstrucción a ejecutar, que pasará al SMC para ponerla en ejecución. Así, se realizarán, por ejemplo, en la SUM: la búsqueda de la instrucción

con sus respectivas microórdenes, la búsqueda del operando con sus microórdenes y así sucesivamente hasta finalizar con la operación de SUM. Las operaciones de salto estarán afectadas por las compuertas que activa el biestable E.

“Ventajas:

- Menor tamaño físico
- Menor consumo
- La gran ventaja es la flexibilidad a la hora de reescribir la información de control frente a la rigidez de la cableada: las memorias reescribibles de alta velocidad utilizadas para este modelo, permiten en todo momento re-flashearla”

Microprogramación	Cableado
Programaciones muy flexible	Rígida
Diseño reducido	Tamaño grande
Bajo consumo	Alto consumo
Diseño más económico	Caro
Generación de microórdenes más lentas	Generación más veloz
A mayor complejidad de la arquitectura, mayor tamaño de la palabra de control	

“Peor desventaja los microprogramados (Wilkes): A mayor complejidad de la arquitectura, mayor tamaño de la palabra de control.”

Memoria de Control – Wilkes

Es la memoria que contiene los microprogramas a razón de una o, raramente, varias microinstrucciones por palabra de memoria.

La primera característica exigida a una memoria de control es que su velocidad de lectura sea muy superior a la de la memoria central. En efecto, para que el conjunto sea coherente, es preciso que un microprograma completo registrado en la memoria de control se desarrolle totalmente en el intervalo de tiempo correspondiente a los pocos accesos a la memoria central que conciernen a la instrucción y sus operandos.

Un segundo elemento importante a la hora de elegir la tecnología de las memorias de control, es que, en el desarrollo de los programas, aquellas no se utilizan más que en la lectura.

Estas dos consideraciones llevaron a Wilkes y a sus sucesores a utilizar como memorias de control, memorias muertas, que permiten una gran rapidez de lectura por un coste razonable. Están habitualmente organizadas por palabra y el punto de memoria consiste en la presencia o ausencia de un acoplamiento entre hilo de palabra e hilo de bit.

Codificación de las Micro-Instrucciones

Nos concentraremos únicamente en la primera parte de la microinstrucción. Se tienen diferentes métodos para pasar de la configuración binaria de la microinstrucción a las diversas microórdenes correspondientes.

Métodos	Funcionamiento	Ventajas	Desventajas
Wilkes	Asociar un bit de la microinstrucción a cada microórden.	Permite generar varias microórdenes simultáneas.	- Exagerada longitud de microinstrucción - Dificultad para el programador de microprogramas
Inverso de Wilkes	Codificar el conjunto de las posibles microórdenes con el número mínimo de bits.	Si es corta la longitud de la microinstrucción se gana en el costo de la memoria de control (siempre y cuando el número de palabras no sea excesivo)	- Se agrega un decodificador - Se necesitaría una microinstrucción por cada microórden
Tipo instrucción	La microinstrucción tiene una estructura semejante a la de una instrucción: código de operación y dirección de operando	Pequeña longitud de microinstrucción	La decodificación de las microinstrucciones para obtener las correspondientes microórdenes es relativamente compleja
Por campos	Se dividen las distintas microórdenes en grupos independientes de tal forma que sea imposible que, en cada grupo, se den microórdenes simultáneas.	Decodificación más directa	Microinstrucción más larga

Direccionamiento de las Micro-instrucciones

Tipo	Funcionamiento	Ventajas	Desventajas
Secuencial	Una microinstrucción direcciona implícitamente la microinstrucción que se encuentra en la próxima dirección. El registro de selección de la memoria de control funciona como contador de programa. Los saltos pueden realizarse especializando un bit de la microinstrucción como indicador afirmativo o negativo de salto, dejando el resto de la misma para albergar la dirección de la próxima microinstrucción y, cuando sea el caso, la condición de salto.	▪ Resulta simple de programar ▪ No se alarga la microinstrucción	▪ Un salto ocupa por sí sólo una microinstrucción y desperdicia el tiempo de su ejecución. ▪ Dificultad para el programador de microprogramas
Codificación asociada Tipo instrucción			
Explícito	Utilizado por Wilkes, pero no sigue su método para realizar los saltos, porque impone un cableado particular de la memoria de control. Se fracciona en dos partes la parte de microinstrucción que define a la instrucción siguiente: - la primera parte contiene de manera explícita los bits de mayor peso de la dirección - la segunda parte contiene los bits de menor peso de la dirección (si esta fuera conocida) o la dirección de un elemento de registro que los tenga almacenados (si fuera condicional).	▪ Minimiza el número total de microinstrucciones, ya que algunas son comunes a varios microprogramas ▪ No supone pérdida de tiempo en los saltos incondicionales	▪ Difícil de usar ▪ Aumenta la longitud de la microinstrucción

Unidad 9

Von Neumann - Harvard

Arquitectura Von Neumann

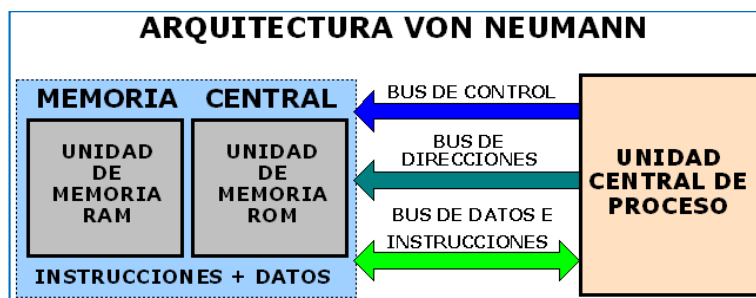
La arquitectura tradicional de computadoras y microprocesadores se basa en el esquema propuesto por John Von Neumann, en el cual la unidad central de proceso, o CPU, está conectada a una memoria única que contiene las instrucciones del programa y los datos.

El tamaño de la unidad de datos o instrucciones (palabra de memoria) está fijado por el ancho del bus de la memoria. Es decir que un microprocesador de 8 bits, que tiene además un bus de 8 bits que lo conecta con la memoria, deberá manejar datos e instrucciones de una o más unidades de 8 bits (bytes) de longitud. Cuando deba acceder a una instrucción o dato de más de un byte de longitud, deberá realizar más de un acceso a la memoria. Por otro lado este bus único limita la velocidad de operación del microprocesador, ya que no se puede buscar de memoria una nueva instrucción, antes de que finalicen las transferencias de datos que pudieran resultar de la instrucción anterior.

Es decir que las **dos principales limitaciones** de esta arquitectura tradicional son:

- La longitud de las instrucciones está limitada por la unidad de longitud de los datos (palabra de memoria), por lo tanto el microprocesador debe hacer varios accesos a memoria para buscar instrucciones complejas.
- La velocidad de operación (o ancho de banda de operación) está limitada por el efecto de cuello de botella que significa un bus único para datos e instrucciones que impide superponer ambos tiempos de acceso.

La arquitectura von Neumann permite el diseño de programas con código automodificable, práctica bastante usada en las antiguas computadoras que solo tenían acumulador y pocos modos de direccionamiento, pero innecesaria, en las computadoras modernas.



Arquitectura Harvard

La arquitectura conocida como Harvard, consiste simplemente en un esquema en el que el CPU está conectado a dos memorias por intermedio de dos buses separados. Una de las memorias contiene solamente las instrucciones del programa, y es llamada *Memoria de Programa*. La otra memoria solo almacena los datos y es llamada *Memoria de Datos*. Ambos buses son totalmente independientes y pueden ser de distintos anchos. Esto permite además del paralelismo de operación, que las memorias de instrucciones y datos puedan tener los tamaños adecuados (el caso particular de este modelo es 14 bits para instrucciones y un byte para datos).

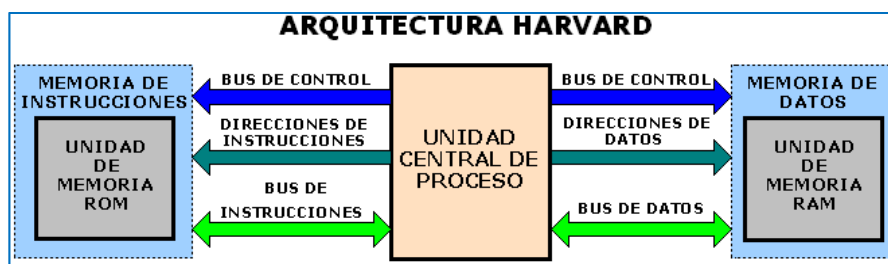
Para un procesador de Set de Instrucciones Reducido, o RISC (Reduced Instrucción Set Computer), el set de instrucciones y el bus de la memoria de programa pueden diseñarse de manera tal que todas las instrucciones tengan una sola posición de memoria de programa de longitud.

Además, como los buses son independientes, el CPU puede estar accediendo a los datos para completar la ejecución de una instrucción, y al mismo tiempo estar leyendo la próxima instrucción a ejecutar.

Las **principales ventajas** de esta arquitectura son:

- El tamaño de las instrucciones no está relacionado con el de los datos, y por lo tanto puede ser optimizado para que cualquier instrucción ocupe una sola posición de memoria de programa, logrando así mayor velocidad y menor longitud de programa.
- El tiempo de acceso a las instrucciones puede superponerse con el de los datos, logrando una mayor velocidad de operación.

Una **pequeña desventaja** de los procesadores con arquitectura Harvard, es que deben poseer instrucciones especiales para acceder a tablas de valores constantes que pueda ser necesario incluir en los programas, ya que estas tablas se encontraran físicamente en la memoria de programa (por ejemplo en la EPROM de un microprocesador).



A su vez, esta organización favorece a la segmentación o "pipeline", que aprovecha la separación de las memorias de la arquitectura Harvard, pudiendo ejecutar una instrucción (lo que implica acceder a la memoria de datos) mientras se busca la siguiente (en la memoria de programa). Esto permite solapar ambas fases, permitiendo que cada instrucción se ejecute de manera efectiva en un ciclo (excepto por las de salto que tardan dos), aumentando así el número de instrucciones ejecutadas simultáneamente en un determinado periodo de tiempo. Si bien la duración de cada una no se reduce, si se reduce la duración del conjunto de ellas. Esto último es a su vez, una característica importante de los procesadores RISC (una instrucción por ciclo), por lo que dichos procesadores hacen uso de arquitectura Harvard.

"La arquitectura Harvard no deja de ser una Von Neumann nada más que tiene una arquitectura que está orientada a otro concepto. Un concepto de pequeñas instrucciones muy sencillas pero poca cantidad, donde tiene que aprovechar la segmentación. La segmentación significa ganar un ciclo: buscar una nueva instrucción mientras se está terminando la instrucción en curso. La única

manera de lograr esto es cuando la maquina puede saber cuántos ciclos le lleva a la instrucción en curso.”

“Harvard utiliza tres direccionamientos: directo, indirecto e inmediato. Para indexados se utiliza el indirecto.”

“Harvard tiene un contador de programas de múltiples niveles.”

Unidad 10

CISC - RISC

CISC surgió con la idea de hacer que varias instrucciones sencillas se plasmaran en un microprograma haciendo más fácil la tarea del programador..

El **objetivo** de esta arquitectura es que el compilador (programa traductor entre ambos niveles) sea más sencillo de fabricar y que los tiempos de compilación disminuyan y esto se logra por medio de aumentar las equivalencias entre sentencias de alto nivel e instrucciones de máquina. Es decir, obtener la menor disparidad entre los lenguajes de alto nivel y el lenguaje de máquina.

Un procesador CISC necesariamente debe contener una memoria de control con los microcódigos, para poder ejecutar las instrucciones complejas.

CISC – Computador con set de instrucciones complejas

- La arquitectura CISC se caracteriza por tener un repertorio de **instrucciones complejas**.

“Las instrucciones en CISC son instrucciones que hacen más de una operación simultáneamente.”

“Un arquitecto trata de hacer una arquitectura con instrucciones muy complejas para acortar la brecha entre el lenguaje de alto nivel y lenguaje de máquina.”

- Muchas instrucciones complejas (potentes) para lograr una implementación más directa con lenguajes de alto nivel.

- La ejecución de estas instrucciones puede ser bastante complejas.

“Complejas porque el formato de la instrucción de maquina no es solamente poner CO y DO; muchas veces son instrucciones de movimiento de información de un registro a otro con niveles de octogonalidad con movimiento y copia de registro borrando el anterior.”

- Considera que la compilación de programas de alto nivel requieren muchas menos instrucciones de máquina y con tiempos más cortos.
- En la práctica, lograr elevadas velocidades de ejecución se torna difícil, requiere chips con superficie elevada de silicio, es decir, semiconductores.

Criterios para definir una arquitectura CISC

- Cantidad de instrucciones en lenguaje de maquina tiene que ser tan grande como sea posible.
- Cantidad de modos de direccionamiento tan grande como sea posible.

- Cantidad de formatos de instrucción tan grande como sea posible.

“Un formato de instrucción significa que no se basa en campos fijos (código de operación, dirección de la primer palabra, dirección segunda, tipo de direccionamiento) sino que puede ser variable. Variable significa que hay un formato para una familia de instrucciones distinto para otra familia. Se debe tener un campo al principio que indique a la máquina que formato de familia me refiero para que sepa todo lo que viene a posterior en que formato está.”

- Muchas instrucciones requieren más de un ciclo para su ejecución.
- Varios tipos de instrucciones tienen acceso a memoria.

- Existencia de registros de propósitos específicos.

“Agrega muchos registros con propósitos específicos como registros bases, registros índices.”

- Control Microprogramado.

“Para lograr todas las características de CISC es obvio que la UC tiene que ser microprograma. Son instrucciones de máquina hipercomplejas, el cableado es imposible porque se tendrían que cablear millones de instrucciones que den soporte a una instrucción de máquina compleja. Para dar soporte a un set de instrucciones complejas la UC tiene que ser microprogramada.”

- Instrucciones de máquina de un relativo alto-nivel (cercano al alto nivel de las sentencias de los lenguajes de alto nivel).

“Las sentencias de lenguaje de maquina tiene una cierta similitud a la de alto nivel, lo cual lo hacen más complejo.”

- Las instrucciones compuestas son decodificadas internamente y ejecutadas con una serie de microinstrucciones almacenadas en una ROM interna.

“La única manera de ejecutar una instrucción compleja es codificándola por partes, y a su vez, por cada grupo subdecodificado lanzar un conjunto de microórdenes; como una codificación por campos. La instrucción en el formato tan complejo que está hay que subdividirlas. Una instrucción de maquina compleja está relacionada con varios cronogramas distintos. ”

- La microprogramación es una característica importante y esencial de casi todas las arquitecturas CISC, significa que cada instrucción de maquina es interpretada por un microprograma localizado en una memoria.

RISC— Computador con set de instrucciones reducidas

Esta arquitectura surgió luego de realizar estadísticas sobre las instrucciones de máquinas más usadas. Esto sirvió para planificar procesadores con un set de instrucciones simples que presentan muy pocos modos de direccionamiento. Por ser simples, estas instrucciones se ejecutan en un solo pulso de reloj, luego de haber sido decodificada. Pero, para traducir de un lenguaje de alto nivel a este tipo de instrucciones, empleando un mínimo de ellas, se requiere un programa compilador inteligente muy elaborado. Es decir, a fin de tener un hardware más simple se requiere un compilador complejo.

No tiene memoria de control. Los procesadores RISC incorporan un caché interno para datos y otros para instrucciones, que reducen los accesos a memoria principal.

- La arquitectura RISC se caracteriza por tener un repertorio de instrucciones simples: *“Se utiliza el termino Reduced para hacer referencia a simpleza (no a cantidad). Una instrucción reducida está referido a que es sencilla.”*
- Se necesitan muchas más instrucciones para realizar la misma tarea que una instrucción CISC: *“Una instrucción en formato RISC está obligada a hacer una única operación en máquina, por ejemplo, una RISC sería copiar el contenido de un registro a otro, una acción directa sencilla y única. Acá existe un solo formato de operación (código de operación, dirección del primero operando, direc del segundo operando y resultado), no como en CISC.”*
- A pesar de la cantidad necesaria de instrucciones para realizar una tarea son muy convenientes cuando la ejecución se puede realizar segmentando instrucciones logrando velocidades elevadas de ejecución: *“Como cada instrucción es sencilla, se predice que en un ciclo de memoria estaría completada la ejecución siempre que no sea un salto. En un salto puede variar la cantidad de ciclos dependiendo de adonde salte y cuando regrese. En el salto no hay segmentación, pero en todas las otras si, y significa que mientras está ejecutando la instrucción en curso ya está buscando y decodificando la nueva.”*
- Su repertorio es utilizado convenientemente por compiladores optimizadores.
- Ventaja relacionada con la tecnología de fabricación VLSI (Very long scale integration) ya que necesitan menos área para control y procesamiento de secuencia de instrucciones (mayor espacio para registros y cache): *“Al tener menos área para el control de ejecución de instrucciones complejas se achica y se logra integración. Esto posibilita a tener más espacios a caché de nivel 2, registros específicos, que permiten mejorar la performance.”*
- Reducción de acceso a datos e instrucciones por fuera del chip logran mejores prestaciones.
- Unidad de Control cableada debido a la sencillez de la arquitectura e implementación de operaciones por hardware: *“La UC puede ser cableada ya que es tan sencilla la arquitectura que hasta incluso provoca la necesidad de hasta cablear algunas operaciones aritméticos-logicas en pos de lograr mayor velocidad. Para lograr la eficiencia de esta arquitectura lo que se hace es trabajar con una UC cableada, debido a que la arquitectura es muy sencilla.”*

“En RISC, el éxito está en 4 partes fundamentales:

1. *Instrucciones complejas se reduce, de manera tal que se logre la segmentación*
2. *UC cableada. Lo puedo hacer porque la arquitectura es sencilla. En combinación con la sencillez, logro velocidad.*
3. *Se aprovecha el ítem anterior y se cablea algunas operaciones aritméticos-lógicas*
4. *Obtención de compiladores optimizadores”*

Criterios para definir una arquitectura RISC

- Analizar el objeto sobre el cual se desarrollan las aplicaciones a fin de determinar cuáles son las operaciones más frecuentes.
- Optimizar los caminos que deben recorrer los datos para ejecutar las operaciones o instrucciones (anteriores) tan rápido como sea posible

- Incluir otras instrucciones solo si forman parte de los caminos optimizados previamente y si son de relativa frecuencia y si su inclusión no hace más lenta la ejecución de las instrucciones más frecuentes: *“Cuidar de la definición de instrucciones adicionales que pudieran impactar la performance. Tener un criterio de no agregar instrucciones por agregar. No agregar una nueva instrucción de la arquitectura si puedo lograr una mezcla de dos. El arquitecto del RISC trata de definir pocas instrucciones, que son las que más usan pero a su vez la que más aprovechan la segmentación.”*
- Aplicar una estrategia similar a los otros recursos del procesador; incluir un recurso solo si éste está justificado por su frecuencia de uso, y su inclusión no enlentece a otros recursos más utilizados.
- Tratar de trasladar lo que más se pueda la complejidad en tiempos de ejecución al momento de compilación, recargando el software de compilación y liberando el hardware de ejecución.

Características Comunes en un arquitectura RISC

- Muchas de las instrucciones se ejecutan en un solo ciclo de memoria. *“Predictibilidad en tiempo de ejecución.”*
- Conjuntos de instrucciones para cargar/almacenar (load, store). Es decir, se accede a memoria exclusivamente mediante las instrucciones load y store. El resto de las instrucciones realizan sus operaciones entre registros. *“Esto también es una desventaja porque que haya conexión todos los registros contra todos los registros implica que haya muchos buses redundantes. Mucha complejidad para la transferencia dentro de esta arquitectura.”*
- Decodificación de instrucciones hardware (en oposición a la técnica de microprograma): *“Hay decodificación de instrucción a nivel cableado.”*
- Existen realmente pocas instrucciones y modos de direccionamiento
- Todas las instrucciones deberían tener la misma longitud para facilitar la tarea de decodificación y homogenizar los tiempos de carga de las mismas.
- La complejidad se ha desplazado hacia los compiladores optimizados: *“La cantidad de instrucciones de bajo nivel que genera una CISC es mucho menor que a la de una RISC. Un compilador genera muy pocas instrucciones de lenguaje de maquina en CISC, pero genera muchísimas instrucciones para una misma tarea.”*
- Existe un alto grado de pipeline en los caminos de los datos para obtener mucha concurrencia
- Gran cantidad de registros (Windowed o no-windowed)
- Muchos niveles de jerarquía de memorias
- Conjunto de instrucciones diseñado para determinada clase de aplicaciones: *“El conjunto de instrucciones es más orientado a aplicaciones de control.”*
- Soporta lenguajes de alto nivel (esto último debido a que como hay pocas instrucciones sobraría lugar en el chip para colocar elementos que ayuden a lenguajes de alto nivel, como manejo de listas, stacks, etc.). *“La gran escala de integración que se logra permite al mismo tiempo brindar mayor cantidad de registros de nivel 2, de alto nivel de impacto.”*

CISC vs RISC

	CISC	RISC
Set de instrucciones	Complejas que hacen más de una operación simultáneamente.	Simples. Una instrucción está obligada a hacer una única operación en máquina. Se necesitan muchas más instrucciones para realizar la misma tarea que una instrucción CISC.
Formato de instrucciones	Variable. No se basa en campos fijos (puede tener más campos además de CO – D).	Fijo. Todas las instrucciones tienen la misma longitud → mayor sencillez y mayor velocidad de procesamiento.
Compilador	Se busca acortar la brecha entre el lenguaje de alto nivel y lenguaje de máquina, haciendo un compilador más simple de fabricar (programa traductor entre ambos niveles) y reduciendo los tiempos de compilación.	Se requiere un compilador complejo e inteligente para traducir un lenguaje de alto nivel a este tipo de instrucciones simples
Modos de direccionamiento	Muchos	Pocos
Acceso a memoria	Cualquier instrucción tienen acceso a memoria.	Se utilizan dos instrucciones para ordenar mover datos de memoria a registro y en sentido contrario: LOAD y STORE. Pero se trata de utilizarlas lo menos posible porque requieren dos pulsos de reloj.
Ciclos de reloj por instrucción	Las instrucciones complejas requieren más de un ciclo para su ejecución.	Las instrucciones simples requieren un solo pulso de reloj, luego de haber sido decodificados.

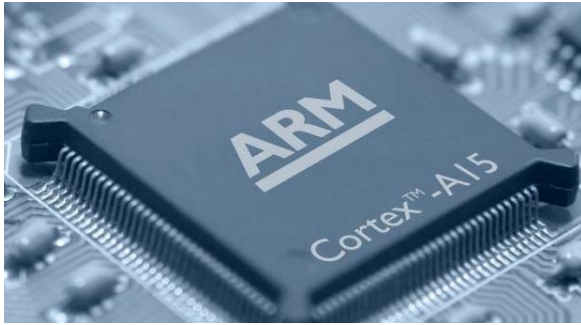
		Muchas de las instrucciones se ejecutan en un solo ciclo de memoria. "Predictibilidad en tiempo de ejecución."
Memoria de control	Necesariamente debe contener una memoria de control con los microcódigos para poder ejecutar las instrucciones complejas.	No contiene memoria de control (de microcódigos), lo que permite ganar mayor velocidad de procesamiento ya que se evita el acceso a la misma.
Unidad de control	La unidad de control debe ser microprogramada para lograr todas las características de la CISC porque son instrucciones de máquina muy complejas, el cableado es imposible porque se tendrían que cablear millones de instrucciones que den soporte a una instrucción de máquina compleja. Cada instrucción de máquina es interpretada por un microprograma localizado en una memoria.	Unidad de control cableada para mayor eficiencia debido a la sencillez de la arquitectura e implementación de operaciones por hardware (provoca hasta cablear algunas operaciones aritméticas en pos de lograr mayor velocidad).
Velocidad de procesamiento	Reducida por los accesos a memoria de control	Mayor ya que se evita el acceso a memoria de control (no tiene).
Registros	Agrega registros de propósitos específicos (como registros bases, registros índices, etc.)	Gran cantidad de registros. Todas las instrucciones (menos LOAD/ STORE) realizan sus operaciones entre registros. Esto es una desventaja porque que haya conexión todos los registros contra todos los registros implica que haya muchos buses redundantes. Mucha complejidad para la transferencia dentro de esta arquitectura.
Decodificación de las instrucciones	Las instrucciones compuestas son decodificadas internamente y ejecutadas con una serie de microinstrucciones almacenadas en una ROM interna: "La única manera de ejecutar una instrucción compleja es codificándola por partes, y a su vez, por cada grupo subdecodificado lanzar un conjunto de microórdenes; como una codificación por campos. La instrucción en el formato tan complejo que está hay que subdividirlas. Una instrucción de máquina compleja está relacionada con varios cronogramas distintos."	Decodificación de instrucciones hardware (en oposición a la técnica de microprograma). Hay decodificación de instrucción a nivel cableado.

Unidad 11

Microcontroladores

Un **microcontrolador** es un circuito integrado programable con capacidad de ejecutar órdenes que fueron grabadas en su memoria. Además, contiene todos los elementos de una computadora, con la diferencia que su capacidad es menor y generalmente se lo destina a una tarea específica. Son dispositivos que operan uno o más procesos. Por lo general los microcontroladores están basados en la arquitectura de Harvard, la cual consiste en dispositivos de almacenamiento separados (memoria de programa y memoria de datos).

El termino microcontrolador está dado por dos palabras que son “*Micro*” y “*Controlador*” las cuales tienen por significado “pequeño (en tamaño)” y “manipular o controlar (función principal)” procesos los cuales son definidos mediante la programación.



Ofrecen una solución práctica a muchos problemas de diversos campos. Algunos ejemplos:

- Un **microcontrolador** es un **procesador “dedicado”**. Esto significa:

- ## DIFERENCIA ENTRE MICROCONTROLADOR Y MICROPROCESADOR

108

abierto (configuración variable) con el que puede construirse una computadora con las características que se desee, acoplándole los módulos necesarios.

	Microprocesadores	Microcontroladores
CPU	El microprocesador tiene mucha más potencia de cálculo, por lo cual solamente realiza sus funciones con lo que tiene (datos) y su algoritmo o programa establecida	Es una de sus partes principales, la cual se encarga de dirigir sus operaciones.
Memorias RAM y ROM	Son dispositivos externos que lo complementan para su óptimo funcionamiento	Las incluye en un solo circuito integrado.
Velocidad de operación	Rápida	Lenta en comparación con la de un microprocesador
Tamaño	La configuración mínima básica de un Microprocesador está constituida por un Microprocesador, una memoria RAM, una memoria ROM, un decodificador de direcciones, lo cual lo convierte en un circuito bastante engorroso.	El Microcontrolador incluye todo estos elementos en un solo Circuito Integrado por lo que implica una gran ventaja en varios factores, como por ejemplo, la disminución en el tamaño del circuito impreso por la reducción de los circuitos externos.
Costos	Para el Microprocesador, el costo es muy alto en la actualidad.	El costo para un sistema basado en Microcontrolador es mucho menor.
Interferencias	Son más susceptibles a la interferencia electromagnética debido a su tamaño y a su cableado externo que lo hace más propenso al ruido.	El alto nivel de integración reduce los niveles de interferencia electromagnética
Tiempo de desarrollo	El tiempo de desarrollo de un microprocesador es lento.	Por el contrario, el de un microcontrolador es rápido.

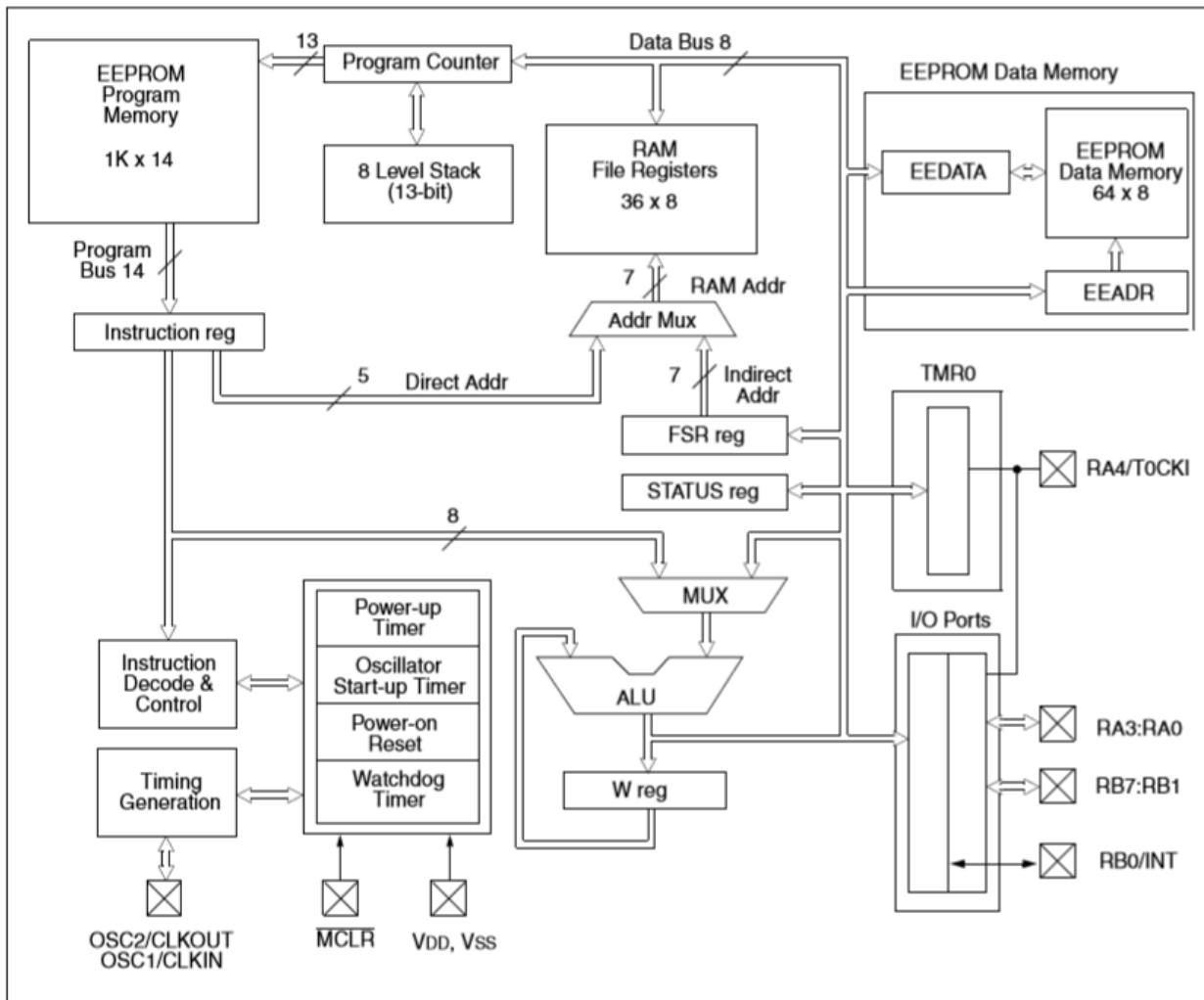
ARQUITECTURA DEL MICROCONTROLADOR

Un microcontrolador posee:

1. Procesador
2. Memoria no volátil para almacenar el programa
3. Memoria de Lectura / Escritura para guardar los datos
4. Líneas I/O para los controladores de periféricos:
 - a. Transmisión en Paralelo
 - b. Transmisión en Serie
 - c. Diversos ports de comunicación (ej.: USB)
5. Recursos auxiliares:
 - a. Circuito de Reloj
 - b. Temporizadores
 - c. WatchDog.
 - d. Conversores (AD y DA)
 - e. Comparadores analógicos
 - f. Protección ante fallas de alimentación
 - g. Estado de reposo o de bajo consumo

Los microcontroladores PIC 16C84 poseen:

- **Procesador tipo RISC:** Posee un juego de instrucciones reducido y simple, de manera que CASI todas las instrucciones se ejecutan en un solo ciclo de instrucción.
- **Procesador segmentado:** Se pueden realizar simultáneamente las dos fases en que se descompone la instrucción, es decir que mientras se está desarrollando la fase de ejecución de una instrucción se realiza la fase de búsqueda de la siguiente.
- **Arquitectura HARDVARD:** En este tipo de Arquitectura existen dos memorias separadas, una para instrucciones y otra para datos y cada una posee su sistema de buses para el acceso. Esto permite además del paralelismo de operación, que las memorias de instrucciones y datos puedan tener los tamaños adecuados (el caso particular de este modelo es 14 bits para instrucciones y un byte para datos).



Un **ciclo de Instrucción** está dividido en cuatro partes llamadas ciclos "Q" (Q_1 , Q_2 , Q_3 , Q_4). La búsqueda y la ejecución de una instrucción necesitan dos ciclos de instrucción. Debido a la separación de las memorias se puede realizar la búsqueda de una instrucción (memoria de programa) y solaparla con la ejecución de la anterior (memoria de datos). Este mecanismo se conoce como *pipeline*. Es por esto que cada instrucción se ejecuta efectivamente en un solo ciclo de instrucción, excepto que la instrucción sea un salto, caso en el cual se necesitan los dos ciclos.

El ciclo de búsqueda comienza con el incremento del contador en Q_1 . En el ciclo de ejecución, la instrucción sacada de memoria es colocada en el registro de instrucción en Q_4 . En la fase de ejecución se decodifica y ejecuta la instrucción. Específicamente, la lectura del operando desde la memoria de datos se hace en Q_2 (operand read) y la escritura durante Q_4 (destination write).

MANUAL PIC 16C84

Descripciones Generales

El PIC 16c84 es un microcontrolador de bajo costo, alto rendimiento, CMOS, completamente estático y de 8 bit.

Todos los microcontroladores PIC 16/17 emplean una arquitectura avanzada RISC. Los dispositivos PIC 16xxx han mejorado características del núcleo, ocho niveles de pila profunda y múltiples internas y externas fuentes de interrupción. Los buses separados de instrucciones y datos de la arquitectura Harvard permiten un bus de 14 bit para instrucciones y separado un bus de 8 bits para los datos. Pipeline permite que las instrucciones de dos etapas se ejecuten en un solo ciclo, exceptuando los saltos. Un total de 35 instrucciones están disponibles. Además, un gran conjunto de registros es usado para lograr un nivel de alto rendimiento.

El PIC16c84 tiene 36 bytes de RAM, 64 bytes de una memoria de datos EEPROM y 13 pins de E/S. También se encuentra disponible un timer/counter.

La familia PIC16xxx tiene especiales características para reducir los componentes externos, reduciendo el costo, mejorando la fiabilidad del sistema y reduciendo el consumo.

Los dispositivos PIC16C84 contienen una ALU de 8 bits y un registro "Working". La ALU es una unidad aritmética-lógica. Esta permite operaciones aritméticas y booleanas entre el contenido del registro W y cualquier otro registro. Las operaciones aritméticas son naturalmente en complementos a dos. En instrucciones con dos operandos, un operando debe estar en el registro W y el otro operando es el contenido de un registro o una constante. En instrucciones de un operando puede ser cualquiera de estos. El registro W es un registro de 8 bit usado para las operaciones con la ALU. Pero no es direccionable. Dependiendo de que instrucción se trate, las operaciones en la ALU pueden afectar los valores de los bits Carry (C), Digit Carry (DG) y Zero (Z) en el registro STATUS.

Organización de la Memoria

Existen dos bloques de memoria en el PIC16C84. Estos son la memoria de programas y la memoria de datos. Cada bloque tiene su propio bus por lo tanto, el acceso a cada bloque puede ocurrir durante el mismo ciclo.

La memoria de datos está dividida en dos zonas "General Purpose Register" (GPR) y "Special Functions Registers" (SFR). Los 64 bytes de la memoria EEPROM de datos tienen para direccionar el rango 0h-3Fh.

Los microcontroladores PIC 16XXX tienen un contador de programa de 13 bits capaz de direccionar 8 K palabras de 14 bits. El 16C84 implementa solamente 1K palabras de 14 bits (0000h – 03FFh). Acceder a una posición superior (si bien se puede hacer) produce el acceso a la posición de los 10 bits menos significativos. Es decir, acceder a la posición 44h, 444h, 844h, C44h, 1044h, 1444h, 1844h o 1C44h es obtener la instrucción ubicada en la posición 44h.

El contador de programa está implementado sobre la memoria de datos. Al tener implementado solo 1 K posiciones de memoria de programa, se ignoran los tres bits más significativos del contador de programa.

La dirección 0000h está reservada para el vector de RESET y la 0004h para el vector de INTERRUPCION.

Unidad 12

Lenguaje de Máquina

LENGUAJE DE MÁQUINA

El lenguaje máquina es el único lenguaje que puede ejecutar una computadora, es específico en cada arquitectura, es un código que es interpretado directamente por el microprocesador, está compuesto por un conjunto de instrucciones ejecutadas en secuencia que representan acciones que la máquina podrá tomar.

El lenguaje máquina es el único que entiende directamente la computadora, utiliza el alfabeto binario.

Las **desventajas** de un programa en lenguaje de maquina son:

- Extremadamente difícil
- Costoso
- Insume mucho tiempo
- Es probable la existencia de numerosos errores
- Muy difícil de efectuar correcciones

Las **ventajas** de un programa en lenguaje de maquina son:

- Posibilidad de cargar (transferir un programa a memoria) sin necesidad de traducción posterior, lo que supone una velocidad de ejecución superior a cualquier otro lenguaje de programación.

Programación en lenguaje de maquina

A la computadora para que lleve a cabo un trabajo debemos suministrarle *instrucciones*.

Calcularemos la siguiente expresión en lenguaje de máquina: $A = (B+3D)/(C+E)$. Aunque la máquina opera en binario, escribiremos nuestro programa en la representación octal para facilitar su lectura.

Supondremos:

1. Que la instrucción va codificada con 18 bits; 6 de los bits, es decir 2 cifras octales, para el código de operación y 12 bits, o sea 4 cifras octales, para la dirección.
2. Que los valores binarios de b, c, d, e se encuentran en las células de memoria de dirección octal 40, 41, 42, 43; que el resultado a debe almacenarse en la dirección octal 100; que las posiciones de memoria 101, 102, Están libres.
3. Que en la dirección 77 octal está almacenado el valor 3 en binario; que los códigos de operación responden al siguiente cuadro de codificación:

Carga del acumulador	CAR	00
Almacenamiento del acumulador	ALM	01
Suma	SUM	02
Sustracción	SUS	03
Multipliación	MUL	04
División	DIV	05

4. Que el programa comienza en la dirección 15 (en octal).

El programa resulta:

POSICION EN MEMORIA	INSTRUCCIÓN		COMENTARIOS
	C.O.	DIRECCION	
.....			Instrucción precedente
0015	00	0043	Carga e en el acumulador
0016	02	0041	Suma c al acumulador
0017	01	0101	Almacena $c + e$ en memoria
0020	00	0042	Carga d en el acumulador
0021	04	0077	Multiplica por 3
0022	02	0040	Suma b
0023	05	0101	Divide por $c + e$
0024	01	0100	Almacena el resultado a
0025			Siguiente instrucción
.....			
0040			Valor de b
0041			Valor de c
0042			Valor de d
0043			Valor de e
.....			
0077	00	0003	Valor 3
0100			Valor de a
0101			Posiciones de trabajo en la memoria
.....			

INTRODUCCION AL LENGUAJE ENSAMBLADOR

Los traductores pueden dividirse a grandes rasgos en dos grandes grupos, dependiendo de la relación entre el lenguaje fuente y el lenguaje objetivo.

- Si el lenguaje fuente es en lo esencial una representación simbólica de un lenguaje de maquina numérico, el traductor se llama **ensamblador** y el lenguaje fuente se llama **lenguaje ensamblador**.
- Si el lenguaje fuente es un lenguaje de alto nivel como Java o C y el lenguaje objetivo es un lenguaje de maquina numérico o una representación simbólica de tal lenguaje, el traductor se llama **compilador**.

Un **lenguaje ensamblador** puro es un lenguaje en el que cada enunciado produce exactamente una instrucción de máquina. En otras palabras, existe una correspondencia uno a uno entre las instrucciones de máquina y los enunciados del programa en ensamblador. Si cada línea del programa en lenguaje ensamblador contiene exactamente un enunciado y cada palabra de maquina contiene exactamente una instrucción de máquina, entonces un programa ensamblador de n líneas producirá un programa en lenguaje de máquina de n palabras.

En el **lenguaje ensamblador o lenguaje simbólico**, el programador escribe las mismas instrucciones que el lenguaje de máquina, pero usando nombres nemotécnicos para los códigos de operación y desentendiéndose de las posiciones en memoria, ya que pueden anotarse en forma simbólica las direcciones de operandos e instrucciones.

El ensamblador produce, a partir del programa fuente escrito en lenguaje simbólico un programa objeto transcrito instrucción por instrucción en lenguaje de máquina. Para poder ser ejecutado, este programa objeto debe ser almacenado en memoria con ayuda de un cargador.

Existen dos tipos de ensambladores:

1. **No trasladables:** que transcriben directamente las direcciones simbólicas en direcciones absolutas.
2. **Trasladables:** que autorizan la carga del programa a partir de cualquier dirección de memoria. El ensamblador transcribe todas las direcciones simbólicas como si el programa tuviera que ser implantado a partir de la dirección cero, y es el cargador quien se ocupa de ejecutar los necesarios traslados.

Las **ventajas** del lenguaje ensamblador son:

- *Más fácil de programar en comparación a lenguaje de maquina:* La razón por la que la gente usa lenguaje ensamblador en lugar de programar en lenguaje de maquina es que es mucho *más fácil programar en lenguaje ensamblador*. El uso de nombres simbólicos y direcciones simbólicas en lugar de direcciones binarias u octales representa una gran diferencia. El programador en lenguaje ensamblador solo tiene que recordar los nombres simbólicos porque el ensamblador los traduce a instrucciones de máquina.
- *Correspondencia uno a uno entre enunciados en lenguaje ensamblador e instrucciones de máquina,* a diferencia de los lenguajes de alto nivel.
- *Todo lo que puede hacerse en lenguaje de maquina puede hacer en lenguaje ensamblador;* pero muchas instrucciones, registros y características similares no están disponibles para el programador en lenguaje de alto nivel.
- Un programador experto en lenguaje ensamblador a menudo puede producir código que es mucho más pequeño y rápido que el que puede producir un programador en lenguaje de alto nivel.

Las **desventajas** del lenguaje ensamblador son:

- Un lenguaje ensamblador solo puede ejecutarse en una familia de máquinas, un lenguaje de alto nivel tiene el potencial para ejecutarse en muchas maquinas.
- Los programas son extensos y complejos al igual que en lenguaje de maquina
- Un programa en lenguaje ensamblador toma mucho más tiempo en escribir, tarda mucho más en depurarse y es mucho más difícil de mantener que el mismo programa en un lenguaje de alto nivel.

Programación en lenguaje ensamblador

El mismo programa escrito en la sección de lenguaje de maquina pero ahora en lenguaje ensamblador:

Lenguaje de Máquina	Ensamblador	Instrucción	Lenguaje de Máquina	Ensamblador	Instrucción
000	SAN	Salto Cond	001	CAR	Cargar
010	SUM	Sumar	011	ALM	Almacenar
100	MUL	Multiplicar	101	SAL	Salto Incond
110	DIV	Dividir	111	SUS	Restar

Calcular la siguiente expresión: $A = (B+3D)/(C+E)$

Cód. Op.	Direcc.Simbólica	Direcc. Simbólica	Pseudo Instrucción
CAR	E	B	RES1
SUM	C	C	RES1
ALM	DEN1	D	RES1
CAR	D	E	RES1
MUL	TRES		
SUM	B	TRES	ATR3
DIV	DEN1	A	RES1
ALM	A	DEN1	RES1

LENGUAJES EVOLUCIONADOS O DE ALTO NIVEL

Los **lenguajes** de programación de **alto nivel** (Ada, BASIC, COBOL, FORTRAN, Modula-2, Pascal, etc.) son aquellos en los que las instrucciones o sentencias a la computadora son escritas con palabras similares a los lenguajes humanos (en general lenguaje inglés) lo que facilita la escritura y la fácil comprensión por el programador.

Estos lenguajes constituyen el gran paso hacia la mínima complejidad y la máxima universalidad. Reducen al máximo la cantidad de detalles a considerar por el programador humano, se logra esto transfiriendo la mayor parte del esfuerzo de comunicación a la máquina.

Las **ventajas** de los lenguajes de alto nivel son:

- Los programas escritos en lenguaje de alto nivel funcionan directamente o pueden funcionar con muy pocas modificaciones en distintos equipos
- El tiempo de formación de programadores es corto comparándolo con el necesario para lenguajes inferiores
- El programador no necesita conocer las características del funcionamiento del ordenador donde se correrá el programa
- El tiempo de codificación y puesta a punto de los programas es considerablemente menor
- Los cambios y correcciones del programa son más fáciles
- Se reduce costo y mantenimiento de los programas

Las **desventajas** de los lenguajes de alto nivel son:

- Aumento del tiempo de compilación
- No aprovechamiento de posibles ventajas de la arquitectura interna del sistema
- Se incrementa la ocupación de memoria central, tanto por el propio compilador como por el programa objeto resultante
- El tiempo de ejecución es mayor, ya que las instrucciones generadas por el compilador son más numerosas que las correspondientes al mismo programa escrito directamente en lenguaje de máquina

Programación en lenguaje de alto nivel

Calcular la siguiente expresión: $a = (b+3d):(c+e)$

En la mayoría de los lenguajes evolucionados se escribirá:

$$A = (B + 3D) / (C + E)$$

COMPILADORES E INTÉRPRETES

El **compilador** es el programa que analiza un programa escrito en lenguaje de alto nivel y produce un código objeto, susceptible de ser cargado por otro programa (cargador) para su ejecución. El compilador es un programa que siempre está dado en lenguaje de máquina.

Se denomina **intérprete** a aquellos programas efectúan traducción y ejecución sucesiva, instrucción a instrucción. Difieren entonces de los compiladores en que estos traducen el programa completo sin ejecutarlo a medida que avanzan en la traducción. El intérprete reside en la memoria central con el programa por ejecutar, lee las instrucciones en lenguaje de alto nivel, detecta los errores, los comunica, y si no hay errores convierte las instrucciones en código interno y las ejecuta cuando se lo indica.

Las **diferencias** entre compilador e intérprete son:

- La memoria necesaria para procesar con un intérprete es mayor que la necesaria para procesar con un programa compilado.
- El programa compilado se ejecuta más rápidamente que el interpretado.
- Es más fácil programar con un intérprete ya que nos avisa de los errores tan pronto como los cometemos.

Unidad 13

Intercambios de Información con el Exterior

“Toda arquitectura se encontrará ejecutando un programa en curso pero también debe atender a requerimientos de dispositivos de entrada/salida. La arquitectura no puede desconocer este vínculo entre ella y los dispositivos de E/S que hacen al ordenador. Entonces, la arquitectura aparte de ocupar ciclos de memoria para el programa en curso debe atender determinadas interrupciones efectuadas por determinados dispositivos.”

“Para poder establecer una comunicación hacia el exterior, o desde el exterior vamos a tener que compartir con el programa en curso el órgano memoria. El problema más grave es que los dispositivos externos no trabajan con la misma frecuencia que la memoria.”

*“El impacto producido por las diferencias de velocidades entre la memoria central y los dispositivos externos hicieron necesario el desarrollo de canales que pudieran automatizar estas tareas y liberar a la CPU para poder continuar con la ejecución del programa en curso. Entonces, un **canal** es un dispositivo lógico que vincula a la memoria central con el conjunto de controladores de periféricos con el fin de automatizar de la mejor manera posible los procesos de transferencias de información desde la memoria a los periféricos y viceversa.”*

“La comunicación puede darse por dos motivos y en ambos casos se consumen ciclo de memoria:

- a. Interrupción de un periférico: teclado, mouse o pantalla táctil interrumpen al procesador y solicitan ciclos de memoria para actualizar su estado.*
- b. Una instrucción de entrada salida del programa en curso solicita transferir desde o hacia un periférico.*

En todos estos casos el canal deberá tomar el control de la transferencia con el fin de minimizar el impacto producido por las asimetrías en los tiempos de respuesta de la memoria central y de los periféricos.”

INTRODUCCIÓN

Los intercambios de información con el exterior exigen dos tipos de unidades entre el ordenador o su memoria, por una parte, y los órganos periféricos por otra.

1. Las **unidades de intercambio o canales** capacitan para las transferencias de información entre el calculador y cualquier unidad externa.
2. Las **unidades de control**, o unidades de **enlace** o también **controladores de periféricos**, que se encargan de gobernar uno o varios órganos periféricos de una determinada clase, obedeciendo órdenes recibidas del calculador.

A estos dos tipos de unidades se añadirá el sistema de **interrupciones** del calculador, que permite al medio exterior avisar al calculador sobre acontecimientos exteriores, como puede ser el final de una operación de entrada – salida.

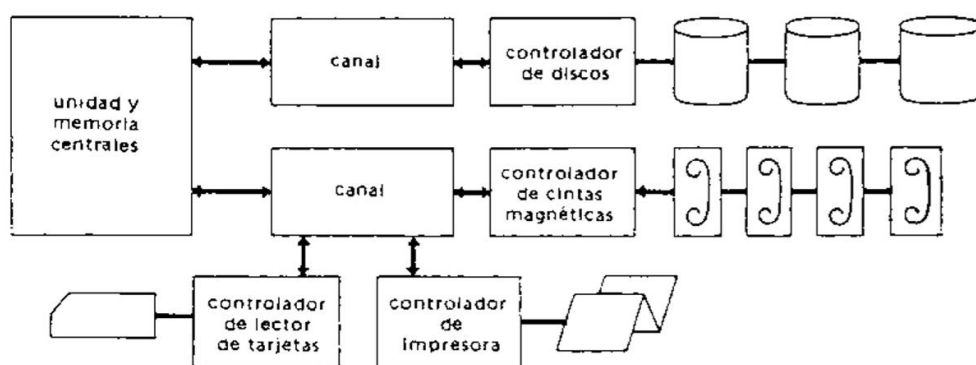


Fig. 1. Configuración ilustrativa de la función de los canales y de los controladores

SIMULTANEIDAD DE LOS PROCESOS DE ENTRADA/SALIDA

El problema radica en la diferencia de velocidad entre la ejecución de las instrucciones del programa en el computador y los ritmos de transferencia mucho más lentos impuestos por los periféricos. En un principio, tal problema se resolvió bloqueando al computador mientras que la operación de entrada/salida no estaba totalmente concluida. El aumento de potencia de las máquinas muy pronto hizo inadmisibles esta solución.

“Consiste en diseñar estrategias con el fin de lograr la mayor cantidad de simultaneidad en el uso de la CPU por parte del programa en curso y de las transferencias.”

Para resolver los problemas de simultaneidad se explicaran cronológicamente las diferentes ideas creadas:

1. **Modo Bloqueado:** el calculador ejecuta el programa de entrada/salida esperando, a cada nueva instrucción de transferencia, que el elemento periférico esté disponible. De esta forma, el calculador se encuentra bloqueado durante el desarrollo completo de la entrada/salida, con la consecuencia de que el porcentaje del tiempo activo de la unidad central puede llegar a ser muy pequeño (del 0,1% con las unidades periféricas más lentas).

“El calculador se detiene ante una instrucción de transferencia a la espera de que el dispositivo esté listo. Cuando se logra el control del periférico las transferencias se hacen al ritmo del dispositivo externo.”

2. **Modo por prueba de estado:** con el objeto de introducir una cierta simultaneidad, parece lógico no ordenar la operación de transferencia de una palabra hasta que la unidad periférica este presta a ejecutarla completamente. Para ello, se incluye en el programa, con una periodicidad adecuada, puntos de prueba para conocer si la unidad periférica estaba presta a realizar la operación. Esta técnica, muy enojosa para el programador, no puede hacer frente más que a necesidades muy particulares.

“El programador tiene la facultad de preguntar por el estado del periférico antes de usarlo. Se trata de una instrucción de prueba de estado del periférico que permite al programador la posibilidad de preguntar por el estado del periférico antes de transmitir. Es como un salto condicional pero por estado del periférico libre. El problema de este modo es que la actitud del programa depende del tiempo de respuesta del periférico.”

3. **Modo por interrupción de programa:** en este procedimiento, al calculador se le descarga de la vigilancia periódica de los periféricos y son estos los que lo interrumpen para avisarle que están disponibles. La interrupción permite detener momentáneamente el programa en curso para permitir que el programa de transferencia sea ejecutado con prioridad. El modo por interrupción permite cierta simultaneidad entre cálculos y entradas – salidas, retrasando al programa de cálculo una cantidad de tiempo igual a la suma de los tiempos de ejecución del programa de interrupción.

“No tiene el problema del anterior, que depende de respuesta del periférico porque es el periférico quien interrumpe al programa y el que le da ritmo a la rutina de transferencia.”

4. **Modo automático por suspensión de programa o modo canal:** En este procedimiento se crean unidades automáticas de intercambio o canales, capaces de tomar completamente a su carga la transferencia de todo un bloque de informaciones. *“Permite transferir una o varias palabras contiguas desde o hacia la memoria.”*

Cuando el canal necesita intercambiar una información con la memoria, no produce ya una interrupción de programa, sino simplemente una solicitud de servicio para que le sea concedido un ciclo de memoria, con objeto de realizar dicho intercambio. Por consiguiente, la simultaneidad entre programa y entrada/salida es casi total.

La realización de una operación de entrada/salida se desarrolla de la forma siguiente:

- I. Es el programa quien inicializa la operación: las instrucciones proveen al canal de todas las informaciones necesarias a la operación: número y emplazamiento de los datos por transferir, dirección de la unidad de entrada/salida, condiciones bajo las cuales debe terminarse la transferencia, etc.; una instrucción especial lanza la transferencia y el programa puede continuar independientemente.
- II. La transferencia se efectúa al ritmo del equipo periférico bajo control del canal, que roba ciclos al calculador.
- III. El canal avisa que la transferencia ha acabado generalmente mediante una interrupción. El programa de interrupción podrá preguntar entonces al canal en que condiciones se ha efectuado aquella.

Se distinguen dos tipos de canales automáticos: los **canales simples o selectores** que no pueden gestionar más de una transferencia a la vez, y los **canales multiplados** en el tiempo, capaces de gestionar varias transferencias simultáneamente.

5. **Encadenamiento automático de las transferencias:** En el modo precedente, el ordenador está obligado a inicializar cada transferencia cargando las informaciones necesarias en los registros del canal. A fin de aliviar al ordenador de esta tarea, se concibe el canal como una unidad poseedora de su propio programa.

La unidad central generará en memoria central un **programa de control** de entrada/salida o **programa de canal** definiendo las diferentes transferencias sucesivas a realizar por el canal. Para lanzar este programa de canal, la unidad central proporcionará su dirección al canal que se encargará de su ejecución completa. El programa de control de entrada/salida permite encadenar varias transferencias sin intervención de la unidad central.

Se distinguen dos clases de encadenamiento: el **encadenamiento de datos**, que después de transferir datos desde (o hacia) una tabla en memoria, permite pasar automáticamente a otra tabla; el **encadenamiento de funciones**, que permite hacer ejecutar sucesivamente varias transferencias sobre varios registros diferentes en un mismo órgano periférico.

“Es una mejora del modo anterior, porque aparte de lograr el automatismo de la transferencia también permite lograr que la información este dispersa en bloques, es decir, que no sea contigua, que es lo que sucede normalmente en la memoria.”

NOCIÓN DE INTERFASE

Se denomina **interfase** a las especificaciones de conexión entre un canal y cualquier tipo de unidad periférica. Es la caja negra que permite adaptar las señales del periférico a las especificaciones de la interfase ordenador.

“Todo canal se comunica con una controladora de periférico, por eso se diseñó la interfase. La interfase es un circuito lógico que permite iniciar, sincronizar y transferir información de un canal hacia una controladora.”

La interfase, está formada físicamente por un cierto número de conectores, salidos del canal, en los cuales pueden ir a enchufarse. Es necesario precisar además las especificaciones de formato y de tiempo de las señales dadas o esperadas por el canal.

Básicamente, la interfase **se compone** de dos tipos de hilos:

1. **Hilos de información**, que llevan en paralelo la información por transmitir, en forma de niveles;
2. **Hilos de gobierno y sincronización**, que capacitan para el diálogo de las dos unidades interconectadas.

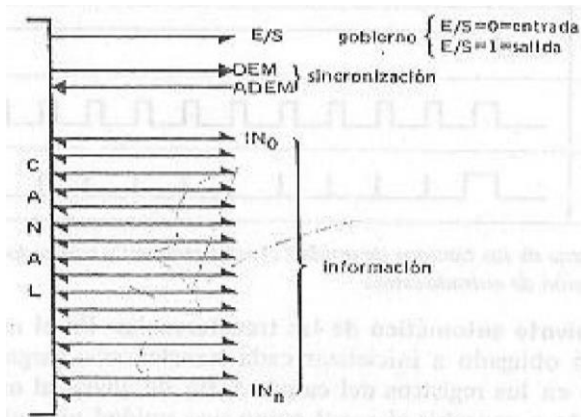


Fig. 3. Esquema de interfase

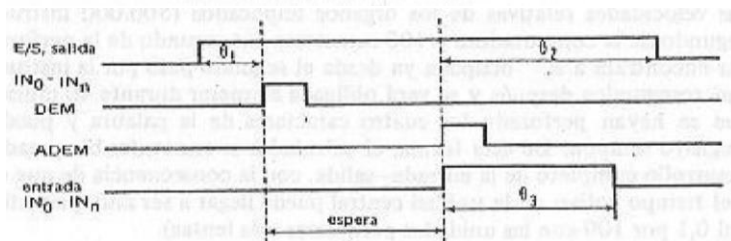


Fig. 4. Ejemplo de cronograma asociado a la interfase de la figura anterior¹

. Descripción de una salida de información: Descripción de una Salida de Información: La unidad de intercambio posicionará la señal de nivel E/S para indicar que se trata de una salida de información, posicionará la información por salir bajo la forma de niveles en las líneas de información IN0 a INn y, por último, enviará un impulso DEM de demanda de servicio. Después permanecerá en espera.

El equipo exterior reconocerá la demanda DEM, probará el hilo de gobierno E/S para saber si la operación consiste en una entrada o en una salida de información. Como E/S vale 1, el equipo exterior muestreará los hilos de información IN0 a INn (apertura, mediante un impulso, de las puertas de entrada a un registro tampón). Dicho impulso podrá ser también enviado al canal como señal de acuerdo, que indica que la demanda de salida ha sido reconocida y muestreada la información (ADEM).

. Descripción de una entrada de información: El canal pondrá E/S a 0 y enviará el impulso DEM. La solicitud de entrada será reconocida por el equipo periférico, que deberá efectuar los niveles de información IN0 a INn, y, después, enviará la señal de acuerdo ADEM. La lectura será realizada por el canal tras recepción de ADEM.

*“El **problema** que se presenta es que el canal debe poder comunicarse con varias controladoras (disco, usb, etc.). Existen dos técnicas de comunicación entre un canal y sus controladores: la primera es concentración por multiplexación y la segunda concentración por líneas ómnibus.”*

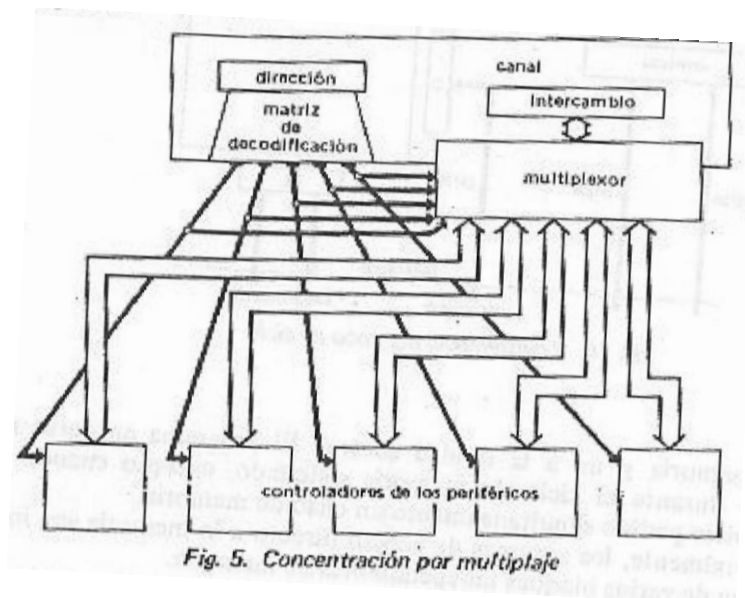
NOCIÓN DE MULTIPLAJE

El multiplexaje permite concentrar varias vías de entrada en una sola o distribuir una vía de salida en varias vías. También permiten conectar varias unidades externas a un mismo canal. A veces, se utilizará este método en los **canales simples**, es decir, aquellas unidades de intercambio capaces

solo de controlar una transferencia simultáneamente. Entonces el direccionamiento de las unidades externas se efectuará en estrella, un hilo por unidad, después de decodificar la dirección en el canal.

“La dirección del periférico es decodificada a través del canal, el cual realiza dos operaciones:

- a. Selecciona con una señal de nivel uno de los controladores afectados a la tarea*
- b. Establece un canal único de comunicación entre la memoria central y el controlador seleccionado”*

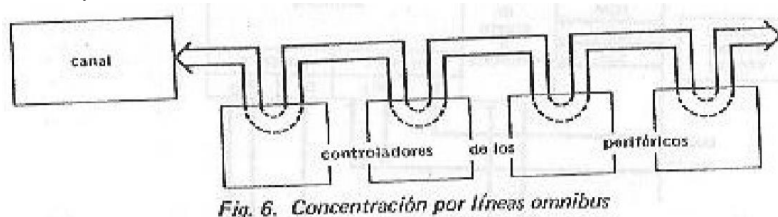


LÍNEAS ÓMNIBUS

Para conectar varias unidades externas en un canal, puede también emplearse una línea ómnibus que las una. Si la unidad externa no resultase seleccionada se transmite la información sin alterarla.

“Solo aquel controlador que identifique la dirección transmitida por el canal es el que lo va a tomar y liberar de la circulación para que venga una nueva palabra. Si la dirección transmitida no coincide con la del controlador, este deja pasar la información al siguiente y así sucesivamente.”

El direccionamiento puede realizarse siguiendo la técnica en estrella. Es posible, también, emplear la línea ómnibus para el direccionamiento: un hilo especial indica, entonces, que la información emitida por el canal es una dirección. La unidad externa atravesada comprueba este hilo a cada transferencia. Para el caso en que aquella detecte que se trata de una dirección, compara la dirección emitida con la suya propia. Si hubiera identidad entre ambas, la unidad externa se pone en estado de dialogo con el canal; si no sucediera así, transmite la dirección a la unidad contigua. Esta técnica de direccionamiento y de transferencias por bus la emplean, sobre todo, los canales multiplexados en el tiempo.



CONCENTRACIÓN Y FRACCIONAMIENTO DE LAS INFORMACIONES

Los periféricos trabajan normalmente a nivel de carácter (6 u 8 bits), mientras que los ordenadores lo hacen, en general, a nivel de la palabra (varios caracteres). Ello obliga, ya sea en el canal o en la unidad de control de un determinado periférico, a realizar las concentraciones y fraccionamientos pertinentes. Generalmente, esta operación se llevará a cabo en el canal, puesto que este es común a varios periféricos.

Las operaciones de fraccionamiento y concentración se conseguirán a menudo por sucesivos desplazamientos sobre dos registros supuestos uno seguido del otro: un registro de palabra de comunicación con la memoria y un registro de carácter de comunicación con el exterior.

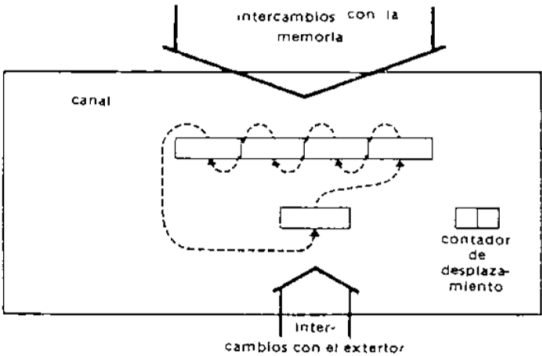


Fig. 7. Fraccionamiento de las palabras y concentración de los caracteres en el canal

En la figura, con motivo de una operación de salida, los 4 caracteres de la palabra de memoria serán enviados sucesivamente hacia el exterior, siendo transferido primero el carácter de mayor peso.

TECNICAS DE EJECUCIÓN DE UNA TRANSFERENCIAS ELEMENTAL

Llamaremos **transferencia elemental** a la transferencia de una información (carácter o palabra) desde una posición de memoria central a una unidad externa (o viceversa, según se trate de una entrada o salida).

*“Una **técnica de transferencia** es una estrategia planteada por la arquitectura con el fin de controlar de forma más eficiente un proceso de entrada salida con la memoria del computador.”*

La ejecución de una transferencia elemental supone que se conozca las informaciones siguientes:

- Información pidiendo el lanzamiento de la transferencia: DEM
- Indicación del sentido de la transferencia (entrada o salida): SEN
- Dirección de almacenamiento en memoria de la información transferida: DIR

Las diferentes técnicas de ejecución de una transferencia elemental dependen de la manera como sean suministradas al computador estas tres informaciones:

1. Transferencia Programada

Se ejecuta cuando el programa llega a una instrucción de transferencia (es una instrucción dentro del set de instrucciones de la maquina). Es el ordenador quien inicializa la demanda de transferencia (DEM).

Las informaciones complementarias son suministradas por la instrucción, que puede adoptar dos formas:

EN	DIR	Entrar información y almacenar en la dirección DIR
T		
SAL	DIR	Leer información almacenada en la dirección DIR y hacerla salir

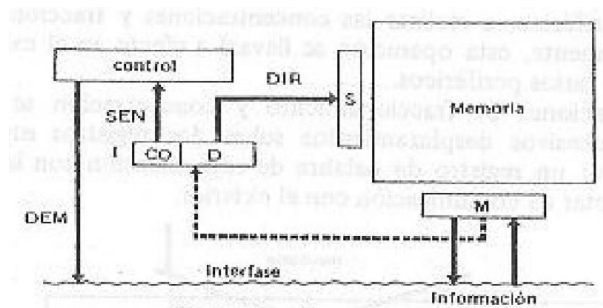


Fig. 8. Transferencia programada

“La instrucción puede ser de entrada o salida y esto va a ser determinado dependiendo del código de operación (CO), es decir, el CO le brinda al controlador el sentido de la transferencia. Si es entrada activa ENM y realiza una escritura (ESC). Si es salida activa SRM y realiza una lectura (LEC). Pero en ambos casos realiza las señales inicio de ciclo de memoria (ICM) y puesta a cero del registro palabra de memoria (PACM).”

“El dispositivo **control** de transferencia es parte de la unidad de control. Este recibe si la instrucción es de entrada o salida y dependiendo de esto articula la entrada o salida, desde o hacia el dispositivo externo.”

2. Transferencia por instrucción forzada

La transferencia por instrucción forzada es inicializada por la unidad externa, que genera la instrucción de entrada/salida y fuerza su ejecución prioritaria en cuanto que la unidad central ha concluido su instrucción en curso. El programa queda suspendido durante el tiempo de ejecución de esta instrucción.

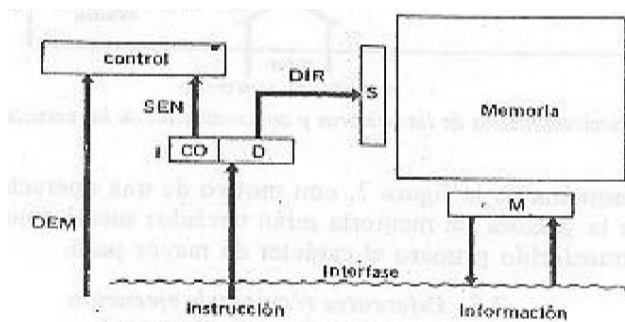


Fig. 9. Transferencia por instrucción forzada

“Por ejemplo, el controlador del mouse, genera la instrucción cuando uno mueve el mouse.”

“Antes que el contador de programa se incremente ingresa esta instrucción, por tanto, es la que se va a ejecutar y por eso recibe el nombre de forzada.”

3. Transferencia por robo de ciclo

La transferencia por robo de ciclo se inicializa a petición de la unidad exterior. La unidad central termina el ciclo de memoria en curso y concede el próximo ciclo a la unidad exterior. Esta indica si se trata de una lectura o de una escritura y proporciona la dirección de almacenamiento. El programa en curso queda en suspenso durante un ciclo de memoria.

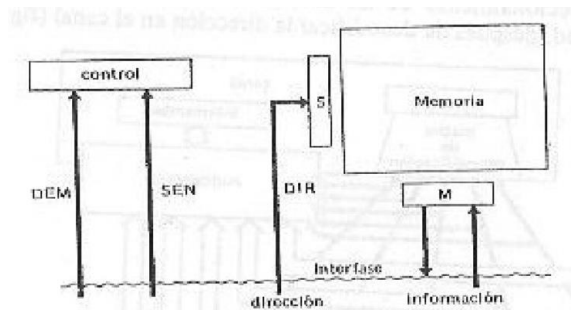


Fig. 10. Transferencia por robo de ciclo

“Robo de ciclo es más óptima porque no genera una instrucción que tiene que entrar a la mezcla de instrucciones, va directamente a la lectura o grabación de memoria.”

“El CO va directamente a la unidad de control de transferencia y la dirección se mete directamente al registro de selección de memoria. No pasa por el registro I. Esto quiere decir que ni bien termina la instrucción anterior lo realiza, es decir, entre una búsqueda de instrucción y otra búsqueda de instrucción, entre dos tratamientos de instrucción roba un ciclo. Es más veloz.”

“No suspende el programa en curso solamente roba un ciclo. Apenas está libre el ciclo de memoria realiza la transferencia.”

4. Transferencia por acceso directo a memoria

Se supone que la memoria posee dos vías de acceso (podrían llamarse muelles de acceso). Una de las vías está reservada al ordenador, la otra a la unidad exterior. Las peticiones de ciclo de memoria son dirigidas a la unidad de control de acceso a memoria, tanto si proceden de la unidad central como de la unidad exterior. Esta unidad de control gestiona las prioridades de acceso de la unidad central y de la unidad exterior.

El programa en curso no resulta suspendido durante el ciclo de memoria solicitado, excepto cuando la unidad central hubiera pedido simultáneamente un ciclo de memoria.

Este sistema es interesante si se dispone de varios bloques independientes de memoria.

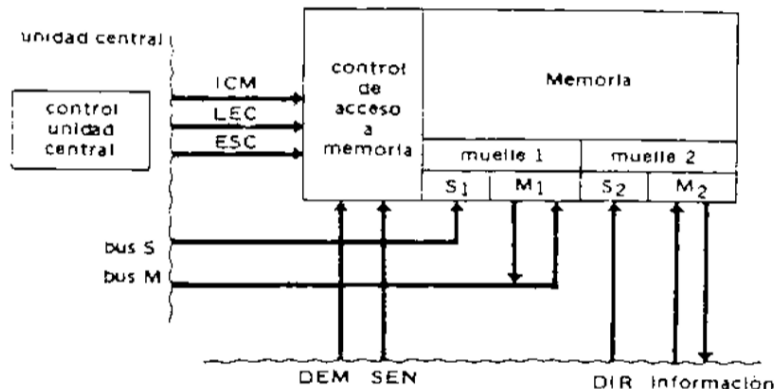


Fig. 11. Transferencia por acceso directo a la memoria

“Una palabra de memoria puede ser accedida o bien por una dirección del registro selección 1 (S1) y sacada por el registro palabra de memoria 1 (M1). Pero la misma palabra también puede ser seleccionada por otro acceso (S2) y sacada por M2. Se tiene dos formas de acceso a una palabra.”

*“El **objetivo** de tener dos accesos es dedicar el muelle 1 exclusivamente para el bus S y M, es decir, el programa en curso y el muelle 2 exclusivamente para la comunicación con el exterior entonces se puede lograr un cierto paralelismo. En la misma lectura se podría acceder a una palabra distinta a otra. Esto me permite que el mismo ciclo de memoria, con suerte, pueda estar destinado para buscar un operando para el programa en curso pero también para estar leyendo una palabra de*

memoria. Lo que no se puede hacer es grabar y leer, o grabas o lees. En el mismo ciclo puedes estar leyendo una nueva instrucción o un nuevo operando y simultáneamente una palabra que no esté en conflicto con las direcciones del operando o de la instrucción. También se puede estar grabando el resultado de un operando producto del programa en curso pero también guardando información del exterior en otra zona.”

Comparación de las diferentes técnicas

TECNICA EMPLEADA	TIEMPO MAXIMO DE ESPERA DE LA UNIDAD EXTERIOR	TIEMPO RESTADO AL PROGRAMA EN CURSO
Programada	Hasta el final de la instrucción en curso, más inicio del programa solicitado de interrupción.	Procesamiento de un programa de interrupción.
Por instrucción forzada	Duración de la instrucción más larga del repertorio.	Una instrucción generalmente ejecutada en uno o dos ciclos de memoria.
Por robo de ciclo	Un ciclo de memoria.	Un ciclo de memoria.
Por acceso directo	Un ciclo de memoria.	Nada, o un ciclo de memoria.

CANALES

1. Enlace Programado

“Es una técnica, interfase lógica para poder gobernar a un periférico ante la acción de una instrucción de entrada/salida. Es un esquema lógico que permite gobernar y sincronizar las transferencias con el exterior en el modelo abacus. No llega a ser un canal.”

No es propiamente un canal sino un medio de acceso controlado por el programa. En efecto, cada transferencia elemental está gobernada por una instrucción del programa en curso. Por el hecho de estar controlados por el programa, tales enlaces pueden utilizarse tanto en modo bloqueado como, mucho más a menudo, en modo por interrupción. Casi siempre aparecen integrados en la unidad central.

Son necesarias cuatro instrucciones de entrada/salida para realizar entradas/salidas en modo programado:

Instrucciones de gobierno de transferencias de información	SAL M	Para extraer el contenido de la dirección M hacia las líneas de salida de información de la interfase.
	ENT M	Para almacenar en la dirección M una información presente en las líneas de entrada de información a la interfase.
Instrucciones de control general de las operaciones de entrada – salida	GCP	Permite direccionar una unidad de control de periférico y especificarle la operación por ejecutar.
	PRE	Permite adquirir una información descriptiva del estado del periférico anteriormente direccionado.

Las señales de interfase SI, NI, SC y NC corresponden respectivamente a estos cuatro tipos de información. Además:

- SI: niveles lógicos posicionados sobre las líneas de salida de información.
- NI: peticiones de entrada de información.
- SC: niveles lógicos posicionados sobre las líneas de salida de control.
- NC: peticiones de entrada de estados.

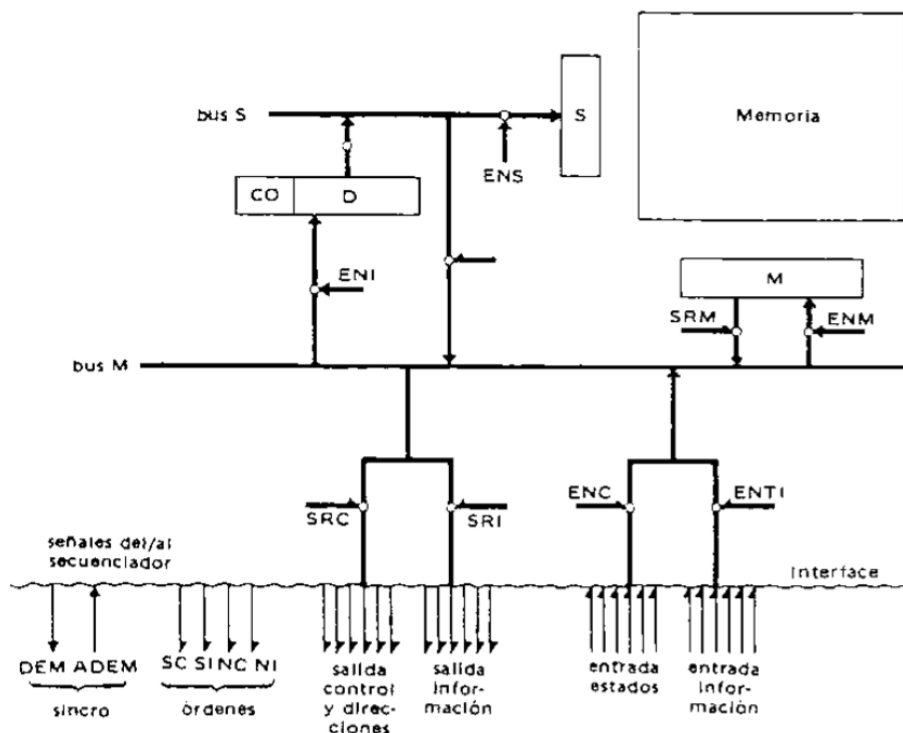


Fig. 12. Enlace programado de Abacus

“Este esquema solo permite transmitir una sola palabra desde o hacia el periférico. Pero no permite bloques, solamente una palabra.”

“El enlace programado trabaja por transferencia programada y con una transferencia por línea omnibus.”

Aclaración: Al ser enlace programado la operación va a estar definida por el CO y la instrucción va a estar definida en el set de instrucciones, es decir, se realizará una instrucción de transferencia cuando la instrucción a realizar en el contador de programa sea una transferencia. El secuenciador será el encargado de detectar el CO y además de activar DEM en caso de ser necesario.

En la **operación de salida SAL**, la información permanece memorizada en el registro M hasta ser tomada en cuenta por el órgano exterior, las puertas SRM y SRI permanecen abiertas para

mantener activos los niveles lógicos de las líneas de salida de información y la señal SI (salida de niveles de información) permanece activa.

El computador envía entonces al controlador del periférico previamente escogido, una señal impulsional DEM. Al recibirla éste, suponiendo que está disponible, comprueba los hilos de órdenes y tras haber detectado que se trata de una salida, muestrea la información y trasmite al computador una señal de acuerdo ADEM para liberarle.

En la **operación de entrada ENT**, el ordenador posiciona la señal de orden de entrada de información NI y realiza una demanda de servicio DEM. Luego el periférico comprueba los hilos de órdenes y reconoce una demanda de información. El ordenador posiciona las líneas de entrada de información activando ENTI y el periférico envía la información por las líneas de entrada de información. El ordenador muestrea el registro M mediante ENM y el periférico finaliza la operación con la señal ADEM.

En la operación **GCP** se activa TSM SRC DEM SC ADEM.

En la operación **PRE** se activan TSM SRC NI salida de control luego se producen entrada de estados ENC ENM.

2. Canal automático: Modo Canal

Los canales automáticos son capaces de gestionar el conjunto de una operación de entrada/salida sin intervención de la unidad central, excepto para inicializar y concluir dicha operación. Las transferencias elementales pueden ser efectuadas, bien por instrucción forzada, bien por robo de ciclo, bien por acceso directo a la memoria.

Se les llama **canales simples** cuando no gestionan más que una operación de entrada/salida a la vez.

Describiremos una unidad de este tipo en que las transferencias se consiguen por robo de ciclo, interesándonos exclusivamente en los problemas de intercambio de informaciones.

Desde este estricto punto de vista, la operación de entrada/salida queda enteramente definida por:

- el sentido de la transferencia por realizar (entrada o salida)
- zona de almacenamiento en memoria de las informaciones por transferir. Esta zona puede delimitarse mediante la dirección de su primera palabra (DIR) y su número de palabras (CDP)

La unidad central es quien inicializa la operación por programa. Selecciona el periférico (indicando eventualmente el sentido de la transferencia por efectuar) y carga las informaciones que definen a la zona de almacenamiento en memoria en dos registros del canal, a los que llamaremos DEC (dirección en curso) para la dirección de la zona y CDP (cuenta de palabras), para el número de palabras.

Una vez efectuada la inicialización, la operación de entrada/salida se desarrolla ya al ritmo de la unidad periférica. Es a petición de ésta que el canal ejecuta una transferencia elemental, utilizando el contenido del registro DEC como dirección de almacenamiento en memoria, y después actualiza la dirección en curso, sumando 1 al registro DEC, y la cuenta de palabras, restando 1 al registro CDP.

La transferencia termina cuando el canal detecta que el registro contador de palabras llega a cero o cuando el controlador de periférico envía una señal de fin de información. En este momento

genera el canal una interrupción para indicar a la unidad central que ha concluido la operación de entrada/salida.

Tal modo de funcionamiento, soportado por dos registros, uno para la dirección en curso y otro para la cuenta de palabras, es designado generalmente como modo canal. A la correspondiente unidad de intercambio se le llama canal, por excelencia.

“El canal toma el gobierno por sí mismo de la transferencias de una o más palabras contiguas en memoria.”

“La operación de entrada/salida tiene que proveer el sentido (entrada o salida), la dirección de la primer palabra y la cantidad de palabras. No necesita proveer condición para finalizar porque finaliza cuando termina la cantidad de palabras a transferir ($CDP=0$) o cuando el periférico por una excepción manda fin de transmisión (la señal fin de información).”

. Aclaración: Para realizar una operación de salida SAL; el sentido (salida) va a la unidad de control para que esta sepa cómo desarrollar la instrucción, la dirección de la primer palabra se carga en DEC y la cantidad de palabras en CDP. Luego se activa la señal “inicio de operación”, se manda el sentido por “orden direcciones” y también se activa la señal “DEM” para empezar a transmitir. Después, se lee la palabra de memoria que está en la dirección de DEC (por medio de activar las señales SDEC y ENS) y pasa al registro M. A continuación, activando las señales SRM y ENT (entrada al registro tampón) pasa la palabra de memoria que se encontraba en el registro M al registro T.

En esta etapa, la computadora puede seguir con el programa en curso e incluso ejecutando varios ciclos de memoria o distintas instrucciones. Leyó la primer palabra y la almacenó en el registro tampón. En este momento, ya es responsabilidad del canal ver como transmite la palabra del registro T al medio exterior (al ritmo del periférico). Cuando el periférico se encuentre libre se produce la señal SRT con una aceptación de demanda ADEM y se transmite la palabra del registro T al periférico. Además, cuando se produce esta transferencia, se incrementa la dirección de DEC y se decrementa CDP produciéndose así, una lectura de la nueva dirección que se encuentra en DEC y dejará almacenada una nueva palabra en el registro T.

Es importante aclarar que mientras el periférico no esté libre la palabra quedará en el registro T todo el tiempo necesario hasta que este se libere, pero con la gran ventaja de que la computadora puede estar ejecutando sus instrucciones normalmente, es decir, queda bloqueado el canal pero no la CPU.

Toda vez que se produce un incremento de la dirección de DEC y un decremento de CDP se produce una señal de PCM (petición de ciclo de memoria).

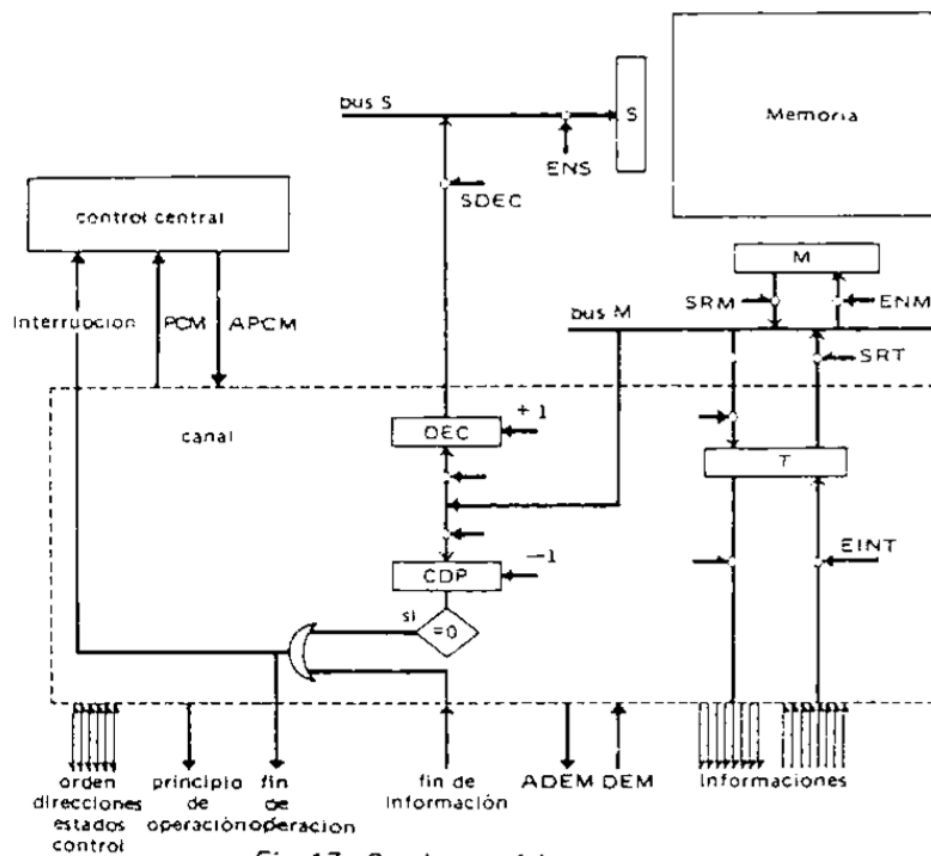


Fig. 17. Canal automático para Abacus

3. Encadenamiento de Datos

En el canal automático, para pasar de una zona de memoria a otra, la unidad central debía desarrollar, después de la interrupción de fin de transferencia correspondiente a la primera zona, un programa cuyo objetivo sería recargar los registros DEC y CDP con las características de la segunda zona y relanzar la operación de entrada o de salida.

El encadenamiento de datos consiste en efectuar esta operación de carga de dichos registros bajo control del canal, de tal suerte que se pase automáticamente de una zona a otra dentro de una operación de entrada – salida, sin implicar a la unidad central. Para conseguirlo se agregan al final de la zona en curso, las nuevas palabras de control y direcciones características de la zona siguiente, en el caso de que hubiera encadenamiento, y suponiendo, además que las informaciones de control caben en dos palabras de máquina.

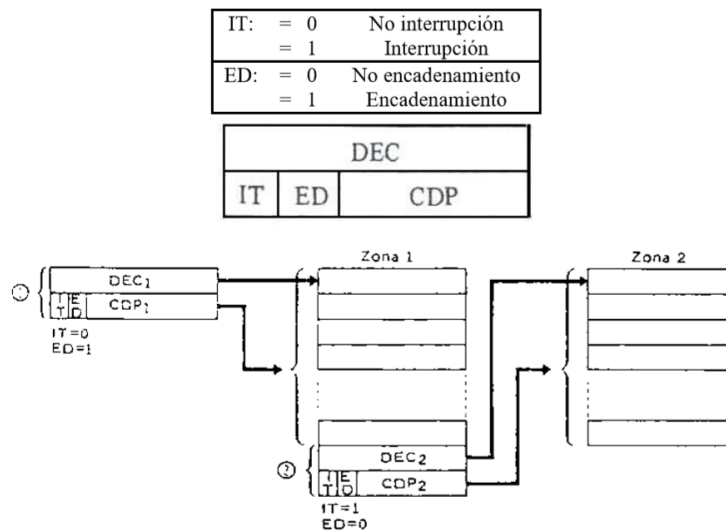


Fig. 20. Ejemplo de encadenamiento de datos

La operación de entrada/salida definida por las zonas de la figura se descompone según el siguiente esquema:

Nivel de la UC	Nivel del Canal
. Cargar la doble palabra de control 1 en los registros de canal . Iniciar la transferencia	. Transferir la primera zona . Cargar las palabras de control 2, puesto que el encadenamiento está pedido en 1 . Transferir la segunda zona . Generar una interrupción pedida en 2

4. Programa de Canal

El canal de los calculadores evolucionados es una unidad programable, capaz de leer, de codificar y ejecutar un programa de gobierno de entrada/salida o programa de canal, registrado en memoria central, con los mismos derechos que la unidad central lee, decodifica y ejecuta un programa de procesamiento.

“El programa canal está compuesto de tres grandes órganos: la memoria, el canal y el controlador. La memoria se encuentra dividida por 3 partes: programa unidad central, programa de canal y zona de E/S. Dentro de la zona de programa unidad central están las distintas operaciones sum, res, e tc, cuando llega una operación de transferencia, a diferencia del canal automático (indicaba dirección de inicio y cantidad de palabras), en el programa canal la instrucción de transferencia de la unidad central de proceso provee una indicación de inicio de transferencia, la dirección del periférico y la dirección de la primer línea del programa canal que se va a encargar de transferir.

La dirección del periférico va a ir a un decodificador (dentro del canal) que habilitará, por medio del controlador, el periférico deseado. La dirección del programa de E/S irá a un registro del canal llamado dirección de instrucción de canal que puede incrementarse para poder acceder a cada una de las líneas del programa canal que se está ejecutando.

En el programa canal cada palabra tiene:

- *Código de Operación de Periférico (COP): que le manda al periférico. Una orden específica.*
- *Dirección. Será la dirección transmitida a dirección en curso.*
- *Cantidad de palabras a transmitir*

- 3 banderas de control: Si hay o no interrupción, si hay o no encadenamiento de datos y si hay o no encadenamiento de gobierno (el programa también puede estar fragmentado)

Además, sigue teniendo dirección en curso y cuenta de palabras y están referidas a la zona de memoria de E/S, donde están los bloques de memoria que se van a transferir. El registro “información” dentro de canal es el tampón, que es el encargado de mantener la información hasta que el controlador le permita transmitir la información.

Una **transferencia elemental** es una transferencia de un dato de la memoria hacia el periférico o del periférico hacia la memoria. La transferencia elemental va a estar controlada por el controlador del periférico; es el quien marca la necesidad de leer o grabar una palabra de memoria. Ese gobierno es parte del controlador. Por cada transferencia de información se actualiza la dirección en curso y la cuenta de palabras.”

Encadenamiento de Gobierno (EG)

“Cuando hay encadenamiento de gobierno significa que las palabras del programa canal dejaron de ser contiguas y tiene que ir a actualizar el registro dirección de instrucción de canal para continuar con el programa de gobierno. En ese momento, suspende la transferencia de datos momentáneamente porque tiene que pasar a actualizar la dirección del programa canal a una nueva dirección. Para hacer esto, una palabra completa del programa canal va a estar destinada a actualizar la nueva dirección en el registro dirección de instrucción de canal. Esto implica que la información que va a venir a continuación en el programa canal se va a perder, no va ser de transferencia, sino que va a ser de actualización de la dirección de instrucción de canal.

En el caso que haya encadenamiento de gobierno hay que volver replantear cual es la dirección de la nueva instrucción de canal (la dirección de la palabra a ejecutar del programa canal). Esa dirección es la palabra que se va a obtener cuando el contador de dirección de instrucción de canal haga un +1 en el programa canal, y cuando aumente +2 va a ser la dirección de la palabra en el programa canal a partir de la cual tiene que tomar el encadenamiento. En otras palabras, la dirección de instrucción de canal se tiene que incrementar en 2 cuando hay encadenamiento de gobierno: tendrá que hacer una transferencia y después la búsqueda de la próxima instrucción de programa canal.”

Funcionamiento

La **operación de una entrada/salida** consiste en:

1. Durante la ejecución del programa de la unidad central, se genera en memoria una lista de instrucciones de canal conocido como **programa de gobierno de E/S**, o también llamado **programa canal**.
2. La unidad central ejecuta una instrucción de entrada/salida que proporciona al canal la dirección del órgano periférico interesado y la de la primera instrucción del programa canal. Si el órgano periférico designado está libre, se le pone en actividad (si está ocupado la unidad central es avisada de que el programa de canal no debe arrancar). La dirección de la primera instrucción de canal es enviada y almacenada en el contador de instrucciones del canal. A partir de este instante, el programa de canal se desarrollará sin intervención de la unidad central.
3. El canal va a buscar en memoria la primera instrucción de canal, que consta de los siguientes elementos:

C. O. Periférica	Dirección 1ª palabra	Cuenta de palabras	Indicadores
------------------	----------------------	--------------------	-------------

El canal transmite al controlador del periférico el código de operación que le es destinado. En este momento, la operación de entrada/salida queda inicializada y un proceso idéntico se desarrolla cada vez que el canal pide una nueva instrucción de canal.

4. Las demandas de transferencia elemental de información, una vez inicializada la operación de entrada /salida, emanan del controlador de periférico, que trabaja a su propio ritmo. Al recibir una de estas demandas, el canal efectúa la transferencia pedida utilizando y actualizando las informaciones dirección en curso (que sustituye a dirección de la primera palabra) y cuenta de palabras.
5. Una transferencia se terminará cuando sea nulo el contenido del contador de palabras o bien cuando se reciba una señal de fin de transferencia procedente del controlador de periférico. El canal comprobará entonces los tres indicadores para saber lo que ha de hacer:
 - a. **Encadenamiento de datos:** si este indicador está puesto, el canal pasará a la siguiente instrucción de canal, ignorando el nuevo código de operación periférica.
 - b. **Encadenamiento de gobierno:** si el indicador está puesto, el canal pasará a la siguiente instrucción de canal, utilizando el nuevo COP.
 - c. **Interrupción:** se generará una interrupción al final de la transferencia, si el correspondiente indicador está activado.

El programa de canal **está controlado** por un cierto número de **instrucciones de entrada/salida**, entre las cuales he aquí las más importantes:

Instrucciones de E/S	
Inicio de E/S	Permite inicializar una operación de E/S, señalando el periférico y la dirección de la primera instrucción de canal.
Detención de E/S	Permite detener una operación de E/S, señalando el periférico y la dirección de la primera instrucción de canal.
Prueba de E/S	Se emplea para obtener informaciones acerca del estado actual de una operación de E/S, sin alterar el funcionamiento de esta.
Prueba de órgano periférico	Permite obtener informaciones, más precisas que con la instrucción de prueba de E/S, acerca del estado de un órgano periférico; tales informaciones son suministradas por el controlador del periférico.
Identificación de una interrupción	Esta instrucción permite, tras recibirse una interrupción emitida por el canal, identificar al órgano provocador de la interrupción y conocer la causa de ella. La información de estado devuelta por el canal comprende, por una parte, la dirección del periférico causante de la interrupción, por otra parte, indicadores concernientes a la operación que ha dado lugar a la interrupción. Por ejemplo, desbordamiento de datos, contador de palabras nulo, final de transferencia, longitud incorrecta, etc.

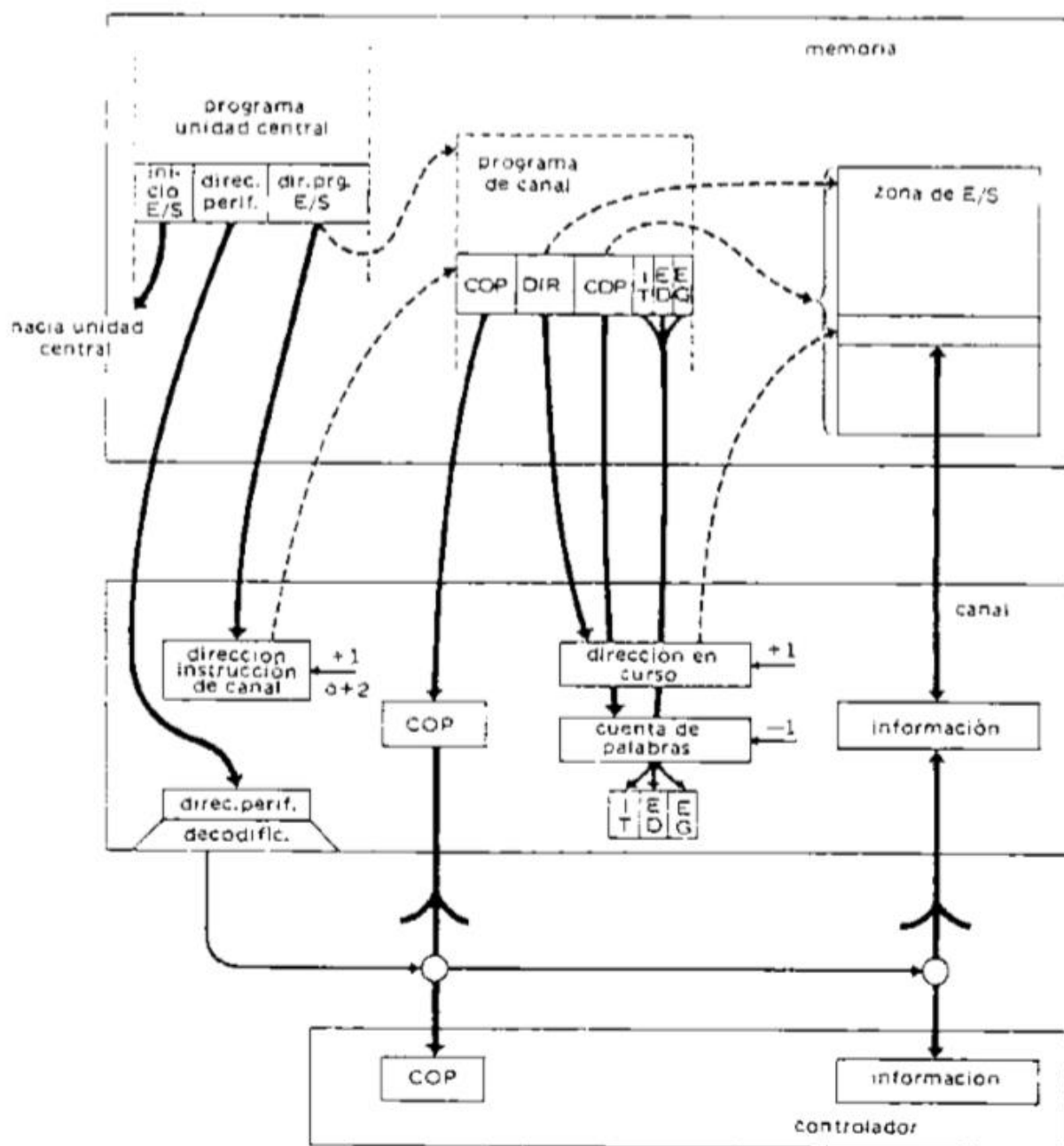


Fig. 21. El programa de canal

Unidad 14

Dispositivos de Entrada Salida

PERIFÉRICOS

Se denominan **periféricos** tanto a las unidades o dispositivos a través de los cuales el ordenador se comunica con el mundo exterior, como a los sistemas que almacenan o archivan la información, sirviendo de memoria auxiliar de la memoria principal.

Se entenderá por **periférico** a todo conjunto de dispositivos que, sin pertenecer al núcleo fundamental de la CPU-Memoria Central, permitan realizar operaciones de E/S, complementarias al proceso de datos que realiza la CPU.

Los periféricos se conectan a la CPU a través de grupos de hilos que se conoce como **buses**. El bus que conecta la CPU con los otros elementos del procesador se conoce como **bus local** o bus de la CPU. Las conexiones entre los periféricos y los controladores se realizan a través de un bus más general llamado **bus del sistema**. Algunos periféricos requieren un bus especializado que se adapte a su velocidad de transferencia, sus niveles de tensión, la naturaleza de sus señales de control y otros requerimientos. A estos buses se les llama **bus de entrada/salida** o bus de expansión.

UNIDADES DE ENTRADA

TECLADO

El **teclado** es básicamente un hardware periférico de entrada que provee de una serie de interruptores, llamados teclas, dispuestos en una determinada distribución, adecuada a diferentes lenguajes y modos de escritura. Estas teclas, que pueden alcanzar hasta las 108, pueden ser mecánicas o electrónicas, y su principal función es la permitirnos ingresar texto u órdenes a la computadora.

El **teclado** es un periférico de entrada inspirado en una máquina de escribir que posee teclas en una disposición denominada QWERTY y que se basa en la idea de que las teclas más utilizadas (en inglés) estén lo más separadas posibles entre sí.

Funcionamiento

Los teclados poseen internamente una matriz de teclas y un circuito integrado que se encarga de supervisar esta matriz y detectar las pulsaciones. Cuando se detecta una pulsación, o la liberación de una tecla, se escribe su código en un buffer interno del teclado y a continuación se sigue un proceso hasta que su código queda almacenado en memoria, y de forma básica es el siguiente:

1. El circuito del teclado detecta la pulsación. Genera el código de muestreo (llamado scan code) de la tecla pulsada y lo envía al puerto del teclado.
2. Se informa al procesador del suceso.
3. El procesador accede a la dirección de memoria donde se guarda el código de la tecla pulsada y la lee.
4. Verifica si hay alguna tecla de control pulsada: mayus, ctrl, alt, etc.
5. Traduce y guarda el código de la tecla pulsada hasta que el software solicite la entrada del teclado.

Cuando se suelta la tecla el circuito del teclado genera un código de tecla soltada (break code). El carácter que aparezca, se corresponde con la tecla pulsada y depende del idioma utilizado (página de códigos) en el sistema operativo.

El sistema de transmisión del teclado es bidireccional, controlado por un reloj, en el que se mueven bloques de datos de 11 bits de longitud, que se corresponden con la siguiente secuencia: 1 bit de inicio, 8 bits de datos que se transmiten, 1 bit de paridad y 1 bit de parada.

Composición

Según las normas ANSI (American National Standards Institute) los teclados deben contener los siguientes tipos de letras:

- **Teclado principal:** Caracteres alfabéticos, numéricos y especiales, como una máquina de escribir más o menos.
- **Teclas de gestión de imagen:** sobre la pantalla aparece un “cursor” que indica donde va a aparecer el siguiente carácter a teclear, pues bien, deben haber unas teclas que nos permitan mover y situar este cursor en la posición que nosotros elijamos. Estas teclas también debe permitirnos borrar un carácter o línea completa.
- **Teclado numérico:** son teclas que contienen los caracteres numéricos y operaciones básicas sobre ellos (+, -, *, /), además del punto decimal.
- **Teclas de funciones:** son teclas cuya función puede ser predefinida por el usuario mediante los programas que este desarrolle o lo están por los programas que estamos utilizando en cada momento. Son de F1 a F12. Hoy en día existen además múltiples teclas programables por los programas o el usuario.
- **Teclas de funciones locales:** controlan funciones propias del terminal, como la impresión del contenido de la pantalla, teclas de escape, avance de página, etc.

Tecnología de Teclado

Desde el punto de vista del funcionamiento de las teclas en sí, existen dos tipos fundamentales de teclado: **teclado de contacto** y **teclado capacitivo**.

El *teclado de contacto* puede ser de dos tipos:

- **Mecánico:** usa pequeños interruptores individuales para cada tecla. Cuando se presiona una tecla se cierra el interruptor y permite el paso de corriente. Tiene el inconveniente de ser bastante vulnerable a la suciedad, de forma que después de un tiempo de uso es posible que alguna de sus piezas empiece a fallar; este teclado se nota porque al pulsar una tecla, suena el tic del pulsador.
- **Membrana:** las teclas llevan una membrana de goma que al pulsar hace que se unan dos pistas conductoras en la parte inferior y permite el paso de corriente: son similares a las que llevan las calculadoras. Estas membranas pueden ser individuales para cada tecla o bien una lámina completa con membranas que se mueven con cada tecla. Al pulsar una tecla, no suena como el mecánico.

El *teclado capacitivo* es de más capacidad y más caro que el de contacto, teniendo una vida de uso bastante más larga. Éste teclado está construido sobre una tarjeta de circuito impreso grabada, de tal forma que cuando se pulsa una tecla, esta hace presión sobre un condensador, el cual produce una señal eléctrica que es detectada e interpretada por el procesador de teclado.

Tipos de Teclado

- **Teclado Ergonómico:** Se basan en el principio que dividiendo el teclado principal colocando en ángulo cada una de las mitades, los codos descansan en una posición mucho más natural, y cambiando la curvatura del teclado y añadiendo un pequeño “reposamuñecas”, el ángulo de escritura es mucho más cómodo para el usuario. Pero tienen una desventaja, y es que hace falta acostumbrarse a una disposición de teclas muy diferente.
- **Teclado Programable:** El propio teclado lleva un microcontrolador interno (que se comunica con la controladora de teclado de la placa base) con un programa integrado que interpreta las señales producidas al cerrarse el circuito cuando dos terminales (tecla y circuito integrado) entran en contacto. Este programa reside en una ROM, la cual puede almacenar muchos otros datos, además del código del teclado (país) y la posición de las teclas, pero para interpretarlos, se ha de instalar un driver o controlador del dispositivo que interprete las señales.
- **Teclados inalámbricos:** Caracterizados por la ausencia de cable. Los primeros utilizaban infrarrojo actualmente utilizan la radiofrecuencia. Su gran virtud es que no necesitamos tener una línea visual entre el teclado y el sensor, lo que nos da una extraordinaria movilidad. El gran problema de estos dispositivos ha sido siempre el gasto de pilas. La controladora no reside en el propio teclado, sino en el receptor que se conecta al conector de teclado en el PC.

RATON

El **mouse** es uno de los periféricos de entrada que forman parte de un computador, a través de él se puede interactuar directamente con la computadora mediante un puntero (indicador) que se muestra en la pantalla. Básicamente es un dispositivo de hardware cuya función principal es controlar las funciones que le ordena el usuario, debido a que gracias al complemento de su puntero en la pantalla puede desplazarse a través de ella y por medio de un botón puede seleccionar cualquiera de las aplicaciones que estén contenidas en el ordenador.

La función principal del ratón es transmitir los movimientos de nuestra mano sobre una superficie plana hacia el ordenador. Allí, el software denominado driver se encarga realmente de transformarlo a un movimiento del puntero por la pantalla dependiendo de varios parámetros. En el momento de activar el ratón, se asocia su posición con la del cursor en la pantalla. Si desplazamos sobre una superficie el ratón, el cursor seguirá dichos movimientos.

Existen diferentes **tipos** de ratones:

- **Mecánicos:** Son sencillos y tienen un bajo coste. Se basan en una bola de silicona que gira en la parte inferior del ratón a medida que desplazábamos éste. Dicha bola hace contacto con dos rodillos, uno perpendicular al ratón y otro transversal, de forma que uno recoge los movimientos de la bola en sentido horizontal y el otro en sentido vertical. En cada extremo de los ejes donde están situados los rodillos, existe una pequeña rueda conocida como “codificador”, que gira en torno a cada rodillo. Estas ruedas poseen en su superficie, y a modo de radios, una serie de contactos de metal, que a medida que gira la rueda toca con dos pequeñas barras fijas conectadas al circuito integrado en el ratón. Cada vez que se produce contacto entre el material conductor de la rueda y las barras, se origina una señal eléctrica. Así, el número de señales indicará la cantidad de puntos que han pasado éstas, lo que implica que, a mayor número de señales, mayor distancia habrá recorrido el ratón. Tras convertir el movimiento en señales eléctricas, se enviaban al software del ordenador por medio del cable.
- **Ratones Ópticos:** Los ratones ópticos carecen de bola y rodillos, y poseen unos foto-sensores o sensores ópticos que detectan los cambios en los patrones de la superficie por

la que se mueve el ratón. Estos ratones ópticos disponen de un LED que ilumina la superficie sobre la que se mueve el ratón. Una microcámara se encarga de digitalizar con un CCD una pequeña región y luego un circuito integrado compara la imagen producida con la imagen anterior para decidir hacia donde se está desplazando el ratón.

- **Ratones Inalámbricos:** en este caso el dispositivo carece de un cable que lo comunique con la computadora, sino que utiliza algún tipo de tecnología inalámbrica. Para ello requiere un receptor de la señal inalámbrica que produce, mediante baterías, el *mouse*. El receptor normalmente se conecta a la computadora por USB, o por PS/2. La tecnología inalámbrica usada puede ser:
 - **Radio Frecuencia (RF):** Es el tipo más común y económico de este tipo de tecnologías. Funciona enviando una señal a una frecuencia de 2.4Ghz, popular en la telefonía móvil o celular.
 - **Infrarrojo (IR):** Esta tecnología utiliza una señal de onda infrarroja como medio de transmisión de datos, popular también entre los controles o mandos remotos de televisiones.
 - **Bluetooth (BT):** Bluetooth es la tecnología más reciente como transmisión inalámbrica (estándar IEEE 802.15.1), que cuenta con cierto éxito en otros dispositivos.

UNIDADES DE SALIDA

MONITORES

El **monitor** es un dispositivo electrónico de salida (puede considerarse entrada/salida si es táctil) de la computadora en el que se muestran las imágenes y textos generados por medio de un adaptador gráfico o de video de ésta. El término monitor se refiere normalmente a la pantalla de vídeo, y su función principal y única es la de permitir al usuario interactuar con la computadora. El monitor es una de las partes más importantes de la computadora, ya que es el primer medio por el que el usuario conoce los resultados de su trabajo. El monitor se conecta a la tarjeta gráfica, estando dicha tarjeta integrada en la placa base, o conectada a un bus de expansión.

TIPOS DE MONITORES

- **Monitor CRT:** Un monitor CRT tiene un tubo de rayos catódicos en la parte trasera. La electricidad causa que el rayo catódico libere electrones, los cuales aceleran y golpean la pantalla. A medida que los electrones chocan con la pantalla, que está hecha con fósforo con frecuencia, la misma fluoresce para crear una imagen.

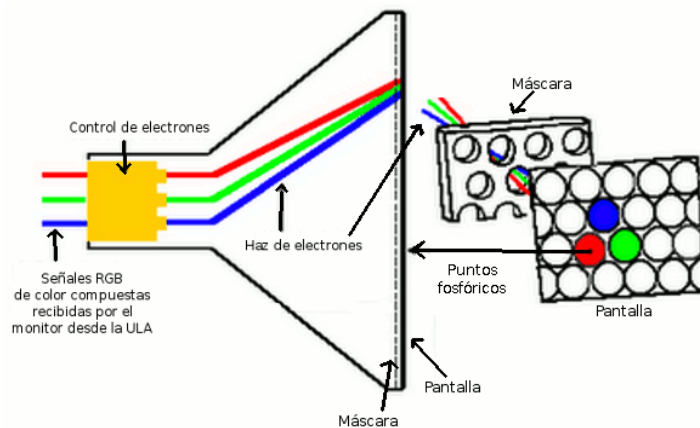
La interacción entre los electrones y el fósforo genera tres colores: rojo, verde y azul. El monitor CRT utiliza estos colores en distintas combinaciones para formar todos los otros colores necesarios. Es por esto que el cable de monitor que se conecta con la computadora es a veces llamado el cable RGB.

El monitor CRT utiliza información enviada desde la computadora para crear imágenes una por una, esencialmente "refrescando" la pantalla un cierto número de veces por segundo. Cuanto más alto sea el número de refrescos que hace la pantalla por segundo, más suaves serán las imágenes en la pantalla. Una tasa de refresco baja causa el efecto titilante que se ve comúnmente en viejos monitores CRT y televisores.

Características:

- Excelente calidad de imagen (definición, contraste, luminosidad).
- Económico.
- Tecnología robusta.
- Resolución de alta calidad.
- Presenta parpadeo por el refrescado de imagen.

- Consumo de energía.
- Generación de calor.
- Generación de Radiaciones eléctricas y magnéticas.
- Alto peso y tamaño.



- **Pantallas LCD:** A este tipo de tecnología se le conoce por el nombre de pantalla o display LCD, sus siglas en inglés significan "Liquid Crystal Display". Características:
 - Poco peso y tamaño.
 - Buena calidad de colores.
 - No contiene parpadeo.
 - Poco consumo de energía.
 - Poca generación de calor.
 - No genera radiaciones eléctricas y magnéticas.
 - Alto costo.
 - Angulo limitado de visibilidad.
 - Brillo limitado.
 - Bajo tiempo de respuesta de píxeles.
 - Contiene Mercurio.

IMPRESORAS

La **impresora** es un periférico del ordenador, que está conectado a la unidad central de procesamiento de una computadora, su función es hacer una copia física de aquellos documentos que han sido almacenados en un formato electrónico. Estos documentos pueden ser textos o imágenes que son impresos en una hoja o transparencia utilizando cartuchos de tintas o tecnología láser. Son diseñadas para realizar trabajos repetitivos de poco volumen. Sin embargo, las impresoras son generalmente dispositivos lentos y el gasto por página es relativamente alto. La mayoría de las impresoras, están unidas a un ordenador por medio de un cable, aunque también existen las impresoras de red, las cuales poseen una interfaz de red interna, que le permite a cualquier usuario de la red imprimir documentos.

Algunas de las características que tenemos de las impresoras son:

- Velocidad de impresión: paginas por minutos (ppm) o en caracteres por segundos (cps),
- Resolución: se refiere a la calidad de la impresión, es decir, cantidad de puntos (píxeles) que la impresora puede crear sobre el papel
- Buffer de memoria (Zona de almacenamiento temporal de datos en la impresora)
- Interfaz de conexión
- Cartuchos

- Memoria integrada
- Tipo de papel que utiliza

TIPOS DE IMPRESORAS

1. Impresora de Agujas

También llamadas matriciales, utilizan una serie de agujas o pines dispuestos en forma vertical, que se disparan mediante electroimanes, impactando en una cinta entintada que se encuentra en la aguja y el papel, forma un punto sobre el papel. Así, los caracteres están formados por puntos creados por el choque de un número determinado de agujas cada vez. El número de puntos que constituye un carácter (o el número de agujas) determina la resolución, y por tanto, la calidad de impresión, siendo las más comunes las de 9, 18 y 24 agujas.



Funcionamiento

1. La computadora envía los caracteres a imprimir por el puerto de la impresora.
2. Los caracteres son almacenados en el buffer de la impresora.
3. El procesador de la impresora reconoce cada uno de los caracteres enviados gracias a una memoria ROM en la que están grabadas qué agujas son necesarias accionar para formar el carácter.
4. El cabezal se sitúa en la posición adecuada y el procesador envía una señal eléctrica a las agujas que deben accionarse. Cada aguja en su extremo está compuesta de un pequeño martillo que en presencia de tensión golpea a la aguja correspondiente gracias a un electroimán.
5. La aguja golpea sobre una cinta impregnada en tinta, provocando la transferencia de la tinta al papel. Después, la aguja vuelve a su posición original y el cabezal se encuentra listo para realizar la siguiente ráfaga.
6. El cabezal irá accionando distintas combinaciones de agujas a medida que se desplaza por la página para ir representando los distintos caracteres.
7. Una vez el cabezal ha terminado de imprimir una línea, el tambor (rodillo) que arrastra el papel girará un poco subiendo el papel para que sea impresa una nueva línea.

Ventajas:

- Pueden trabajar con papel continuo en lugar de requerir hojas individuales.
- Conforme se termina la tinta, la impresión pierde intensidad gradualmente en lugar de terminar repentinamente durante un trabajo.
- Bajo costo de mantenimiento (el costo de la cinta entintada es insignificante comparado con el tóner de una láser o los cartuchos de tinta de inyección).
- Fiables
- Conveniente para grandes volúmenes de impresión que no necesiten tanta calidad.

Desventajas:

- Baja velocidad
- Son muy ruidosas (debido al sonido de las agujas que golpean el papel)

- Calidad inferior comparada con otras del mercado.

2. Impresora de Inyección a tinta

Imprime línea a línea con la ayuda de un cabezal de impresión móvil que se desplaza (junto con los cartuchos) por todo el ancho de la hoja, de forma parecida a como lo hace la de agujas, pero a diferencia de esta usa un sistema de no impacto, por el cual el cabezal no toca en ningún momento la hoja de papel, sino que pulveriza pequeños chorros de tinta sobre el mismo. Para esto, el cabezal de impresión tiene un cierto número de pequeños depósitos llenos de tinta, cada uno con un orificio (inyector), de un grosor inferior al de un cabello, por los que se expulsa la tinta formando minúsculos puntos sobre el papel. Según el dato a imprimir se expulsará la tinta por unos orificios u otros.

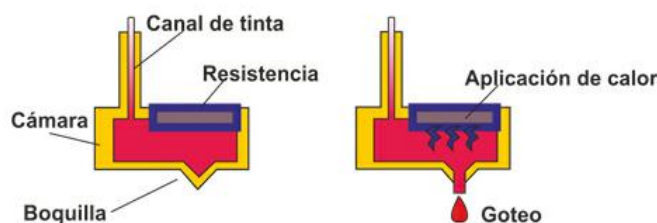
Características:

- Bastante silenciosa
- Mucha mejor calidad y mayor velocidad que las matriciales
- Bajo precio
- Más rápida que las de aguja, aunque más lenta que las láser.

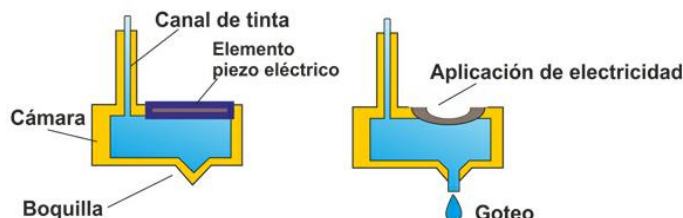
Existen 2 métodos para inyectar la tinta desde el cabezal:

a. Modo Térmico:

- I. En el interior de cada depósito existe una resistencia, a través de la cual se pasa una pequeña corriente eléctrica durante varias millonésimas de segundo. Esto provoca que una fina capa de tinta en el fondo de la cámara se caliente a más de 450°C.
- II. La tinta comienza a hervir y se forma una burbuja de vapor que va expandiéndose.
- III. Al expandirse, la burbuja de vapor empuja la tinta a través del inyector hasta formar una minúscula gota en su extremo.
- IV. La minúscula gota, gracias a la presión de la burbuja de vapor, cae sobre la hoja de papel formando un pequeñísimo punto del color de la tinta.
- V. La resistencia comienza a enfriarse de forma que la burbuja de vapor va desapareciendo (o explota). Como resultado se produce una succión que llena el depósito de tinta nueva procedente del cartucho, encontrándose preparado para una nueva gotita.



b. Método Piezoeléctrico: en la reserva de tinta de cada inyector existe un elemento piezoeléctrico (cristal que se deforma con la corriente eléctrica). El cristal recibe una pequeña carga eléctrica que hace que vibre. Cuando el cristal vibra para adentro, fuerza una pequeña cantidad de tinta hacia afuera del inyector. Cuando vibra para afuera, estira algo más de tinta hacia la reserva, reponiendo la tinta que salió.



Existen de dos tipos: las que llevan los *inyectores fijos en la propia impresora* y las que llevan los *inyectores en los propios cartuchos*. Los inyectores fijos son de mayor calidad, y están fabricados para un tiempo de vida de la impresora bastante alto, mientras que los que van en los cartuchos están pensados para la duración del cartucho, ya que al cambiar el cartucho se cambian los inyectores.

3. Impresora Láser

La impresora láser se creó en la década de los 70 y desde la época hasta ahora han evolucionado para consolidarse como una de las impresoras más rápida, silenciosa y eficiente, tanto así que en pocos segundos pueden imprimirse textos, gráficos e imágenes de gran calidad.

El dispositivo central que utiliza este tipo de impresión es un material fotosensible que se descarga con luz, denominado **cilindro fotorreceptor**.

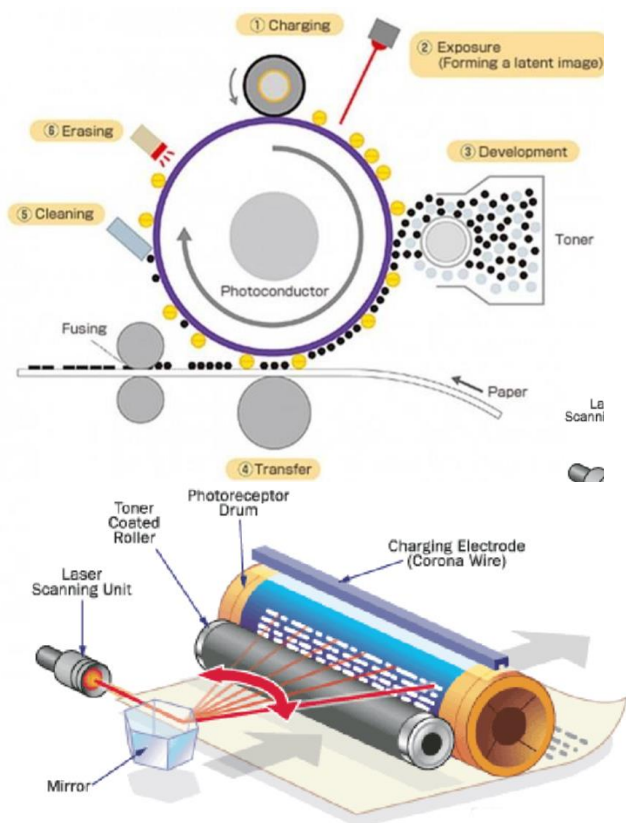
Cuando es enviado un documento a la impresora, todo lo que se desee imprimir (caracteres, números, imágenes) va a ser dibujado sobre el cilindro con un láser. La carga del láser va a ionizar al cilindro y también le va a poner una carga electrostática. Este pequeño láser se encarga de cargar electrostáticamente diminutas partes del cilindro, con lo cual se forma la imagen electrostática (no visible) de nuestro documento a imprimir sobre este cilindro fotorreceptor. Posteriormente el cilindro es bañado por un polvo muy fino de color negro, el cual posee carga contraria a la del cilindro y por lo tanto es adherido a las partes que se encuentran con una carga distinta en el cilindro. Esto se debe a la ley de cargas, la cual enuncia que cargas iguales se repelen y cargas diferentes se atraen. Las partes cargadas con igual carga repelen este polvo llamado tóner (tinta seca) con lo cual queda formada la imagen visible sobre el tambor.

Hay que aclarar también, que el papel cuando ingresa al sistema se carga electrostáticamente con una carga distinta a la que tiene el cilindro. Por lo tanto, mientras el cilindro sigue girando y el pasa el papel por ahí se produce como una lluvia de polvo del cilindro al papel que dibuja sobre el papel el carácter, gráfico, etc. que se desea imprimir. Hasta este momento, el polvo se encuentra sobre el papel con el dibujo deseado pero no está adherido, es decir, se puede perder fácilmente el dibujo formado sobre la hoja.

A continuación, el tóner que se transfirió al papel es adherido a éste por medio de un par de rodillos, encargados de generar calor y presión sobre la hoja; a esta unidad se le denomina de fijado y es el paso final de la impresión láser. Para regresar al estado inicial, el tóner restante en el cilindro fotorreceptor es limpiado por medio de una lámina plástica y al mismo tiempo se incide luz sobre el cilindro para dejarlo completamente descargado para un nuevo proceso de impresión.

Características:

- Muy rápidas
- Muy silenciosas
- Altísima calidad de impresión, superior a las impresoras a tinta
- Reducción de costos: a pesar que el tóner es más costoso que los cartuchos de tinta de otras impresoras, el número de impresiones que pueden hacerse es mucho mayor, lo que se traduce en un valor por impresión mucho menor.
- Se necesita un cartucho de tóner por cada color. Así pues, habría que elegir entre una impresora capaz de imprimir en color aunque se pierda algo de resolución (inyección) o una de gran calidad (láser) pero resignarse a no imprimir en color.



UNIDADES DE MEMORIA MASIVA AUXILIAR

Se denomina **dispositivo de almacenamiento masivo** a las unidades de disco que almacenan los bits utilizando tecnología magnética u óptica, y que permiten el almacenamiento “masivo” de bits por su bajo costo. Estas memorias deben ser no volátiles, que en su mayoría se pueden leer y escribir, y reciben el nombre de **memorias externas, masivas o secundarias**. Si bien es necesario que esas memorias estén asociada a la CPU para acceder a ellas, el soporte de bits, por ejemplo, un CD, puede guardarse en un cajón sin que se pierda la información grabada en él.

Las **unidades de almacenamiento masivo** corresponden a aquellos dispositivos encargados de guardar la información permanente los datos y/o programas para ser utilizados en el momento adecuado y poder ser modificados, vueltos a guardar y recuperados cuando se desee. Son por lo tanto dispositivos que guardan permanentemente la información en ausencia de alimentación, siendo mucho de ellos capaces de ser transportables, es decir, poder llevar la información a otro equipo o guardarla como copia de seguridad.

Las *memorias secundarias* almacenan programas ejecutables y no ejecutables, base de datos, archivos de imágenes, de video y de toda información digital que se quiera almacenar en forma permanente. La CPU intercambia instrucciones y datos con distintos periféricos, como la unidad de disco rígido, una tarjeta de memoria o una unidad de DVD. Estos periféricos se denominan **dispositivos de E/S** y están conectados a la CPU en la mayoría de las computadoras, con un bus individual denominado bus de E/S que actúa de colector auxiliar desde el bus del sistema y hacia el mismo.

Las *memorias externas* son más lentas para acceder a su información en comparación con las de semiconductores, pero su capacidad de almacenamiento es notablemente mayor. Las *memorias externas* se incluyen en todas las computadoras debido a su carácter no volátil, o sea que son imprescindibles para retener información cuando el sistema está desconectado de la fuente de alimentación. Las *memorias externas* actúan de depósito y toda información que no se necesite para el proceso actual se almacena en ellas hasta que sea demandada.

En un principio la CPU obtiene un programa y los datos necesarios desde el almacenamiento secundario. El programa se copia literalmente desde su ubicación en el almacenamiento secundario a la memoria principal (DRAM) de la computadora, lo que permite ejecutarse con mayor rapidez. Además, la CPU solo tiene acceso directo a esta última por medio del bus de direcciones. De la misma forma, cualquier dato necesitado por el programa se copia desde el almacenamiento secundario hacia la memoria y esos datos retornaran al almacenamiento secundario si se los modifica y se quiere guardar los cambios en forma permanente. Los dispositivos de E/S más utilizados para “soportar” estas memorias son el disco rígido y los dispositivos de tecnología óptica, como la unidad de DVD.

1. Discos Magnéticos

Son los dispositivos de E/S de mayor uso en nuestros días. Los discos magnéticos pueden ser rígidos, dispuestos en unidades de cabezas fijas o móviles, en grupo de uno o más platos.

Un disco magnético es un plato circular construido con un material no magnético, llamada sustrato, cubierto por un material magnetizable. Tradicionalmente, el sustrato es aluminio o una aleación de aluminio. Recientemente, se han utilizado sustratos de cristal.

Los datos se graban y después se recuperan del disco a través de una **bobina**, llamada **cabeza**, en muchos sistemas, hay dos cabezas, una de lectura y otra de escritura. La cabeza es un dispositivo relativamente pequeño, capaz de leer o escribir en una zona del plato que rota bajo ella.

Esto da lugar a que los datos se organicen en un conjunto de anillos concéntricos en el plato, llamado **pistas**. Cada pista es del mismo ancho que la cabeza. Usualmente hay cientos de pistas por platos. Las pistas adyacentes están separadas por **bandas vacías**. Esto previene, o por lo menos minimiza, los errores debido a desalineamientos de la cabeza o simplemente a interferencias del campo magnético.

Los datos se transfieren al y desde el disco en **sectores**. Normalmente hay cientos de sectores por pista, y estos pueden tener una longitud variable o fija. En la mayoría de los sistemas de hoy se utilizan sectores de longitud fija (512 bytes).

En la mayoría de los discos, la cubierta magnetizable se aplica a ambas caras del plato.

Denominándose estos discos de **doble superficie**. Algunos discos, menos caros, son de **una superficie**.

Algunas unidades de discos poseen varios platos apilados verticalmente y separados por una distancia de alrededor de una pulgada. Disponen de varios brazos. Los discos de varios platos utilizan una cabeza que se mueve, con una cabeza de lectura-escritura por cada superficie del plato. El conjunto de todas las pistas que tienen la misma posición relativa en el plato se denomina **cilindro**.

Operación de Lectura y Escritura

Durante una *operación de lectura o escritura*, la cabeza permanece quieta mientras el plato rota bajo ella:

El **mecanismo de escritura** se basa en el hecho de que un flujo eléctrico atravesando una bobina crea un campo magnético. Se envían pulsos eléctricos a la cabeza de escritura, y se graban los patrones magnéticos en la superficie bajo ella, con patrones diferentes para corrientes positivas y negativas. La propia cabeza de escritura está hecha de un material fácilmente magnetizable y tiene forma rectangular con un agujero en un lado y varias vueltas de cable conductor a lo largo del lado opuesto. Una corriente eléctrica en el cable induce un campo magnético a lo largo del agujero,

que magnetiza una pequeña área del medio grabable. Si se cambia la dirección de la corriente, cambia el sentido de magnetización del medio de grabación.

El **mecanismo de lectura** se basa en el hecho de que un campo magnético en movimiento respecto a una bobina, induce una corriente eléctrica en la bobina. Cuando la superficie del disco pasa bajo la cabeza, en esta se genera una corriente de la misma polaridad que la de la que produjo la grabación magnética. La estructura de la cabeza de lectura es, en este caso, esencialmente la misma que la de escritura y, por tanto, se puede usar la misma cabeza para ambas operaciones.

Este mecanismo de lectura es muy antiguo. Actualmente, es necesario una cabeza de lectura separada posicionada, por conveniencia, cerca de la cabeza de escritura. La cabeza de lectura consiste en un sensor magnetoresistivo parcialmente blindado. Funciona con una resistencia eléctrica que depende de la dirección de la magnetización del medio que se mueve bajo él.

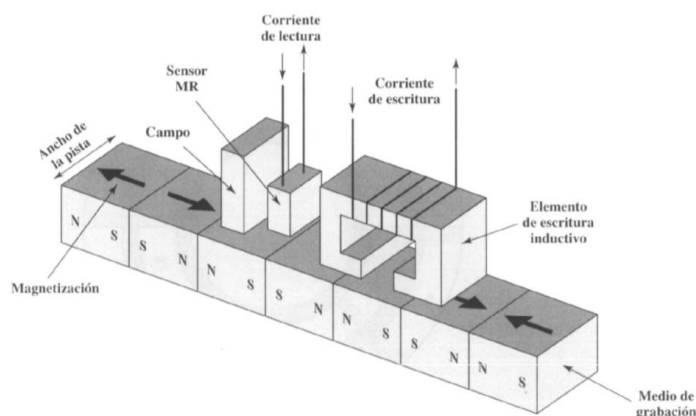


Figura 6.1. Cabeza de (escritura por inducción) / (lectura magnetoresistiva).



2. Discos Estado Sólido (SSD)

Una memoria de estado sólido es un dispositivo de almacenamiento secundario hecho con componentes electrónicos en estado sólido pensado para utilizarse como memoria auxiliar. La unidad de estado sólido, SSD (solid-state drive) es un tipo de dispositivo de almacenamiento de datos que utiliza memoria no volátil, como la memoria flash, para almacenar datos, en lugar de los platos o discos magnéticos de las unidades de discos duros (HDD) convencionales. Los SSD almacenan los archivos en microchips con memorias flash interconectadas entre sí.

En comparación con los discos duros tradicionales, las unidades de estado sólido son menos sensibles a los golpes al no tener partes móviles, son prácticamente inaudibles, y poseen un menor

tiempo de acceso y de latencia, lo que se traduce en una mejora del rendimiento exponencial en los tiempos de carga de los sistemas operativos.

En contrapartida, su vida útil es muy inferior, ya que tienen un número limitado de ciclos de escritura, pudiendo producirse la pérdida absoluta de los datos de forma inesperada e irre recuperable. Las SSD hacen uso de la misma interfaz SATA que los discos duros, por lo que son fácilmente intercambiables sin tener que recurrir a adaptadores o tarjetas de expansión para compatibilizarlos con el equipo.

PRINCIPALES VENTAJAS	SSD	HDD
CAPACIDAD	En general entre 256 GB y 4 TB	En general entre 1 y 10 TB
CONSUMO	Menor consumo	Mayor consumo
COSTE	Bastante más caros	Mucho más económicos
RUIDO	Más silencioso por no tener partes móviles	Algo más ruidoso por tener partes móviles
VIBRACIONES	No vibra por no tener partes móviles	El giro de sus discos puede provocar leves vibraciones
FRAGMENTACIÓN	No tiene	Puede darse
DURABILIDAD	Sus celdas pueden reescribirse un número limitado de veces	Con partes mecánicas que pueden dañarse con movimientos
TIEMPO DE ARRANQUE DE SO	7 segundos	16 segundos
TRANSFERENCIA DE DATOS	En general, entre 200 y 550 MB/s	En general entre 50 y 150 MB/s
AFECTADO POR EL MAGNETISMO	No	El magnetismo puede eliminar datos